

达尔文哥德尔机：自我改进智能体的开放式进化

Jenny Zhang^{*,1,2} Shengran Hu^{*,1,2,3} Cong Lu^{1,2,3} Robert Lange^{†,3} Jeff Clune^{†,1,2,4}

¹University of British Columbia ²Vector Institute ³Sakana AI ⁴Canada CIFAR AI Chair
{jennyzzt, srhu, conglu}@cs.ubc.ca, robert@sakana.ai, jeff.clune@ubc.ca

摘要

当今大多数人工智能系统受限于人类设计的固定架构，无法自主且持续地改进自身。相比之下，科学方法是一个累积且开放式的系统，每一项创新都建立在先前工件之上，从而促成未来的发现。人们日益期望当前推进人工智能的手动过程本身能够实现自动化。若安全实现，这种自动化将加速人工智能发展，使我们更早受益。这一前景引出一个问题：人工智能系统如何在不断改进自身的同时，更擅长解决相关问题。元学习可以自动发现新颖算法，但受限于一阶改进以及人类对合适搜索空间的设计。哥德尔机 (Schmidhuber, 2007) 提出了一种理论替代方案：一种自我改进的人工智能，以可证明有益的方式反复修改自身。遗憾的是，在实践中证明大多数更改具有净收益是不可能的。我们提出达尔文哥德尔机 (DGM)，一种新颖的自我改进系统，它迭代地修改自身代码（从而也提升其修改自身代码库的能力），并使用编码基准对每次更改进行经验验证。受达尔文进化与开放式研究的启发，DGM 培育一个生成式编码智能体档案。它从该档案中采样智能体，这些智能体自我修改以创建自身新颖有趣的版本。这种开放式探索形成一棵不断生长的多样化高质量智能体树，并允许并行探索搜索空间中的众多不同路径。经验上，DGM 自动提升其编码能力（例如，更好的代码编辑工具、长上下文窗口管理、同行评审机制），将 SWE-bench 上的性能从 20.0% 提升至 50.0%，Polyglot 上从 14.2% 提升至 30.7%。此外，DGM 显著优于无自我改进或开放式探索的基线。所有实验均在安全预防措施下进行（例如，沙箱、人类监督）。总体而言，DGM 代表着迈向自我改进人工智能的重要一步，能够沿着通往无尽创新的路径自行收集垫脚石。所有代码开源，地址为 <https://github.com/jennyzzt/dgm>。

1 引言

科学进步是累积且开放的，每一项突破都建立在无数先前洞见之上。同样，我们最先进的人工智能系统也建立在长期创新谱系之上。例如，变换器 (Vaswani et al., 2017) 作为当前大语言模型 (LLMs) (Brown et al., 2020) 的骨干，并非孤立出现，而是建立在多年过往创新之上，如循环神经网络 (Linnainmaa, 1970; Amari, 1972; Hopfield, 1982; Rumelhart et al., 1985) 和注意力机制 (Schmidhuber & Huber, 1990; Bahdanau et al., 2015; Kim et al., 2017; Parikh et al., 2016)。然而，当今大多数人工智能系统仍受限于固定、人工设计的架构，在预定义边界内学习，缺乏自主重写自身源代码以实现自我改进的能力。因此，人工智能发展的每一步进展仍严重依赖人类干预，制约了进步的速度。本文

* 共同第一作者 † co-senior authors

探讨了安全自动化搜索更优人工智能的诱人可能性。可以设想一种人工智能系统，如同科学发现本身，成为自身进步的引擎：基于自身过往，递归改进，并推动自身迈向更先进的能力。

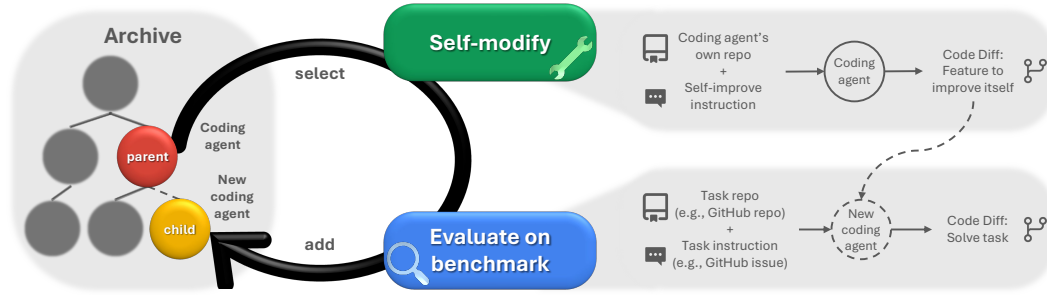


图 1：达尔文哥德尔机。DGM 通过将自我修改与下游任务评估交错进行，迭代构建不断增长的智能体档案。档案中的智能体通过开放式探索被选中进行自我修改。

施米德胡贝（Schmidhuber）于 2007 年提出了一类数学上严格、自指、自改进的问题求解器。该方法依赖形式化证明来论证代码重写的合理性，确保任何自我修改在可证明的意义上均有益处。然而，在实践中，若不对系统施加限制性假设，则无法形式化地证明对人工智能系统的某项修改是否有利。例如，尽管基于大语言模型的编码智能体似乎会因获得更多工具（如代码搜索、测试运行器）而受益，但实际影响在很大程度上取决于模型的训练与任务情境（例如，针对某一环境优化的测试工具在其他环境中使用时可能会使智能体产生困惑）。本文不要求形式化证明，而是依据基准对自我修改进行经验验证，使系统能够基于观测结果进行改进与探索。这一方法类似于生物进化：突变与适应并非事先验证，而是先产生、再试验，最终通过自然选择加以筛选。此外，本文还从达尔文进化论（Darwin, 2023）中获得启发，并考察维护一个由先前发现的智能体所构成的库的有效性，以作为未来世代的垫脚石。

我们提出达尔文哥德尔机（Darwin Gödel Machine, DGM），一种自指、自改进系统，通过编写和修改自身代码来成为更优秀的编码智能体。每次自我修改都要求 DGM 编辑其自身代码库。我们使用图灵完备的 Python，赋予 DGM 构建任何可计算机器的潜力。我们的框架设想智能体能够重写自身的训练脚本（包括训练新的基础模型（Foundation Model, FM））。然而，本文未展示这一点，因为训练 FM 计算密集且会引入大量额外复杂度，我们将其留作未来工作。相反，本文专注于改进使用冻结预训练 FM 的编码智能体设计（例如工具使用、工作流）。DGM 在自我修改阶段和评估阶段之间交替。在自我修改阶段，从档案中选出的编码智能体生成自身的修改版本。在评估阶段，每个修改后的智能体在编码基准上接受测试，以估计其编码能力，然后被加入档案。通过这一循环改进自身能力，DGM 在解决编码任务和进行未来自我改进方面都变得更加出色。一个关键假设是，编码基准上性能的提升表明编码能力更强，因此自我修改和自我改进的能力也更强。此外，DGM 维护一个生成编码智能体的档案，初始仅包含一个智能体，并随时间持续累积所有生成的变体。为支持持续自我改进，DGM 从开放性研究（Wang et al., 2019; Fernando et al., 2024; Faldor et al., 2025）中获得启发，积累多样化的垫脚石（即有趣但次优的解决方案或特征，可能促成未来的突破）。这种开放式探索鼓励发现新颖且潜在有用的自我修改，而不仅仅是眼前的性能提升。

我们在两个编码基准上展示结果：SWE-bench（Jimenez et al., 2024）和 Polyglot（Paul Gauthier, 2024）。DGM 在 SWE-bench 上自动将自身性能从 20.0% 提升至 50.0%，在 Polyglot 上从 14.2% 提升至 30.7%。我们表明自我改进能够实现持续进步，因为 DGM 优于基线，在基线中同一基础智能体被反复用于修改和

生成新智能体而无需自我改进。我们还表明，开放式探索和保留所有先前生成智能体的档案有助于发现更优秀的编码智能体。深度生成模型（DGM）优于没有开放式探索的基线（即没有积累有趣不同踏脚石档案的基线），后者中的编码智能体总是基于其最新版本进行构建。总体而言，深度生成模型（DGM）代表着向能够基于自身先前创新并递归改进的人工智能系统迈出的一步。我们广泛考虑并讨论了安全方面，包括沙盒化和自我修改的可追溯性，以确保负责任的实验（第 5 节）。通过推进安全、自指、自我改进模型的可能性，深度生成模型（DGM）使我们更接近一种不仅学习而且以开放式、自我加速的轨迹进化的人工智能，就像科学本身一样。

2 相关工作

开放式性。推动无界创新的重大挑战在于设计开放式人工智能系统，使其能够持续生成新颖且可学习的工件（Stanley et al., 2017）。Hughes et al. (2024) 将开放性定义为系统从观察者视角生成既新颖又可学习的工件序列的能力。核心难点在于构建和探索庞大的搜索空间，以持续产出对人类有吸引力的工件（Clune, 2019; Jiang et al., 2023）。早期进展借鉴了质量多样性算法、目标导向探索、内在动机和学习进展框架（Pugh et al., 2016; Ecoffet et al., 2019; Lehman & Stanley, 2011; Oudeyer et al., 2007），而近期进展则利用大规模基础模型（FMs）作为人类兴趣的代理，并作为跨领域生成和评估新颖行为的通用引擎（Brown et al., 2020; Hu et al., 2025; Zhang et al., 2024b）。然而，这些方法尚未闭合自我指涉的自我改进循环，即下游任务的改进尚未转化为自我修改能力的增强或进一步创新的加速。我们旨在模拟科学技术的加速发展，其中新工具和新发现催化更多发现的产生。我们如何模拟自然的进化轨迹，它不仅趋向于复杂性，还趋向于不断增强的进化能力（Dawkins, 2019; Gerhart & Kirschner, 2007; Hendrikse et al., 2007）？

元学习基础模型智能体。许多基于基础模型（FM）的智能体是手工设计的。一些构建模块包括提示工程（Chen et al., 2023; Schulhoff et al., 2024）、思维链（Wei et al., 2022; Yao et al., 2023; Hu & Clune, 2024; Guo et al., 2025; Lightman et al., 2023; Muennighoff et al., 2025; Zelikman et al., 2024a）、自我反思（Shinn et al., 2023; Yao et al., 2023; Madaan et al., 2023）、多智能体辩论（Zhuge et al., 2023; Liang et al., 2023; Khan et al., 2024）、记忆（Liu et al., 2023; Zhong et al., 2024; Modarressi et al., 2023）、温度采样（Zhu et al., 2024）以及检索增强生成（Lewis et al., 2020）。这些组件的手工组合将系统的能力限制在其人类设计者的创造力范围内。最近，出现了几种元学习方法，利用基础模型自动优化提示（Fernando et al., 2024; FAIR; Khattab et al., 2023; Cheng et al., 2024; Yuksekgonul et al., 2024; Yuan et al., 2024）并设计智能体模块（Zhang et al., 2024c; Zhou et al., 2024; Yin et al., 2024; Zhuge et al., 2024; Rosser & Foerster, 2025; Zhang et al., 2025a; Ye et al., 2025; Gao et al., 2025; Nie et al., 2025; Su et al., 2025; Zhang et al., 2025b; Niu et al., 2025）。智能体系统的自动化设计（ADAS, Hu et al., 2025）通过固定的元智能体迭代生成下游智能体，根据目标标准对其进行评估，并纳入反馈以改进后续生成。相比之下，DGM 是一个单一系统，既解决下游任务（即编程问题），又改进其自身的实现（即其代码库），从而消除了对固定、手工设计的元智能体的需求，并实现了自我参照的改进。

自我改进的人工智能。早期，多位研究者提出了自我改进的理论与概念方法（Good, 1966; Schmidhuber, 1987; 2007）。一些自动自我改进的实用方法包括由神经网络权重参数化定义的系统（Schmidhuber, 1993; Hall, 2007; Hobbhahn, 2025; Kirsch & Schmidhuber, 2022; Irie et al., 2022; 2025; Lu et al., 2023; Havrilla et al., 2024b）。Metz et al. (2021) 开发了一种基于梯度的优化器，该优化器通过群体训练（Jaderberg et al., 2017）的变体进行自指元训练。Lange et al. (2023) 将此方法扩展到无梯度学习。Silver et al. (2017) 利用自我博弈持续进化智能体，在象棋和围棋等挑战性领域达到了超人类性能。与 DGM 更密切相关的是近期利用基于基础模型的智能体进行自我改进的方法（Yin et al., 2024; Robeyns et al., 2025; Hu et al., 2024; Zelikman

等人, 2024b; 黄等人, 2022; 辛格等人, 2023)。泽利克曼等人 (2024b) 使用元代理生成下游代理, 并根据生成的解决方案所衍生的元效用更新元代理。尹等人 (2024) 使用单一系统同时解决下游任务并递归地修改自身。然而, 下游任务或元效用并不总是与自我改进所需的能力相一致。在 DGM 中, 下游任务的改进直接反映了自我改进能力的提升, 从而实现了自我加速进展的潜力。最相似的是罗本斯等人 (2025) 的同期工作, 该工作同样使用单一代理递归地解决编码问题并修改其自身的代码库。与罗本斯等人 (2025) (以及泽利克曼等人 (2024b); 尹等人 (2024)) 的主要区别在于, DGM 具有开放式探索循环, 鼓励超越即时性能增益的自我修改, 从而避免陷入次优状态。附录 B 还讨论了程序合成和达尔文进化方面的其他相关工作。

3 达尔文哥德尔机

哥德尔机 (Gödel Machine) 是一种理论构想, 指能够搜索可证明改进自身方法的人工智能 (Schmidhuber, 2007)。本文提出达尔文哥德尔机 (Darwin Gödel Machine, DGM), 旨在实现创建哥德尔机的长期梦想。DGM 放宽了哥德尔机不切实际的要求, 即无需从理论上证明某项更改会改进系统, 而是要求通过实验获得经验证据, 以证明所提出的新版本能够提升性能。此外, 由于 DGM 依赖改进的经验证据, 它可能在可能系统 (即所有可计算算法) 的广阔搜索空间中陷入局部最优。为解决这一问题, DGM 在搜索过程中维护已发现解的档案, 促进开放式探索, 而非仅依赖单一解的演化。由于这些原理与达尔文进化论 (Darwin, 2023) (附录 B) 相呼应——即通过从先前发现解的档案中选择一个实体、对其进行修改, 并在其具有有趣新颖性时予以保留, 从而产生新的创新 (Zhang et al., 2024b; Faldor et al., 2025; Stanley & Lehman, 2015) ——我们将该算法命名为达尔文哥德尔机 (图 1)。

编码智能体的自指自改进。深度生成模型 (DGM) 初始仅配备一个编码智能体, 其进展在编码基准上进行评估。编码智能体被定义为单一系统, 通过代码仓库实现, 并由冻结的预训练基础模型 (FMs) 驱动, 能够读取、编写和执行代码。代码以通用图灵完备语言 (如 Python) 表达时, 是构建和改进智能系统的强大媒介, 因为它可以表示任何可计算过程。近期研究 (Hu 等, 2025; Zhang 等, 2024c) 表明, 此类智能体可以通过对其设计 (如提示、工作流和工具) 进行元学习来改进, 这些设计在其代码仓库中实现。因此, 本文将自我改进定义为一项编码任务, 涉及修改智能体自身组件的设计 (即其自身代码, 不包括下一段描述的开放式探索过程)。关键动机在于, 经验证据必须反映系统同时具备自我改进和解决下游任务的能力。通过将深度生成模型配置为编码智能体并测试其编码能力, 观察到的改进不仅展示了下游任务性能的提升, 还体现了进一步自我改进的能力, 因为自我改进本质上是一项修改编码智能体自身代码仓库的编码任务。

基于群体的开放式探索。从单个初始编码智能体出发, DGM 构建了所有已发现智能体的档案库。在每次迭代中, DGM 选择父代智能体进行自我修改并分支产生新的智能体。父代选择大致与每个智能体的性能得分成正比, 与其具有代码库编辑功能的子代数量成反比 (附录 C.2)。这有利于那些尚未被充分探索 (即现有子代较少) 的高性能智能体, 从而同时促进对强者的利用和对有前景但采样较少的谱系的探索。所有智能体都保留非零的选择概率, 确保在足够的计算资源下, 任何改进路径都保持可行。每个被选中的父代分析自身的基准评估日志, 提出下一个要实现的特性, 并将该提议作为问题陈述来执行 (附录 C.3)。随后, 父代将建议的特性实现到自己的代码库中, 生成一个新的编码智能体。每个新生成的智能体都在选定的编码基准上进行定量评估, 以估计其编码能力。只有成功编译并保留编辑给定代码库能力的智能体才会被加入 DGM 档案库, 因为只有它们能够继续自我修改。其余全部被丢弃。

父代选择、自我修改和评估的循环持续进行，逐步扩充解决方案档案库。重要的是，我们注意到已归档的解决方案可以作为垫脚石，在其最初发现之后很久才带来改进，这使得我们的方法与爬山式智能体设计方法（Robeyns 等人，2025）有本质区别。目前，开放式探索过程（即档案维护、父代选择）是固定的，DGM 无法修改，我们将其留作未来工作的方向。附录 C.4 展示了 DGM 算法的伪代码。

4 实验

第 4.1 节描述了实验设置，包括初始化 DGM 的初始编码智能体。我们在两个编码基准上进行了实验：SWE-bench（Jimenez 等人，2024）和 Polyglot（Paul Gauthier，2024）（第 4.2 节）。对于每个基准，我们将 DGM 与两个基线进行比较：没有自我改进智能体的 DGM 和没有开放式探索的 DGM（第 4.3 节）。在所有实验中，我们发现 DGM 优于两个基线，表明自我改进机制和开放式探索对于持续的性能提升至关重要（第 4.4 节）。此外，我们展示了 DGM 发现的特征可以在模型、基准和任务之间迁移（第 4.4 节）。

4.1 实验设置

DGM 由一个单一的编码智能体初始化。这个基础智能体围绕一个冻结的基础模型构建，并增强了工具使用能力（Schick 等人，2023；Anthropic，2024b）。基础模型在每一步自主决定其行动，使用其认为合适的任何工具。基础智能体可以访问两个工具：用于执行 bash 命令的 Bash 工具，以及用于查看和编辑目录及整个文件的编辑工具（附录 C.1）。智能体接收一个单一的输入提示，指定目标仓库的位置、问题陈述以及仓库的测试框架（附录 C.1）。基础智能体被有意设计为轻量级，以提供一个简单的起点，从中可以清晰地展示自我改进。我们运行 DGM 进行 80 次迭代（每次迭代生成一个新的智能体），其中 SWE-bench 并行运行两次迭代，Polyglot 并行运行四次迭代（附录 C.2）。在自我修改期间，编码智能体在 SWE-bench 和 Polyglot 实验中均由 Claude 3.5 Sonnet（新版）（Anthropic，2024a）驱动。在基准评估期间，编码智能体在 SWE-bench 中由 Claude 3.5 Sonnet（新版）驱动，在 Polyglot 中由 o3-mini（OpenAI，2025）驱动（附录 D.1）。

4.2 基准

我们在两个流行的基准上评估了 DGM，这两个基准评估了编码任务的不同方面，以验证算法在各种用例中的有效性。SWE-bench 和 Polyglot 都是评估 AI 智能体自动解决编码任务能力的基准。给定一个代码仓库和任务指令，智能体需要对仓库进行修改以完成任务。SWE-bench 和 Polyglot 都是广泛使用的基准（Zhang et al., 2024d;a; Xia et al., 2024; Cao et al., 2024; Google DeepMind, 2025; Gauthier, 2024），要求 AI 智能体浏览代码仓库，理解不同文件中函数之间的相互作用，并在复杂的代码中发现小错误。SWE-bench 只有 Python 任务，而 Polyglot 有多个编程语言的任务。另一个区别是，每个 SWE-bench 任务可能需要编辑多个文件，而每个 Polyglot 任务主要涉及在单个文件中从头实现解决方案（尽管智能体仍需要检查其他文件以了解需要哪些更改），因此总体文件编辑次数较少。

SWE-bench. 为了避免在无法解决的任务上浪费计算资源，我们使用 **SWE-bench Verified**（OpenAI, 2024），这是 **SWE-bench**（Jimenez et al., 2024）的一个经过人工筛选的子集，其中所有任务都是可解决的。在本文中，术语 **SWE-bench** 默认指 **SWE-bench Verified** 子集。

Polyglot. Polyglot 包含多种编程语言（C++、Rust、Python 等）的任务（Paul Gauthier, 2024）。与 SWE-bench 相比，SWE-bench 是最广泛使用的编码基准之一，很可能包含在基础模型的训练集中，而 **Polyglot** 更为小众，不太可能包含在基础模型的后训练数据中。此外，**Polyglot** 主要由其开发者用于评估 Aider（Gauthier, 2024）。这提供了一个机会，可以将自动设计的智能体与

社区中一个有代表性的编码智能体进行比较，该智能体由人类开发者针对 Polyglot 基准持续开发和测试了一年多。最后，Polyglot 能够测试 DGM 在多种编程语言（不仅仅是 Python）上的泛化能力。我们遵循 Polyglot 排行榜的设置（Paul Gauthier, 2024），但有一个关键区别：排行榜报告 `pass@2`，即智能体可以查看一次真实测试的反馈，而我们报告 `pass@1`，即智能体从未看到真实测试结果。

由于在完整基准上评估成本高昂（附录 E.1），本文采用分阶段评估策略来估计编码智能体的性能。总体而言，当本文对某个编码智能体是相对较强表现者的信心越大时，就会在更多任务上对其进行评估。本文首先在一组 10 个任务上评估每个编码智能体，以验证其基本的代码库编辑功能。只有保留代码编辑能力的智能体才能解决下游编码任务或进行进一步的自我修改。通过这一初始阶段的智能体随后会在扩展的 50 个任务集上进行评估。对于 SWE-bench，任务基于 SWE-bench-verified-mini（Hobbhahn, 2025）进行选择，该子集旨在作为完整基准的代表性样本（附录 E.2）。对于 Polyglot，任务从完整基准中随机抽取子集（附录 E.3）。由于本文使用的大语言模型（LLM）本质上具有随机性，性能可能存在噪声。本文使用较小的子集评估来估计总体有效性并识别最佳候选者。对于 SWE-bench，如果某个编码智能体在 60 个任务的子集上取得超过 40% 的成功率（该启发式阈值基于初步运行中观察到的噪声选定），并且在存档中排名前两位，则会在 200 个任务（140 个加上之前的 60 个）上对其进行评估，以更准确地评估其真实编码能力（附录 E.2）。对于 Polyglot，如果某个编码智能体在 10 个任务的子集上取得超过 40% 的成功率，则会在扩展的 50 个任务子集上对其进行评估（附录 E.3）。

4.3 基线

为了分离并评估 DGM 核心组件的贡献，本文将 DGM 与两个基线进行比较：无自改进智能体的 DGM（DGM w/o self-improve）和无开放式探索的 DGM（DGM w/o open-ended exploration）。无自改进的 DGM 在此设置中复现了 ADAS（Hu 等人, 2025）的方法，其中负责修改编码智能体的元智能体在整个实验过程中保持为基础智能体不变（算法 2）。该基线使我们能够评估迭代自改进在加速更优编码智能体进化中的作用。无开放式探索的 DGM 消除了存档的使用，并始终对自身最新存储版本进行自修改（算法 3）。如果编码智能体自修改到失去编辑代码库所需基本功能的程度，它将无法再修改自身或解决任何编码任务。因此，无开放式探索的 DGM 保留仍保持代码库编辑基本功能的自身最新版本。该基线使我们能够评估拥有存档以及开放式探索的充分记录的有益原则（Clune, 2019; Stanley & Lehman, 2015; Zhang 等人, 2024b; Fernando 等人, 2024; Lee 等人, 2020; Samvelyan 等人, 2024; Colas 等人, 2022b）在引导智能体进化方面的影响。

除了学习到的基线之外，本文将 DGM 与手工设计的开源解决方案进行比较。对于 SWE-bench，我们采用已检查的最先进（SoTA）开源解决方案（即 SWE-bench 团队能够复现结果）（附录 E.4）。对于 Polyglot，我们采用代表性智能体（Aider）（Gauthier, 2024），该智能体是开源的，旨在支持多种编程语言和大规模代码库编辑（附录 E.5）。为了公平比较，我们在用于评估 DGM 的相同基准子集上测量已解决任务的百分比（附录 E.2, 附录 E.3）。这些值在图 2 中以虚线水平线显示。

4.4 结果

经过 DGM 的 80 次迭代后，编码智能体在 SWE-bench 上的性能从 20.0% 提升至 50.0%，在 Polyglot 上从 14.0% 提升至 38.0%（图 2）。由于 DGM 在 Polyglot 实验设置（第 4.2 节）中仅在 50 个任务上进行评估，我们额外在完整的 Polyglot 基准上评估了基础智能体和 DGM 发现的最佳智能体，以更准确地估计改进幅度。在完整的 Polyglot 基准上，DGM 将编码智能体从 14.2% 提升至 30.7%。这表明 DGM 能够自动自我改进，从而创建出更优秀的编码智能体。此外，DGM 发现的最佳智能体在 SWE-bench 上的性能与经过检查的开源人类设计的最先进水平相当（图 2）。在 Polyglot 上，尽管 DGM

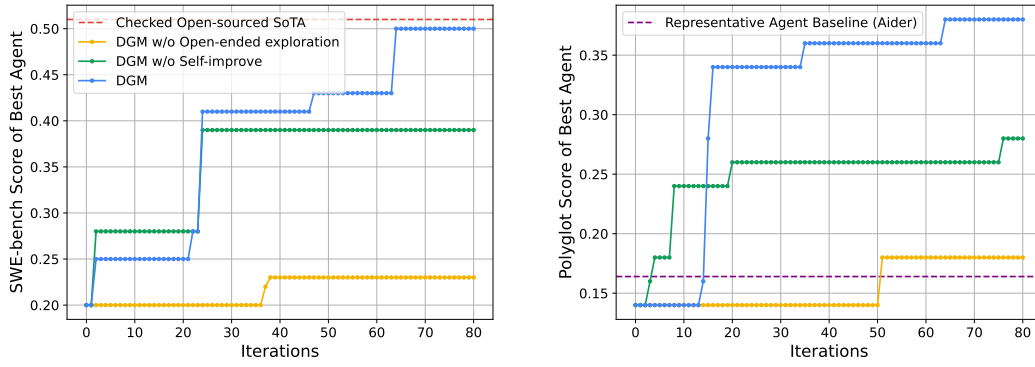


图 2：自我改进和开放式探索使 DGM 能够持续取得进展并提升性能。DGM 自动发现越来越好的编码智能体，并在（左）SWE-bench 和（右）Polyglot 上表现更佳。它优于缺乏自我改进或开放式探索的基线，表明这两个组件对于持续自我改进都是必不可少的。这些分数是通过在第 4.2 节详述的基准子集上进行评估获得的。

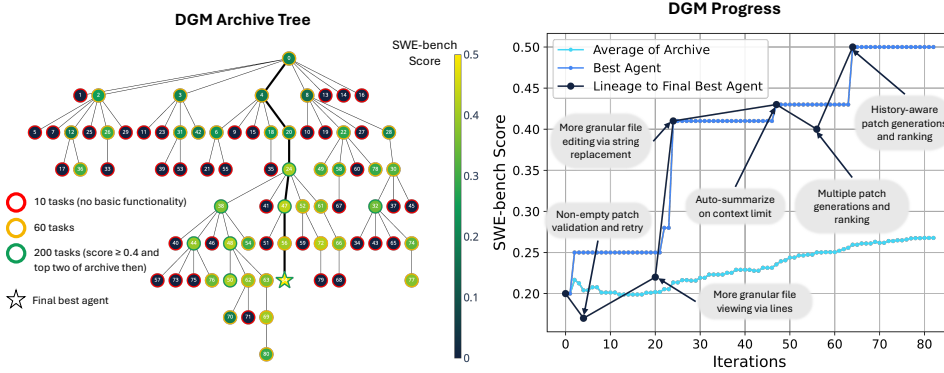


图 3：DGM 自动自我改进，成为更优秀的编码智能体。（左）在 SWE-bench 上运行 DGM 期间生成的编码智能体档案。每个节点代表一个编码智能体，节点 0 对应基础智能体。节点颜色表示在 SWE-bench 上的性能（已解决任务的百分比），边框颜色反映该智能体被评估的任务数量。边显示哪些智能体进行了自我修改以产生后代。许多通往创新的路径经过性能较低的节点，而关键创新（如节点 24）导致在其基础上构建的创新爆发。这两个特性都凸显了开放式搜索的优势。（右）DGM 在 SWE-bench 上的进展图。浅蓝色线显示所有具备基本代码库编辑功能的智能体的平均得分。蓝色线跟踪每次迭代档案中任何智能体达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系，其中包括两次性能下降。这说明了开放式搜索的好处，它探索多样化的有趣垫脚石，而不是只专注于从目前找到的最佳解决方案分支。

从一个性能低于 Aider 的基础智能体开始，它发现了一个远超 Aider 的智能体（图 2）。DGM 发现的智能体在两个基准上与手工设计的智能体相当或更优。虽然最先进的 SWE-bench 智能体和 Aider 是通过人类努力精心塑造的，但 DGM 暗示了一个未来，在这种未来中，这种独创性通过自我指涉的持续自我改进循环实现自动化。

DGM 自动改进工具和利用基础模型的工作流程（图 3）。例如，DGM 增强了编辑工具，允许更细粒度的文件查看（按行）和

更精确的文件编辑（通过字符串替换），而不是总是查看或替换整个文件。工作流程改进包括多次尝试解决任务，并使用另一个基础模型评估和选择最佳解决方案。其他工作流程改进包括在生成后续尝试时考虑之前的尝试。附录 F.1 和附录 F.2 分别展示了在 SWE-bench 和 Polyglot 上最终发现的最佳智能体的所有修改。

由于开放式探索允许从档案中的任何智能体以非零概率进行分支，DGM 能够摆脱性能上的欺骗性低谷或峰值。例如，在 SWE-bench 实验的第 4 次和第 56 次迭代中，尽管智能体的得分暂时低于其父代，DGM 仍能沿该路径探索创新，并创建一个超越所有前代的新智能体（图 3）。此外，开放式探索允许尝试同一目标功能的不同实现。例如，虽然目标是提供更细粒度的编辑工具，但该功能的具体实现可能差异很大，从而导致截然不同的性能（附录 G）。DGM 可以探索多种实现以找到最合适的一种，避免陷入次优实现。

DGM 在两个基准上均优于 DGM 无自改进和 DGM 无开放式探索的基线（图 2）。在不更新修改编码智能体的元智能体的情况下，DGM 无自改进在早期迭代中改进了智能体，但其收益迅速减弱（附录 A.1）。在 DGM 无开放式探索中，仅保留最新的智能体，因此性能不佳的自我修改会使后续改进更难实现（附录 A.1）。

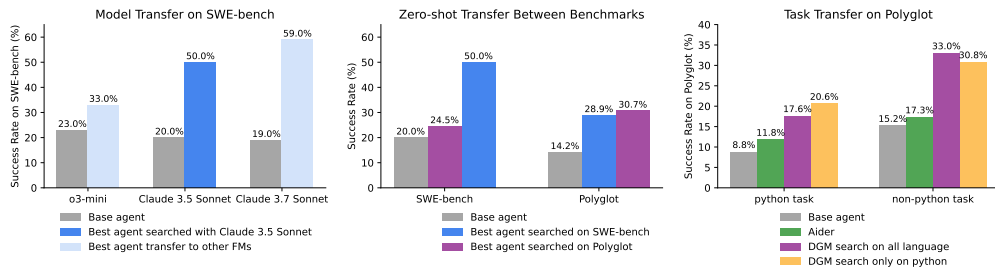


图 4：模型、基准和任务之间的迁移。 DGM 发现的智能体的卓越性能可以跨（左）不同模型、（中）基准和（右）Polyglot 中的不同编程语言任务进行迁移，例如从 Python 任务迁移到 C++ 任务。

为了评估 DGM 改进的通用性，我们使用与优化期间不同的基础模型测试了基础智能体（第 4.1 节）和 DGM 运行期间发现的最佳智能体（图 2）。对于使用 Claude 3.5 Sonnet (New) 运行 DGM 的 SWE-bench，我们将基础模型替换为 Claude 3.7 Sonnet (Anthropic, 2025) 或 o3-mini，并在 200 个任务上进行了评估（图 4，左）。使用 o3-mini 时，基础智能体达到 23.0%，DGM 发现的智能体达到 33.0%。使用 Claude 3.7 Sonnet 时，基础智能体达到 19.0%，DGM 发现的智能体达到 59.5%。这些结果表明，DGM 产生的改进可以跨基础模型泛化，而不是与其运行期间使用的特定基础模型紧密耦合（图 4）。Polyglot 上的模型迁移结果见附录 A.2。

此外，本文研究了深度生成模型（DGM）发现的智能体在不同基准和编程语言之间的可迁移性。首先，本文评估了来自一个基准（如 SWE-bench）的最佳 DGM 发现智能体在完全留出的基准（如 Polyglot）上的表现，反之亦然（图 4，中）。在 SWE-bench 上演化的最佳智能体在 Polyglot 上达到 28.9%，而初始智能体的基线为 14.2%。相反，在 Polyglot 上演化的最佳智能体在 SWE-bench 上达到 24.5%，优于原始基线 20.0%。由于每个智能体在优化过程中从未接触过另一个基准，这些评估代表了真正的留出测试。跨基准的一致性能提升支持了本文的观点，即 DGM 的改进反映了通用技能的获得，而非过拟合或利用基准特定的工件。其次，本文实验了一个仅在 Polyglot 的 Python 任务上训练的 DGM 版本，然后将发现的智能体迁移到其他语言的任务。与在所有语言上训练的 DGM 相比，主要关注 Python 任务略微提高了 Python 任务的性能，但降低了非 Python 任务的性能（图 4，右）。然而，在搜索过程中从 Python 迁移到其他未见语言后，该智能体仍能达到与之相当的性能。

在所有语言上训练的深度生成模型（DGM）显著优于基础智能体和 Aider。这些结果证明了所发现改进的鲁棒性，表明它们并未过拟合于特定编程语言。此外，我们在附录 A 中给出了更多结果。

5 安全性讨论

具备自我改进能力的系统，如 DGM，代表着迈向更自主人工智能发展的一步，与长期以来使人工智能造福人类的领域目标相一致（Schmidhuber, 1987; Clune, 2019; Markoff, 2016; Lehman, 2023）。然而，这种能力带来了独特的安全考量，源于系统自主修改自身代码的能力。仅针对基准性能优化的修改可能无意中引入漏洞或与人类意图不一致的行为，即使它们提升了目标度量（Bostrom, 2020）。特别是，如果评估基准未能完全涵盖所有期望的智能体属性（如安全性和鲁棒性），自我改进循环可能在连续世代中放大不一致性。迭代式自我修改还可能导致日益复杂且难以解释的内部逻辑，阻碍人类的理解、监督和控制（Sheth et al., 2025; Anwar et al., 2024; Greenblatt et al., 2024; Ganguli et al., 2022）。

认识到这些挑战，DGM 当前的实现和实验设置包含了若干安全措施。所有智能体执行和自我修改过程均在隔离的沙箱环境中进行，限制了它们影响宿主系统的能力，从而降低了意外行为的风险。沙箱内的每次执行都受到严格的时间限制，减少了资源耗尽或无界行为的风险。自我改进过程目前仅限于通过修改智能体自身的 Python 代码库来提升特定编码基准性能的明确定义领域，从而限制了潜在修改的范围。此外，我们积极监控智能体性能和代码变更，DGM 存档提供了可追溯的修改谱系以供审查。在此阶段，我们未发现生成的智能体存在有害或恶意行为的证据，自我修改主要集中在提升编码能力上。

相反，自我改进范式的一个显著潜在益处是，原则上它可以被导向增强安全性和可解释性本身。我们对深度生成模型（DGM）如何部署在人工智能安全环境中以开发基础模型（FM）幻觉的对策进行了初步研究（附录 H）。正如深度生成模型学会提升其编码能力一样，它有可能发现并整合更好的内部安全防护措施，或为更高的透明度而修改自身（例如，纳入类似于宪法式人工智能（Constitutional AI）的原则（Bai 等人，2022）），前提是这些属性被纳入其评估标准（Rosser 和 Foerster, 2025）。这表明了一条有前景但充满挑战的路径，即自我改进成为构建更值得信赖的人工智能系统的工具。进一步的研究还可以探索从一开始就融入宪法式人工智能，尽管挑战在于激励系统保留这些指令（一个值得探索的选项是创建系统的一个不可修改部分，以便在停止时评估其余部分）。

深度生成模型展示了自我改进人工智能的潜力，同时由于前沿基础模型当前的局限性和沙盒化等有效缓解措施，仍能在安全的研究边界内运行。附录 I 提供了关于更广泛安全不确定性的额外讨论。我们主动纳入这一安全讨论，以提高对自我改进人工智能系统这一新兴前景及其相关安全影响的认知，特别是随着这些系统不可避免地变得更加强大（Yudkowsky 等人，2008; Bostrom, 2002; Ecoffet 等人，2020; Bengio 等人，2024; Clune, 2019）。因此，我们主张继续研究人工智能生成算法（Clune, 2019）和自我改进系统的安全且有益的演化。

6 结论与局限性

我们介绍了达尔文哥德尔机（DGM），这是第一个由基础模型驱动的具有开放式探索能力的自我改进系统，在其评估基准上的进展可以直接转化为更好的自我改进能力。我们展示了自动发现更好的工具和基础模型系统，从而在两个基准上取得了更好的性能：SWE-bench 和 Polyglot。通过自我改进和开放式探索，达尔文哥德尔机表现出持续的性能提升，使我们离自我加速、自我改进的人工智能系统更近了一步。

我们证明，DGM 能够自主达到与公开可用解决方案相当的性能。然而，它仍不及闭源的最先进 SWE-bench 解决方案。一个开放问题是，让 DGM 运行更长时间是否会持续带来性能提升并最终超越闭源解决方案。这些闭源解决方案通常依赖于由高技能专家团队精心设计的技术。由于基础模型尚未达到此类专家的能力（例如在推理方面），DGM 目前需要大量计算资源来发现改进。如第 4 节所述，DGM 在 SWE-bench 上的单次运行大约需要 2 周时间，并产生显著的 API 成本（附录 E.1）。我们假设，进一步的进展将需要更高效地利用计算资源，并开发更好的推理能力。

由于此版本的 DGM 主要由基础模型驱动，它本质上受限于底层基础模型的能力。因此，一个令人兴奋的未来方向是将自我修改扩展到提示或基础模型工作流之外，纳入计算密集度更高的方法，例如重写其自身的训练脚本以更新基础模型本身。虽然此版本的 DGM 专注于编码，但人工智能系统正越来越多地应用于广泛的领域（例如计算机视觉、创意写作）。另一个有前景的扩展是开发能够超越编码领域进行自我增强的自我改进人工智能系统。本工作的一个关键假设是，编码基准能够很好地反映智能体自我改进的能力，因为自我修改任务要求智能体修改其自身的代码库。然而，可以设想一种替代方法，即共同演化目标任务分布（Faldor 等人，2025；Wang 等人，2023c），从而消除自我改进与单一目标绑定的约束，如同真正的开放式过程。附录 J 介绍了未来工作的其他潜在方向。在我们继续探索这项强大技术的同时，必须将安全置于核心位置，如第 5 节所述。

总之，深度生成模型（DGM）标志着向通过能够编辑自身代码库的自改进系统实现人工智能开发自动化迈出了重要一步。尽管当前计算和推理能力的局限性制约了其全部潜力，但基础模型（FMs）和基础设施的持续进步可能解锁更强大、更通用的自改进能力。只要谨慎应对安全关切（第 5 节），自改进人工智能系统和人工智能生成算法（Clune, 2019）的未来将拥有巨大前景，能够开放地进化人工智能，不断重写或重新训练自身，以追求与人类价值观一致的更强能力。

伦理声明

我们确认遵守国际学习表征会议 (ICLR) 伦理准则。本研究在标准编程基准上评估代码编辑智能体的有限背景下研究自改进人工智能系统。未涉及人类受试者，未收集或处理任何个人信息 (PII)；因此无需机构审查委员会 (IRB) 批准。

安全与滥用。自修改系统若允许无约束地行动，或优化过程无意中引入不安全行为，则可能带来安全风险。为降低此类风险，我们实验中的所有智能体均在具有严格资源和时间限制的隔离沙箱中运行；智能体网络访问受限，且无法修改宿主环境。自改进范围仅限于智能体自身的 Python 代码库和评估工具。我们保留了完整的、可审计的代码变更与评估谱系（存档），支持回滚和事后分析。我们未将发现的智能体部署到真实开发环境中。我们的发布计划（代码、提示和评估工件）将排除任何授予提升系统访问权限的组件，并包含默认沙箱、护栏机制以及明确的预期用途文档。

双重用途、下游影响与局限性。更强的自主编码智能体可能具有双重用途（例如，辅助软件维护，但若被误用，也可能促进有害代码的创建）。我们认为研究收益（例如，推进可控、可审计的自我改进方法，并展示实用的安全措施）大于风险。尽管如此，我们明确反对安全敏感或非沙箱化的部署，并提供了具体的安全建议（第 5 节）。我们对基准优化的实证关注可能无法涵盖所有期望的特性（鲁棒性、可解释性或更广泛的社会价值）。因此，我们将基准提升视为通用人工智能发展的必要但不充分的指标，并讨论了将其他目标（例如，安全性、推理）纳入优化循环的途径。

数据治理、知识产权与许可。我们在 SWE-bench Verified 和 Polyglot 上进行评估，这两个基准由开源仓库和任务组成。我们已尽最大努力遵守数据集许可证和使用条款。我们未引入或分发专有代码。基础模型

（基础模型）通过提供商应用程序编程接口 (API) 在其服务条款下访问；我们未提交敏感数据，也未试图规避使用政策。随本工作发布的日志将清除 API 密钥及任何偶然出现的敏感字符串。

偏差、公平性与公正性。尽管本文的研究领域是软件代码而非以人为中心的文本，基础模型 (FM) 的行为仍可能反映偏差（例如语言或生态系统的偏好），并可能使那些工具在训练数据中代表性更好的社区获得不均衡的收益。本文通过跨多种语言 (Polyglot) 的评估以及报告跨基准的迁移结果，部分地解决了这一问题。未来工作应增加针对偏差失效模式的诊断方法，并纳入更广泛、由社区驱动的任务集。

利益冲突与资金来源。没有任何作者对本文所评估产品的性能拥有经济利益。赞助方和雇主除提供薪资或标准研究支持外，未对实验设计、分析或发表决定施加影响。任何外部计算或应用程序编程接口 (API) 积分均在附录中致谢。

可复现性声明

我们将开源全部代码和完整的智能体日志，包括自我修改的完整归档谱系（差异、提示和配置）以及评估框架。为支持精确复现，我们引用了以下内容：算法细节和伪代码（第 3 节，附录 C.4）；父代选择和开放式探索设置（附录 C.2）；基础模型选择和超参数（附录 D.1）；SWE-bench 和 Polyglot 的基准任务子集（附录 E.2，附录 E.3）；分阶段评估协议和脚本（第 4.2 节）；最佳发现智能体的实现和差异（附录 F.1，附录 F.2）；以及计算和成本估算（附录 E.1）。发布的代码仓库将包含环境规范和脚本，以复现所有结果、图表和表格。

致谢

本研究得到了 Vector Institute、加拿大 CIFAR AI Chairs 项目、Schmidt Futures 的资助、NSERC Discovery Grant 以及 Rafael Cosman 的慷慨捐赠的支持。准备本研究时使用的资源部分由安大略省、加拿大政府通过 CIFAR 以及赞助 Vector Institute 的公司提供

(<https://vectorinstitute.ai/partnerships/current-partners/>)。本材料中表达的任何意见、发现、结论或建议均为作者的观点，不一定反映赞助方的观点。我们还要感谢 Aaron Dharna、Ben Norman、Cédric Colas、Sam Devlin 和 Shyam Sudhakaran 提供的深刻讨论和反馈。

参考文献

- Fuma Aki, Riku Ikeda, Takumi Saito, Ciaran Regan, and Mizuki Oka. Llm-poet: Evolving complex environments using large language models. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 243–246, 2024.
- Rajeev Alur, Rishabh Singh, Dana Fisman, and Armando Solar-Lezama. Search-based program synthesis. *Communications of the ACM*, 61(12):84–93, 2018.
- S-I Amari. Learning patterns and pattern sequences by self-organizing nets of threshold elements. *IEEE Transactions on computers*, 100(11):1197–1206, 1972.
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.
- Anthropic. Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, June 2024a. [Accessed 17 April 2025].
- Anthropic. Claude can now use tools, May 2024b. URL <https://www.anthropic.com/news/tool-use-ga>. Accessed: 2025-05-03.
- Anthropic. Claude 3.7 sonnet and claude code, February 2025. URL <https://www.anthropic.com/news/claude-3-7-sonnet>. Accessed: 2025-05-06.
- Usman Anwar, Abulhair Saparov, Javier Rando, Daniel Paleka, Miles Turpin, Peter Hase, Ekdeep Singh Lubana, Erik Jenner, Stephen Casper, Oliver Sourbut, et al. Foundational challenges in assuring alignment and safety of large language models. *arXiv preprint arXiv:2404.09932*, 2024.
- Dzmitry Bahdanau, Kyung Hyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Adrien Baranes and Pierre-Yves Oudeyer. Active learning of inverse models with intrinsically motivated goal exploration in robots. *Robotics and Autonomous Systems*, 61(1):49–73, 2013.
- Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. Hysynth: Context-free llm approximation for guiding program synthesis. *Advances in Neural Information Processing Systems*, 37:15612–15645, 2024.
- Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. Managing extreme AI risks amid rapid progress. *Science*, 384(6698):842–845, 2024.
- N Bostrom. Existential Risks: analyzing human extinction scenarios and related hazards. *Journal of Evolution and Technology*, 9, 2002.
- Nick Bostrom. Ethical issues in advanced artificial intelligence. *Machine Ethics and Robot Ethics*, pp. 69–75, 2020.

- Herbie Bradley, Andrew Dai, Hannah Benita Teufel, Jenny Zhang, Koen Oostermeijer, Marco Bellagente, Jeff Clune, Kenneth Stanley, Gregory Schott, and Joel Lehman. Quality-diversity through ai feedback. In *The Twelfth International Conference on Learning Representations*, 2024.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- J Richard Buchi and Lawrence H Landweber. Solving sequential conditions by finite-state strategies. In *The collected works of J. Richard Büchi*, pp. 525–541. Springer, 1990.
- Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Wenjing Hu, Yuchen Mao, et al. Spider2-v: How far are multimodal agents from automating data science and engineering workflows? *Advances in Neural Information Processing Systems*, 37:107703–107744, 2024.
- Konstantinos Chatzilygeroudis, Antoine Cully, Vassilis Vassiliades, and Jean-Baptiste Mouret. Quality-diversity optimization: a novel branch of stochastic optimization. In *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, pp. 109–135. Springer, 2021.
- Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv preprint arXiv:2310.14735*, 2023.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms. *Advances in Neural Information Processing Systems*, 37:71596–71642, 2024.
- Jeff Clune. AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.
- Cédric Colas, Pierre Fournier, Mohamed Chetouani, Olivier Sigaud, and Pierre-Yves Oudeyer. Curious: intrinsically motivated modular multi-goal reinforcement learning. In *International conference on machine learning*, pp. 1331–1340. PMLR, 2019.
- Cédric Colas, Tristan Karch, Clément Moulin-Frier, and Pierre-Yves Oudeyer. Language and culture internalization for human-like autotelic AI. *Nature Machine Intelligence*, 4(12):1068–1076, 2022a.
- Cédric Colas, Tristan Karch, Olivier Sigaud, and Pierre-Yves Oudeyer. Autotelic agents with intrinsically motivated goal-conditioned reinforcement learning: a short survey. *Journal of Artificial Intelligence Research*, 74:1159–1199, 2022b.
- Cédric Colas, Laetitia Teodorescu, Pierre-Yves Oudeyer, Xingdi Yuan, and Marc-Alexandre Côté. Augmenting autotelic agents with large language models. In *Conference on Lifelong Learning Agents*, pp. 205–226. PMLR, 2023.
- Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pp. 72–83. Springer, 2006.
- Charles Darwin. Origin of the species. In *British Politics and the environment in the long nineteenth century*, pp. 47–55. Routledge, 2023.
- Richard Dawkins. The evolution of evolvability. In *Artificial life*, pp. 201–220. Routledge, 2019.
- Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.
- Aaron Dharna, Cong Lu, and Jeff Clune. Quality-Diversity Self-Play: Open-Ended Strategy Innovation via Foundation Models. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024.

- Li Ding, Jenny Zhang, Jeff Clune, Lee Spector, and Joel Lehman. Quality diversity through human feedback: towards open-ended diversity-driven optimization. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 11072–11090, 2024.
- Theodosius Dobzhansky. *Genetics of the evolutionary process*, volume 139. Columbia University Press, 1970.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.
- Adrien Ecoffet, Jeff Clune, and Joel Lehman. Open questions in creating safe open-ended AI: tensions between control and creativity. In *Artificial Life Conference Proceedings* 32, pp. 27–35. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info ..., 2020.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.
- Anthony W Fisher Edwards. The genetical theory of natural selection. *Genetics*, 154(4):1419–1426, 2000.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pp. 835–850, 2021.
- Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.
- Meta Fundamental AI Research Diplomacy Team (FAIR)[†], Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378(6624):1067–1074, 2022.
- Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. OMNI-EPIC: Open-endedness via Models of human Notions of Interestingness with Environments Programmed in Code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=YlXkzMJpPd>.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *Forty-first International Conference on Machine Learning*, 2024.
- Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022.
- Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, Bryan Hooi, Min Lin, and Tianyu Pang. Flowreasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.
- Paul Gauthier. Aider: Ai pair programming in your terminal. <https://github.com/Aider-AI/aider>, 2024. Accessed: 2025-05-14.
- Loris Gaven, Thomas Carta, Clément Romac, Cédric Colas, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. MAGELLAN: Metacognitive predictions of learning progress guide autotelic LLM agents in large goal spaces. *arXiv preprint arXiv:2502.07709*, 2025.
- John Gerhart and Marc Kirschner. The theory of facilitated variation. *Proceedings of the National Academy of Sciences*, 104(suppl_1):8582–8589, 2007.
- Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pp. 31–88. Elsevier, 1966.

- Google DeepMind. Gemini model “thinking” updates — march 2025. <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/#gemini-2-5-thinking>, March 2025. Accessed: 2025-05-11.
- Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024.
- Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- John Storrs Hall. Self-improving AI: An analysis. *Minds and Machines*, 17(3):249–259, 2007.
- Alex Havrilla, Andrew Dai, Laura O’Mahony, Koen Oostermeijer, Vera Zisler, Alon Albalak, Fabrizio Milo, Sharath Chandra Raparthi, Kanishk Gandhi, Baber Abbasi, et al. Surveying the effects of quality, diversity, and complexity in synthetic data from large language models. *arXiv preprint arXiv:2412.02980*, 2024a.
- Alex Havrilla, Sharath Raparthi, Christoforus Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskiy, Eric Hambro, and Roberta Raileanu. Glore: When, where, and how to improve llm reasoning via global and local refinements. *arXiv preprint arXiv:2402.10963*, 2024b.
- Jesse Love Hendrikse, Trish Elizabeth Parsons, and Benedikt Hallgrímsson. Evolvability as the proper focus of evolutionary developmental biology. *Evolution & development*, 9(4):393–401, 2007.
- Nathan Herr, Tim Rocktäschel, and Roberta Raileanu. Llm-first search: Self-guided exploration of the solution space. *arXiv preprint arXiv:2506.05213*, 2025.
- Marius Hobbhahn. Swe-bench verified mini. <https://github.com/mariushobbhahn/SWEBench-verified-mini>, April 2025. Accessed: 2025-04-16.
- John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.
- Shengran Hu and Jeff Clune. Thought Cloning: Learning to think while acting by imitating human thinking. *Advances in Neural Information Processing Systems*, 36, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. Self-evolving multi-agent collaboration networks for software development. *arXiv preprint arXiv:2410.16946*, 2024.
- Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. *arXiv preprint arXiv:2210.11610*, 2022.
- Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-endedness is essential for artificial superhuman intelligence. *arXiv preprint arXiv:2406.04268*, 2024.
- Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pp. 9660–9677. PMLR, 2022.
- Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. Metalearning continual learning algorithms. *Transactions on Machine Learning Research*, 2025.

- Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pp. 4940–4950. PMLR, 2021.
- Minqi Jiang, Tim Rocktäschel, and Edward Grefenstette. General intelligence requires rethinking exploration. *Royal Society Open Science*, 10(6):230539, 2023.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, Oleg Klimov, and Jeff Clune. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*, 2021.
- Akbir Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R Bowman, Tim Rocktäschel, and Ethan Perez. Debating with more persuasive llms leads to more truthful answers. *arXiv preprint arXiv:2402.06782*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Yoon Kim, Carl Denton, Luong Hoang, and Alexander M Rush. Structured Attention Networks. In *International Conference on Learning Representations*, 2017.
- Motoo Kimura. The neutral theory of molecular evolution. *Scientific American*, 241(5):98–129, 1979.
- Louis Kirsch and Jürgen Schmidhuber. Self-referential meta learning. In *First Conference on Automated Machine Learning (Late-Breaking Workshop)*, 2022.
- Martin Klissarov, Pierluca D’Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, Pascal Vincent, Amy Zhang, and Mikael Henaff. Motif: Intrinsic motivation from artificial intelligence feedback. *arXiv preprint arXiv:2310.00166*, 2023.
- Martin Klissarov, Mikael Henaff, Roberta Raileanu, Shagun Sodhani, Pascal Vincent, Amy Zhang, Pierre-Luc Bacon, Doina Precup, Marlos C Machado, and Pierluca D’Oro. MaestroMotif: Skill Design from Artificial Intelligence Feedback. *arXiv preprint arXiv:2412.08542*, 2024.
- Varun Raj Kompella, Marijn Stollenga, Matthew Luci, and Juergen Schmidhuber. Continual curiosity-driven skill acquisition from high-dimensional video inputs for humanoid robots. *Artificial Intelligence*, 247:313–335, 2017.
- Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 29–30, 2023.
- Robert Lange, Yingtao Tian, and Yujin Tang. Large language models as evolution strategies. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 579–582, 2024.
- Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.
- Joel Lehman. Machine love. *arXiv preprint arXiv:2302.09248*, 2023.

- Joel Lehman and Kenneth O Stanley. Novelty search and the problem with objectives. *Genetic programming theory and practice IX*, pp. 37–56, 2011.
- Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pp. 331–366. Springer, 2023.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.
- J. Li, Storie J., and J. Clune. Encouraging creative thinking in robots improves their ability to solve challenging problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 193–200, 2014.
- Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. In *International Conference on Computer Aided Verification*, pp. 280–301. Springer, 2024.
- Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.
- Bryan Lim, Manon Flageat, and Antoine Cully. Large language models as in-context ai generators for quality-diversity. In *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*. MIT Press, 2024.
- Seppo Linnainmaa. *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors*. PhD thesis, Master’s Thesis (in Finnish), Univ. Helsinki, 1970.
- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*, 2023.
- Chris Lu, Sebastian Towers, and Jakob Foerster. Arbitrary order meta-learning with simple population-based evolution. In *Artificial Life Conference Proceedings 35*, volume 2023, pp. 67. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , 2023.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024a.
- Cong Lu, Shengran Hu, and Jeff Clune. Intelligent go-explore: Standing on the shoulders of giant foundation models. *arXiv preprint arXiv:2405.15143*, 2024b.
- Cong Lu, Shengran Hu, and Jeff Clune. Automated capability discovery via model self-exploration. *arXiv preprint arXiv:2502.0757*, 2025.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>, 2023.

- John Markoff. *Machines of loving grace: The quest for common ground between humans and robots*. HarperCollins Publishers, 2016.
- Ernst Mayr. *The growth of biological thought: Diversity, evolution, and inheritance*. Harvard University Press, 1982.
- Luke Metz, C Daniel Freeman, Niru Maheswaranathan, and Jascha Sohl-Dickstein. Training learned optimizers with randomly initialized learned optimizers. *arXiv preprint arXiv:2101.07367*, 2021.
- Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*, 2023.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Muhammad U Nasir and Julian Togelius. Practical PCG through large language models. In *2023 IEEE Conference on Games (CoG)*, pp. 1–4. IEEE, 2023.
- Muhammad U Nasir, Steven James, and Julian Togelius. Word2world: Generating stories and worlds through large language models. *arXiv preprint arXiv:2405.06686*, 2024.
- Anh Mai Nguyen, Jason Yosinski, and Jeff Clune. Innovation engines: Automated creativity and improved stochastic optimization via deep learning. In *Proceedings of the 2015 annual conference on genetic and evolutionary computation*, pp. 959–966, 2015.
- Fan Nie, Lan Feng, Haotian Ye, Weixin Liang, Pan Lu, Huaxiu Yao, Alexandre Alahi, and James Zou. Weak-for-strong: Training weak meta-agent to harness strong executors. *arXiv preprint arXiv:2504.04785*, 2025.
- Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, and Tongliang Liu. Flow: Modularized agentic workflow automation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. Technical report, Google DeepMind, 2025.
- OpenAI. Introducing swe-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, August 2024. Accessed: 2025-04-16.
- OpenAI. OpenAI o3-mini. <https://openai.com/index/openai-o3-mini/>, January 2025. Accessed: 2025-05-01.
- Pierre-Yves Oudeyer, Frdric Kaplan, and Verena V Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE transactions on evolutionary computation*, 11(2):265–286, 2007.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A Decomposable Attention Model for Natural Language Inference. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2249–2255, 2016.

- Jack Parker-Holder, Philip Ball, Jake Bruce, Vibhavari Dasagi, Kristian Holsheimer, Christos Kaplanis, Alexandre Moufaret, Guy Scully, Jeremy Shar, Jimmy Shi, Stephen Spencer, Jessica Yung, Michael Dennis, Sultan Kenjeyev, Shangbang Long, Vlad Mnih, Harris Chan, Maxime Gazeau, Bonnie Li, Fabio Pardo, Luyu Wang, Lei Zhang, Frederic Besse, Tim Harley, Anna Mitenkova, Jane Wang, Jeff Clune, Demis Hassabis, Raia Hadsell, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 2: A large-scale foundation world model, 2024. URL <https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/>.
- Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pp. 2778–2787. PMLR, 2017.
- Paul Gauthier. o1 tops aider’s new polyglot leaderboard. <https://aider.chat/2024/12/21/polyglot.html>, December 2024. Accessed: 2025-04-16.
- Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pp. 107–126, 2015.
- Justin K Pugh, Lisa B Soros, and Kenneth O Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A Self-Improving Coding Agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- J Rosser and Jakob Nicolaus Foerster. Agentbreeder: Mitigating the AI safety impact of multi-agent scaffolds via self-improvement. In *Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=j0n3BJJTcT>.
- David E Rumelhart, Geoffrey E Hinton, Ronald J Williams, et al. Learning internal representations by error propagation, 1985.
- Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, et al. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *Advances in Neural Information Processing Systems*, 37:69747–69786, 2024.
- Cansu Sancaktar, Christian Gumbsch, Andrii Zadaianchuk, Pavel Kolev, and Georg Martius. SENSEI: Semantic Exploration Guided by Foundation Models to Learn Versatile World Models. *arXiv preprint arXiv:2503.01584*, 2025.
- Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *International conference on machine learning*, pp. 1312–1320. PMLR, 2015.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- Jürgen Schmidhuber. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.
- Jürgen Schmidhuber. A ‘self-referential’ weight matrix. In *International conference on artificial neural networks*, pp. 446–450. Springer, 1993.

- Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial general intelligence*, pp. 199–226. Springer, 2007.
- Jürgen Schmidhuber. Driven by compression progress: A simple principle explains essential aspects of subjective beauty, novelty, surprise, interestingness, attention, curiosity, creativity, art, science, music, jokes. In *Workshop on anticipatory behavior in adaptive learning systems*, pp. 48–76. Springer, 2008.
- Jürgen Schmidhuber. Powerplay: Training an increasingly general problem solver by continually searching for the simplest still unsolvable problem. *Frontiers in psychology*, 4:313, 2013.
- Jürgen Schmidhuber and Rudolf Huber. *Learning to generate focus trajectories for attentive vision*. Institut für Informatik, 1990.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.
- Ivaxi Sheth, Jan Wehner, Sahar Abdelnabi, Ruta Binkyte, and Mario Fritz. Safety is Essential for Responsible Open-Ended Systems. *arXiv preprint arXiv:2502.04512*, 2025.
- Kensen Shi, Hanjun Dai, Wen-Ding Li, Kevin Ellis, and Charles Sutton. Lambdabeam: Neural program search with higher-order functions and lambdas. *Advances in Neural Information Processing Systems*, 36:51327–51346, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2023.
- Joar Skalse, Nikolaus Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.
- Kenneth O Stanley and Joel Lehman. *Why greatness cannot be planned: The myth of the objective*. Springer, 2015.
- Kenneth O Stanley, Joel Lehman, and Lisa Soros. Open-endedness: The last grand challenge you’ve never heard of. *While open-endedness could be a force for discovering intelligence, it could also be a component of AI itself*, 2017.
- Marilyn Strathern. ‘Improving ratings’: audit in the British University system. *European review*, 5(3):305–321, 1997.
- Jinwei Su, Yinghui Xia, Ronghua Shi, Jianhui Wang, Jianuo Huang, Yijin Wang, Tianyu Shi, Yang Jingsong, and Lewei He. Debflow: Automating agent creation via agent debate. *arXiv preprint arXiv:2503.23781*, 2025.
- Shyam Sudhakaran, Miguel González-Duque, Matthias Freiberger, Claire Glanois, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models. *Advances in Neural Information Processing Systems*, 36:54213–54227, 2023.

- OpenAI Team, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.
- Ren-Jian Wang, Ke Xue, Yutong Wang, Peng Yang, Haobo Fu, Qiang Fu, and Chao Qian. Diversity from human feedback. *arXiv preprint arXiv:2310.06648*, 2023b.
- Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions. *arXiv preprint arXiv:1901.01753*, 2019.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2024.
- Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. *arXiv preprint arXiv:2311.01455*, 2023c.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Marco Wiering and Jürgen Schmidhuber. Hq-learning. *Adaptive behavior*, 6(2):219–246, 1997.
- S Wright. The roles of mutation, inbreeding, crossbreeding and selection in evolution, proceedings of the sixth international congress of genetics. *proc sixth int Congr genet [internet]*. *New York*356366, 1932.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Rui Ye, Shuo Tang, Rui Ge, Yaxin Du, Zhenfei Yin, Siheng Chen, and Jing Shao. Mas-gpt: Training llms to build llm-based multi-agent systems. *arXiv preprint arXiv:2503.03686*, 2025.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. G\'' odel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
- Eliezer Yudkowsky et al. Artificial Intelligence as a positive and negative factor in global risk. *Global catastrophic risks*, 1(303):184, 2008.

- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024a.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *First Conference on Language Modeling*, 2024b.
- Dan Zhang, Sining Zhou, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems*, 37:64735–64772, 2024a.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a.
- Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. OMNI: Open-endedness via Models of human Notions of Interestingness. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=AgM3MzT99c>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024c.
- Yuanshuo Zhang, Yuchen Hou, Bohan Tang, Shuo Chen, Muhan Zhang, Xiaowen Dong, and Siheng Chen. Gnn as predictors of agentic workflow performances. *arXiv preprint arXiv:2503.11301*, 2025b.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1592–1604, 2024d.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.
- Andy Zhou, Kevin Wu, Francesco Pinto, Zhaorun Chen, Yi Zeng, Yu Yang, Shuang Yang, Sanmi Koyejo, James Zou, and Bo Li. AutoRedTeamer: Autonomous Red Teaming with Lifelong Attack Integration. *arXiv preprint arXiv:2503.15754*, 2025.
- Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*, 2024.
- Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Zhi Jin, and Hong Mei. Hot or cold? adaptive temperature sampling for code generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 437–445, 2024.
- Mingchen Zhuge, Haozhe Liu, Francesco Faccio, Dylan R. Ashley, Róbert Csordás, Anand Gopalakrishnan, Abdullah Hamdi, Hasan Abed Al Kader Hammoud, Vincent Herrmann, Kazuki Irie, Louis Kirsch, Bing Li, Guohao Li, Shuming Liu, Jinjie Mai, Piotr Piékos, Aditya Ramesh, Imanol Schlag, Weimin Shi, Aleksandar Stanic, Wenyi Wang, Yuhui Wang, Mengmeng Xu, Deng-Ping Fan, Bernard Ghanem, and Jürgen Schmidhuber. Mindstorms in natural language-based societies of mind. *arXiv preprint arXiv:2305.17066*, 2023.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

附录

目录

A 附加结果 24

A.1 SWE-bench 上的基线	24
A.2 Polyglot 上跨模型的通用性	25
A.3 父代选择的消融	25
A.4 DGM 运行的额外统计	25

B 额外相关工作 26

C 算法细节 27

C.1 初始编码智能体	27
C.2 父代选择	28
C.3 自我改进提示词	28
C.4 伪代码	31

D 实验细节 32

D.1 基础模型超参数	32
-------------------	----

E 基准细节 32

E.1 成本估算	32
E.2 SWE-bench 任务	33
E.3 多语言任务	35
E.4 SWE-bench 最先进水平	36
E.5 多语言代表性智能体	36

F 最佳发现的智能体 36

F.1 SWE-bench 上的 DGM	36
F.2 多语言上的 DGM	50

G 相似目标功能，不同实现 60

H 案例研究：解决幻觉问题 69

I 额外的安全性讨论 71

J 未来工作方向补充 72

A 补充结果

A.1 SWE-Bench 上的基线

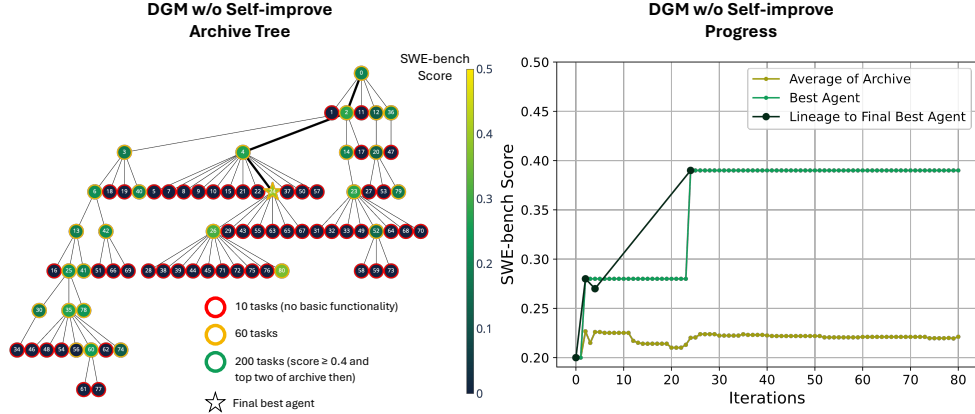


图 5：无自改进智能体的 DGM。保持修改并生成下一代编码智能体的元智能体不变，无自改进的 DGM 无法随时间持续改进。（左）无自改进 DGM 在 SWE-bench 上运行期间生成的编码智能体档案。每个节点代表一个编码智能体，节点 0 对应基础智能体。节点颜色表示在 SWE-bench 上的性能（解决任务的百分比），边框颜色反映该智能体被评估的任务数量。边显示哪些智能体自我修改以产生后代。（右）无自改进 DGM 在 SWE-bench 上的进展图。浅绿色线显示所有具备基本代码库编辑功能的智能体的平均得分。绿色线跟踪档案中任何智能体在每次迭代中达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系。

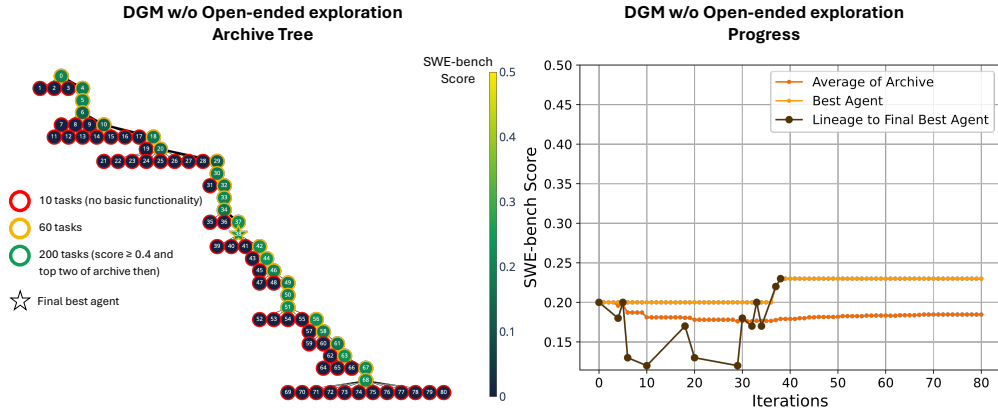


图 6：无开放式探索的 DGM。移除档案后，无开放式探索的 DGM 始终使用最近的智能体进行自我修改，在 SWE-bench 上进展甚微。（左）无开放式探索 DGM 在 SWE-bench 上运行期间生成的编码智能体档案。每个节点代表一个编码智能体，节点 0 对应基础智能体。节点颜色表示在 SWE-bench 上的性能（解决任务的百分比），边框颜色反映该智能体被评估的任务数量。边显示哪些智能体自我修改以产生后代。（右）无开放式探索 DGM 在 SWE-bench 上的进展图。橙色线显示所有具备基本代码库编辑功能的智能体的平均得分。浅橙色线跟踪档案中任何智能体在每次迭代中达到的最佳得分。深色线显示最终发现的最佳智能体及其前驱节点的谱系。

A.2 Polyglot 上跨模型的通用性

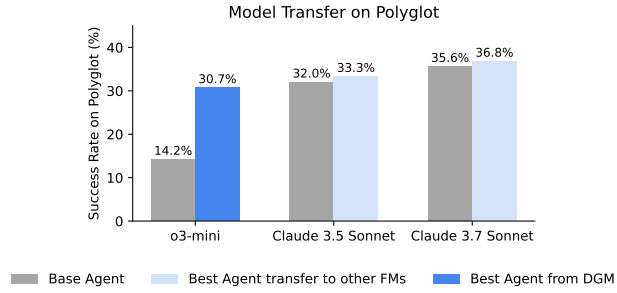


图 7: Polyglot 上模型间的迁移

除了在 SWE-bench 上测试迁移模型（见第 4.4 节，图 2），本节还展示了在 Polyglot 上的迁移结果。在 Polyglot（图 7）上，DGM 使用 o3-mini 运行，我们将 FM 替换为 Claude 3.5 Sonnet（新版）或 Claude 3.7 Sonnet，并在完整基准上进行了评估（图 4，中）。使用 Claude 3.5 Sonnet（新版）时，初始智能体达到 32.0%，DGM 发现的智能体达到 33.3%。使用 Claude 3.7 Sonnet 时，初始智能体达到 35.6%，DGM 发现的智能体达到 36.8%。这些结果表明，DGM 带来的改进在不同 FM 之间具有泛化性，而非紧密依赖于运行期间使用的特定 FM（图 4）。

A.3 父代选择的消融

为了进一步研究 DGM 中父代选择机制的影响，我们引入了 DGM Greedy。DGM Greedy 总是选择性能最佳的节点作为分支的父代，而不是像本实现中的 DGM 那样给每个节点一个非零的分支概率（大致与其性能得分和子节点数量成比例）（附录 C.2）。该消融实验在此设置下复现了 Robeyns 等人（2025）的方法。如表 1 所示，DGM Greedy 在 SWE-bench 和 Polyglot 上分别达到 39.7% 和 30.0%，而本实现的 DGM 达到 50.0% 和 38.0%。这些结果表明，允许存档中的所有解决方案作为潜在的跳板可以带来更大的长期改进，凸显了开放式探索的重要性。

表 1: DGM、其消融版本及基线在 SWE-bench 和 Polyglot 基准上的比较。

Method	SWE-bench	Polyglot
DGM	50.0%	38.0%
DGM w/o Open-ended exploration	23.0%	14.0%
DGM w/o Self-improve	39.0%	28.0%
DGM Greedy	39.7%	30.0%

A.4 DGM 运行的额外统计

具备基本代码编辑功能的生成智能体百分比。为了更深入地了解 DGM 过程，我们分析了在 SWE-bench 基准上具备基本代码编辑功能的生成智能体所占的百分比。如表 2 所示，DGM 在生成具备基本代码库编辑功能的智能体方面表现出最高的百分比。这些结果凸显了 DGM 中开放式探索和自我改进组件的有效性，其中开放式探索使搜索能够跳出局部最优，而自我改进则增强了生成更优智能体的能力。

表 2: 在 SWE-bench 上具备基本代码编辑功能的生成智能体百分比。

Method	Percentage with Basic Code-Editing Functionality
DGM	51.3%
DGM w/o Open-ended exploration	32.5%
DGM w/o Self-improve	32.5%

DGM 运行的稳定性。为评估 DGM 的稳定性, 本文在 Polyglot 基准上运行 DGM 算法三次, 并分析性能的方差。DGM 的平均准确率为 40.7%, 标准差为 2.3%, 表明 DGM 能够在多次运行中取得一致且可复现的结果。

B 补充相关工作

开放式（第二部分）。早期的开放式方法探索了不同的机制来平衡可学习性与趣味性。质量多样性算法旨在用多样且高性能的行为照亮广阔的解空间（Pugh 等, 2016; Chatzilygeroudis 等, 2021; Mouret 和 Clune, 2015; Nguyen 等, 2015）。其他方法强调目标导向的探索（Ecoffet 等, 2019; 2021; Schaul 等, 2015; Andrychowicz 等, 2017; Eysenbach 等, 2018）、内在动机（Lehman 和 Stanley, 2011; Oudeyer 等, 2007; Li 等, 2014; Pathak 等, 2017）或学习进展框架（Kanitscheider 等, 2021; Gaven 等, 2025; Baranes 和 Oudeyer, 2013; Colas 等, 2019; 2022b; Jiang 等, 2021; Dennis 等, 2020; Schmidhuber, 2008; 2013; Kompella 等, 2017）。最近, 大规模基础模型

基础模型（FMs）（Brown 等, 2020; Radford 等, 2019）已成为人类趣味性概念的强大代理（Zhang 等, 2024b; Faldor 等, 2025; Sancaktar 等, 2025），并作为有效的变异算子, 在代码中提出新颖的解决方案（Romera-Paredes 等, 2024; Novikov 等, 2025; Lehman 等, 2023; Faldor 等, 2025; Hu 等, 2025）。基础模型能够引导自驱动智能体（Colas 等, 2022b; 2023; 2022a），建模人类对质量和多样性的偏好（Bradley 等, 2024; Ding 等, 2024; Wang 等, 2023b; Klissarov 等, 2023; 2024; Samvelyan 等, 2024; Lim 等, 2024; Havrilla 等, 2024a），设计奖励函数（Wiering 与 Schmidhuber, 1997; Wang 等, 2023a; Ma 等, 2023; Faldor 等, 2025），创建模拟环境（Sudhakaran 等, 2023; Nasir 与 Togelius, 2023; Aki 等, 2024; Nasir 等, 2024; Bruce 等, 2024; Parker-Holder 等, 2024; Schmidhuber, 2013），驱动不断演化的多智能体动态（Dharna 等, 2024; Zhou 等, 2025），搜索多样的行走机器人形态（Lehman 等, 2023），并在基准或目标优化中搜索广阔的解决方案空间（Lange 等, 2024; Zhang 等, 2024b; Faldor 等, 2025; Hu 等, 2025; Lu 等, 2024b; Romera-Paredes 等, 2024; Fernando 等, 2024; Lu 等, 2024a; Khan 等, 2024; Lu 等, 2025; Liu 等, 2024; Novikov 等, 2025）。

程序合成。程序合成（Alur 等, 2018; Polozov 与 Gulwani, 2015; Buchi 与 Landweber, 1990; Gulwani, 2011; Ellis 等, 2021）旨在生成满足外部规范（如输入输出示例或逻辑公式）的代码。混合方法将符号方法与神经或基础模型引导相结合：例如, Li 等（2024）利用大语言模型建议来引导 SyGuS 环境中的符号搜索, 优于纯枚举。Barke 等（2024）将大语言模型补全与学习到的代理模型相结合, 以引导领域特定语言中的合成。Shi 等（2023）使用神经策略在搜索过程中构建高阶和 lambda 抽象, 在列表操作任务上优于仅使用大语言模型和符号基线。DGM 的不同之处在于, 它不仅关注为外部任务生成程序, 还关注智能体的自我修改, 重写其自身的实现以提高未来自我改进的能力。

达尔文进化论的启发。这项工作深受达尔文进化机制（Darwinian evolution）的启发（达尔文, 2023），特别是变异（突变）、选择以及谱系（踏脚石）的保存, 并将其引入到自修编码智能体的领域。在 DGM 中, 维护着一个过去智能体版本的档案库, 从中采样父代智能体; 然后通过变异（即代码编辑）生成新的子代智能体, 并在编码基准上对其进行经验评估; 成功的智能体被添加到档案库中, 从而能够并行探索多条进化轨迹（第 3 节）。这反映了生物进化（爱德华兹, 2000; 赖特,

1932) 如何保存遗传多样性 (迈尔, 1982), 利用变异 (木村, 1979), 并使用自然选择来保留有益的突变 (多布然斯基, 1970)。

C 算法细节

C.1 初始编码智能体

在本节中, 我们介绍初始编码智能体可用的工具 (第 4.1 节) 及其任务提示的详细信息。

给定 Bash 工具的信息:

```
def tool_info(): 返回 { "名称": "bash", "描述": ""在 bash shell\n 中运行命令 * 调用此工具时, "命令" 参数的内容无需 XML 转义。 \n * 您无法通过此 tool.\n 访问互联网 * 您可以通过 apt 和 pip.\n 访问常见 Linux 和 Python 软件包的镜像 * 状态在命令调用和与 user.\n 的讨论中持续存在 * 要检查文件的特定行范围, 例如第 10-25 行, 请尝试 'sed -n 10,25p /path/to/the/file'. \n * 请避免可能产生大量 output.\n 的命令 * 请在后台运行长时间运行的命令, 例如 'sleep 10 &' 或在 后台启动服务器。 """, "input_schema": { "类型": "对象", "属性": { "命令": { "类型": "字符串", "描述": "要运行的 bash 命令。" }, "required": ["命令"] } }
```

给定 Edit 工具的信息:

```
定义 tool_info(): 返回 { "名称": "编辑器", "描述": ""用于查看、创建和编辑 files\n 的自定义编辑工具。状态在命令调用和与 user.\n 的讨论中持续存在。 * 如果 路径 是文件, 查看 会显示整个文件并带有行号。如果 路径 is a 是 目录, '查看' 会列出最多 2 层的非隐藏文件和目录 deep.\n 。 * 创建 命令在指定的 路径 已作为 file.\n 存在时不能使用。 * If a 命令 生成的长输出将被截断并标记为 <响应 已截断>.\n 。 * 编辑 命令会用提供的 文件文本 ,.\n 覆盖整个文件。 * 不支持部分/行范围编辑或部分查看。 """, "input_schema": { "类型": "对象", "属性": { "命令": { "类型": "字符串", "枚举": ["查看", "创建", "编辑"], "描述": "要运行的命令: '查看'、'创建' 或 '编辑'。" }, "路径": { "描述": "文件或目录的绝对路径, 例如 /仓库/文件.py 或 /仓库。", "类型": "字符串" }, "file_text": { "描述": "'创建' 或 '编辑' 命令的必需参数, 包含整个文件的内容。" } }
```

任务提示：

我已在目录 {self.git_tempdir}. 中上传了一个 Python 代码仓库。请帮助解决  以下问题。

```
<problem_description>
{self.problem_statement}
</problem_description>
```

```
<test_description>
{self.test_description}
</test_description>
```

你的任务是对 {self.git_tempdir} 目录中的文件进行更改，以解决  <problem_description>。我已处理所需的依赖项。

C.2 父代选择

在每次 DGM 迭代中，我们从存档中选择一部分智能体作为父代，进行自我修改并生成新的子代智能体（第 3 节）。受 Ecoffet 等人（2019）启发，父代选择过程的细节概述如下。未来工作可以探索替代的搜索和探索方法（Coulom, 2006; Silver 等人, 2016; Herr 等人, 2025）。

在 DGM 运行的每次迭代 t 中，令

$$\mathcal{A}^t = \{a_1^t, a_2^t, \dots, a_N^t\}$$

作为候选智能体的存档。我们首先定义合格集合

$$\mathcal{E}^t = \{a_i^t \in \mathcal{A}^t : \alpha_i < 1\},$$

即仅包含性能分数尚未达到完美的智能体。然后，我们按比例从 $\mathcal{E}^t$ 中（有放回地）采样 k 个父代，比例依据性能与具有代码库编辑功能的子代数量的综合测度。具体而言，对于每个智能体 $a_i^t \in \mathcal{E}^t$ ：

$$\text{(performance)} \quad \alpha_i = \text{performance}(a_i^t), \quad (1)$$

$$\text{(children count)} \quad n_i = \text{functioning_children_count}(a_i^t), \quad (2)$$

$$\text{(sigmoid-scaled performance)} \quad s_i = \frac{1}{1 + \exp(-\lambda(\alpha_i - \alpha_0))}, \quad (3)$$

$$\text{(novelty bonus)} \quad h_i = \frac{1}{1 + n_i}, \quad (4)$$

$$\text{(unnormalized weight)} \quad w_i = s_i h_i, \quad (5)$$

$$\text{(selection probability)} \quad p_i = \frac{w_i}{\sum_{j=1}^N w_j}, \quad (6)$$

$$\text{(采样)} \quad \{\text{parents}\} \sim \text{分类分布} \left(\{p_i\}_{a_i^t \in \mathcal{E}^t} \right) \quad (\text{draw } k \text{ samples}). \quad (7)$$

此处， $\lambda > 0$ 控制 Sigmoid 函数的锐度， α_0 是中点准确率。我们设置 $\lambda = 10$ 和 $\alpha_0 = 0.5$ 。 k ，后者决定在 DGM 运行的每次迭代中并行执行多少次自我改进。我们在 SWE-bench 实验中设置 $k = 2$ ，在 Polyglot 实验中设置 $k = 4$ 。

C.3 自我改进提示

在每次迭代中，选择一个智能体进行自我修改。在此步骤中，DGM 的当前实现利用一个独立的基础模型来诊断改进机会，然后将这些机会传递给编码智能体以实现。为了识别下一个潜在的改进，所选智能体在编码基准上的评估日志被输入到基础模型中。虽然当前实现依赖于额外的诊断步骤，但不存在根本性限制阻止 DGM 自主分析自身性能、生成改进

假设并实施后续修改。我们选择使用 OpenAI 的 o1 模型 (Team et al., 2024) 来实现此目的, 因为我们发现它在实验时可用的其他基础模型中具有更好的推理能力。用于此诊断的提示如下所示。从诊断输出中, 提取 `implementation_suggestion` 和 `problem_description` 并作为自我修改任务传递给编码智能体。

SWE-bench 实验的自改进诊断提示:

Self-Improve Prompt

```
# Coding Agent Summary

- **Main File**: `coding_agent.py`
  - Primary Class: `AgenticSystem`
  - The `forward()` function is the central entry point.
  - Prompts are located either within the `forward()` function or in the `prompts/`
    ↳ directory.
- **Tools**: `tools/`
  - The `tools/` directory contains various tools that LLMs can use to perform specific
    ↳ tasks.
  - Each tool must have a `tool_info()` function that returns a JSON object containing
    ↳ 'name', 'description', and 'input_schema'. The 'input_schema' should be a JSON
    ↳ object containing 'type', 'properties', and 'required'.
  - Each tool must have a `tool_function()` function that takes the arguments defined
    ↳ in input_schema, performs the tool's task, and returns a string.
  - See other tools for reference.
- **Utilities**: `utils/`
  - The `utils/` directory contains utility functions used across the codebase.

- **Additional Details**:
  - The agent is very good at automatically utilizing the right available tools at the
    ↳ right time. So do not have an agentic flow that explicitly forces a tool's usage.
  - Common tools, such as file editing and bash commands, are easy for the agent to
    ↳ recognize and use appropriately. However, more complex and niche tools may
    ↳ require explicit instructions in the prompt.
  - Tools should be designed to be as general as possible, ensuring they work across
    ↳ any GitHub repository. Avoid hardcoding repository-specific details or behaviors
    ↳ (e.g., paths).
  - Do not use 'while True' loops in the agent's code. This can cause the agent to get
    ↳ stuck and not respond.
  - Verify the implementation details of helper functions prior to usage to ensure
    ↳ proper integration and expected behavior.
  - Do not install additional packages or dependencies directly. Update
    ↳ `requirements.txt` if new dependencies are required and install them using `pip
    ↳ install -r requirements.txt`.

Here is the implementation of the coding agent.

# Coding Agent Implementation
----- Coding Agent Implementation Start -----
{code}
----- Coding Agent Implementation End -----

Your task is to identify ONE detailed plan that would improve the agent's coding
↳ ability. The improvement should not be specific to any particular GitHub issue or
↳ repository.

Here is the log for the coding agent trying to solve the GitHub issues but failed.

# Agent Running Log
----- Agent Running Log Start -----
{md_log}
----- Agent Running Log End -----

# GitHub Issue
The GitHub issue that the agent is trying to solve.
----- GitHub Issue Start -----
{github_issue}
----- GitHub Issue End -----

# Predicted Patch
The agent's predicted patch to solve the issue.
----- Predicted Patch Start -----
{predicted_patch}
----- Predicted Patch End -----

# Private Test Patch
```

```

SWE-bench's official private tests to detect whether the issue is solved. This is not
↳ available to the agent during evaluation. The agent should try to implement its own
↳ tests.
----- Private Test Patch Start -----
{test_patch}
----- Private Test Patch End -----

# Issue Test Results
The test results from SWE-bench using the above official private tests.
----- Issue Test Results Start -----
{eval_log}
----- Issue Test Results End -----

Respond precisely in the following format including the JSON start and end markers:

```json
<JSON>
```

In <JSON>, provide a JSON response with the following fields:
- "log_summarization": Analyze the above logs and summarize how the agent tried to
↳ solve the GitHub issue. Note which tools and how they are used, the agent's
↳ problem-solving approach, and any issues encountered.
- "potential_improvements": Identify potential improvements to the coding agent that
↳ could enhance its coding capabilities. Focus on the agent's general coding
↳ abilities (e.g., better or new tools usable across any repository) rather than
↳ issue-specific fixes (e.g., tools only usable in one framework). All necessary
↳ dependencies and environment setup have already been handled, so do not focus on
↳ these aspects.
- "improvement_proposal": Choose ONE high-impact improvement from the identified
↳ potential improvements and describe it in detail. This should be a focused and
↳ comprehensive plan to enhance the agent's overall coding ability.
- "implementation_suggestion": Referring to the coding agent's summary and
↳ implementation, think critically about what feature or tool could be added or
↳ improved to best implement the proposed improvement. If the proposed feature can be
↳ implemented by modifying the existing tools, describe the modifications needed,
↳ instead of suggesting a new tool.
- "problem_description": Phrase the improvement proposal and implementation suggestion
↳ as a GitHub issue description. It should clearly describe the feature so that a
↳ software engineer viewing the issue and the repository can implement it.

Your response will be automatically parsed, so ensure that the string response is
↳ precisely in the correct format. Do NOT include the `<JSON>` tag in your output.

```

虽然私有测试补丁包含在诊断提示中，但可以想象，智能体在该任务上取得高分的一个简单方法是硬编码解决方案。然而，通过人工检查日志和智能体修改，我们未观察到任何表明记忆或对特定私有测试用例过拟合的问题逻辑或行为。此外，在一个任务或大语言模型上发现的改进通常能很好地推广到其他任务或模型（图 4），这表明这些改进并非特定于任务，而是反映了提升整体性能的通用变化。

Polyglot 实验的附加提示：

Additional Prompt for o3-mini Tool Use and Polyglot

```

### DOC: tool function schema

Carefully consider whether to add/enhance the current tool or edit the workflow in
↳ forward()

Pay special attention to making sure that "required" and "type" are always at the
↳ correct level of nesting. For example, "required" should be at the same level as
↳ "properties", not inside it.
Make sure that every property, no matter how short, has a type and description
↳ correctly nested inside it.
Other arguments than you have seen are not permitted. For example, in
↳ "edit_line_ranges" with "type": "array", arguments like "minItems" and "maxItems"
↳ are not permitted.

...

Here is the log for the coding agent trying to solve a programming task.
A task is in one programming language, but the coding agent needs to deal with
↳ different languages including C++, Go, Java, JavaScript, Python, and Rust.

```


C.4 伪代码

这是第 3 节中描述的 DGM 算法的伪代码:

Algorithm 1: Darwin Gödel Machine

Input: Initial coding agent g_0 , benchmark suite B , maximum iterations T

Output: Archive of agents $\mathcal{A}$

```

 $s_0 \leftarrow \text{evaluate}(g_0, B)$  // Evaluate the base agent
initialize  $\mathcal{A} \leftarrow \{(g_0, s_0)\}$  // Start with the base agent
for  $t \leftarrow 1$  to  $T$  do
   $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$  // Select parent agents
  foreach  $p \in \mathcal{P}$  do
     $c \leftarrow p.\text{modify}(p)$  // Self-modification
     $s \leftarrow \text{evaluate}(c, B)$  // Evaluate on benchmark
    if  $c.\text{is\_valid}()$  then
       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(c, s)\}$  // Keep children capable of code editing
    end
  end
end
return  $\mathcal{A}$ 

```

这是第 4.3 节中描述的无自改进智能体的基线 DGM 的伪代码:

Algorithm 2: Darwin Gödel Machine without Self-improving agents

Input: Initial coding agent g_0 , benchmark suite B , maximum iterations T

Output: Archive of agents $\mathcal{A}$

```

 $s_0 \leftarrow \text{evaluate}(g_0, B)$  // Evaluate the base agent
initialize  $\mathcal{A} \leftarrow \{(g_0, s_0)\}$  // Start with the base agent
for  $t \leftarrow 1$  to  $T$  do
   $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$  // Select parent agents
  foreach  $p \in \mathcal{P}$  do
     $c \leftarrow g_0.\text{modify}(p)$  // Modify with base agent
     $s \leftarrow \text{evaluate}(c, B)$  // Evaluate on benchmark
    if  $c.\text{is\_valid}()$  then
       $\mathcal{A} \leftarrow \mathcal{A} \cup \{(c, s)\}$  // Keep children capable of code editing
    end
  end
end
return  $\mathcal{A}$ 

```

这是第 4.3 节中描述的无开放式探索的基线 DGM 的伪代码:

Algorithm 3: Darwin Gödel Machine without Open-ended exploration

Input: Initial coding agent g_0 , benchmark suite B , maximum iterations T

Output: Archive of agents $\mathcal{A}$

```

 $s_0 \leftarrow \text{evaluate}(g_0, B)$                                 // Evaluate the base agent
initialize  $\mathcal{A} \leftarrow \{(g_0, s_0)\}$                 // Start with the base agent
for  $t \leftarrow 1$  to  $T$  do
     $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$                 // Select parent agents
    foreach  $p \in \mathcal{P}$  do
         $c \leftarrow p.\text{modify}(p)$                         // Self-modification
         $s \leftarrow \text{evaluate}(c, B)$                     // Evaluate on benchmark
        if  $c.\text{is\_valid}()$  then
             $\mathcal{A} \leftarrow \{(c, s)\}$                 // Only keep the latest agent
        end
    end
end
return  $\mathcal{A}$ 

```

D 实验细节

D.1 基础模型的超参数

表 3 展示了每个实验设置中使用的基础模型，如第 4.1 节所述。由于 SWE-bench 是一个更具挑战性的编码基准，我们使用更强的编码模型 Claude 3.5 Sonnet（新版）（基于我们的初步测试）。为了实现更快的迭代并避免与 Claude 相同的速率限制，我们在 Polyglot 实验中使用 o3-mini。所有设置中所有基础模型的温度均设为 1.0。

表 3: 每个实验设置中使用的基础模型。

| 基准 | SWE-bench | Polyglot | 自我修改 | Claude 3.5 Sonnet（新版） | Claude 3.5 Sonnet（新版）评估 | Claude 3.5 Sonnet（新版） | o3-mini |
|----|-----------|----------|------|-----------------------|-------------------------|-----------------------|---------|
| | | | | | | | |

E 基准详情

E.1 成本估算

如第 4 节所述，在 SWE-bench 上完成一次 DGM 运行的估计成本约为 22,000 美元。相比之下，在 SWE-bench 上完成一次任一基线（无自我改进的 DGM 或无开放式探索的 DGM）运行的估计成本约为 10,000 美元。尽管 DGM 的成本远高于基线，但一种能够持续改进的方法，即使成本更高，也优于无法改进或停滞在可能永远无法达到 DGM 性能水平的方法。更细粒度的分解如下：

| LLM | Benchmark | Number of Tasks | Cost Estimate (USD) |
|-------------------------|-----------|-----------------|---------------------|
| Claude 3.5 Sonnet (New) | SWE-bench | 60 | \$350 |
| o3-mini | Polyglot | 60 | \$5 |

我们承认当前在 SWE-bench 上的实验需要大量计算资源。因此，我们还在另一个基准 Polyglot 上进行了实验，其成本显著降低。这表明费用因任务复杂度而异，SWE-bench 属于更复杂且资源密集型的编码基准之一。此外，一些有影响力的方法（例如，LLM 训练初期）最初也以巨大的计算需求为特征。类似于

通过这些开创性的工作，我们希望为未来研究打开大门，以提升我们方法的效率和可扩展性。此外，SWE-bench 排行榜上许多领先的编码智能体背后都有工业公司的支持，这些公司雇佣了专业的全职研究人员和工程师，这带来了大量的人力成本。相比之下，我们的方法通过完全自主的自我改进，在没有人类干预的情况下达到了最先进的性能水平，在考虑专业人工智能开发人才与应用程序编程接口使用的相对成本时，可能提供更高的效率。最后，随着基础模型（Foundation Models）的持续改进和计算成本的持续下降，像 DGM 这样的方法将变得越来越高效和易于使用。

此外，DGM 发现的性能更高的智能体确实比初始智能体产生更高的推理成本，但成本和性能并非严格相关，一些昂贵的智能体表现不如更便宜的智能体。

E.2 SWE-BENCH 任务

用于验证编码智能体基本功能的初始 10 个任务：

- django__django-10973
- django__django-11066
- django__django-12754
- django__django-15930
- django__django-13279
- django__django-16661
- django__django-13346
- django__django-10880
- django__django-10999
- django__django-11087

用于估计编码智能体一般有效性的额外 50 个任务：

- django__django-9296
- django__django-11790
- django__django-11815
- django__django-11848
- django__django-11880
- django__django-11885
- django__django-11951
- django__django-11964
- django__django-11999
- django__django-12039
- django__django-12050
- django__django-12143
- django__django-12155
- django__django-12193
- django__django-12209
- django__django-12262
- django__django-12273
- django__django-12276
- django__django-12304
- django__django-12308
- django__django-12325
- django__django-12406
- django__django-12708
- django__django-12713
- django__django-12774
- sphinx-doc__sphinx-7454
- sphinx-doc__sphinx-7590
- sphinx-doc__sphinx-7748
- sphinx-doc__sphinx-7757
- sphinx-doc__sphinx-7985
- sphinx-doc__sphinx-8035
- sphinx-doc__sphinx-8056
- sphinx-doc__sphinx-8265
- sphinx-doc__sphinx-8269
- sphinx-doc__sphinx-8475
- sphinx-doc__sphinx-8548
- sphinx-doc__sphinx-8551
- sphinx-doc__sphinx-8638
- sphinx-doc__sphinx-8721
- sphinx-doc__sphinx-9229
- sphinx-doc__sphinx-9230
- sphinx-doc__sphinx-9281
- sphinx-doc__sphinx-9320
- sphinx-doc__sphinx-9367
- sphinx-doc__sphinx-9461
- sphinx-doc__sphinx-9698
- sphinx-doc__sphinx-10449
- sphinx-doc__sphinx-10466
- sphinx-doc__sphinx-10673
- sphinx-doc__sphinx-11510

用于更准确评估编码智能体性能的额外 140 个任务:

- astropy__astropy-12907
- astropy__astropy-13033
- astropy__astropy-13236
- astropy__astropy-13398
- astropy__astropy-13453
- astropy__astropy-13579
- astropy__astropy-13977
- astropy__astropy-14096
- astropy__astropy-14182
- astropy__astropy-14309
- astropy__astropy-14365
- astropy__astropy-14369
- astropy__astropy-14508
- astropy__astropy-14539
- astropy__astropy-14598
- astropy__astropy-14995
- astropy__astropy-7166
- astropy__astropy-7336
- astropy__astropy-7606
- astropy__astropy-7671
- astropy__astropy-8707
- astropy__astropy-8872
- django__django-10097
- django__django-10554
- django__django-10914
- django__django-11095
- django__django-11099
- django__django-11119
- django__django-11133
- django__django-11138
- django__django-11141
- django__django-11149
- django__django-11163
- django__django-11179
- django__django-11206
- django__django-11211
- django__django-11239
- django__django-11265
- django__django-11276
- django__django-11292
- django__django-11299
- django__django-11333
- django__django-11400
- django__django-11433
- django__django-11451
- django__django-11477
- django__django-11490
- django__django-11532
- django__django-11551
- django__django-11555
- django__django-11603
- django__django-11728
- django__django-11734
- django__django-11740
- django__django-11749
- django__django-11820
- django__django-12125
- django__django-12419
- django__django-12663
- django__django-12741
- django__django-12858
- django__django-12965
- django__django-13012
- django__django-13023
- django__django-13028
- django__django-13033
- django__django-13089
- django__django-13109
- django__django-13112
- django__django-13121
- django__django-13128
- django__django-13158
- django__django-13195
- django__django-13212
- django__django-13297
- django__django-13315
- django__django-13343
- django__django-13344
- django__django-13363
- django__django-13401
- django__django-13406
- django__django-13410
- django__django-13417
- django__django-13449
- django__django-13512
- django__django-13513

- django__django-13516
- django__django-13551
- django__django-13568
- django__django-13569
- django__django-13590
- django__django-13658
- django__django-13670
- django__django-13741
- django__django-13786
- django__django-13794
- django__django-13807
- django__django-13809
- django__django-13810
- django__django-13820
- django__django-13821
- django__django-13837
- django__django-13925
- django__django-13933
- django__django-13964
- django__django-14007
- django__django-14011
- django__django-14017
- django__django-14034
- django__django-14053
- django__django-14089
- django__django-14122
- django__django-14140
- django__django-14155
- django__django-14170
- django__django-14238
- django__django-14311
- django__django-14315
- django__django-14349
- django__django-14351
- django__django-14373
- django__django-14376
- django__django-14404
- django__django-14434
- django__django-14493
- django__django-14500
- django__django-14534
- django__django-14539
- django__django-14559
- django__django-14580
- django__django-14608
- django__django-14631
- django__django-14672
- django__django-14725
- django__django-14752
- django__django-14765
- django__django-14771
- django__django-14787
- django__django-14792
- django__django-14855

E.3 多语言任务

用于验证编码智能体基本功能的初始 10 个任务：

- go__dominoes
- cpp__all-your-base
- python__dominoes
- java__sgf-parsing
- javascript__robot-name
- rust__variable-length-quantity
- python__beer-song
- go__book-store
- javascript__bottle-song
- rust__bowling

用于估计编码智能体一般有效性的额外 50 个任务：

- javascript__queen-attack
- rust__wordy
- python__dot-dsl
- java__satellite
- cpp__diamond
- rust__accumulate
- go__error-handling
- cpp__queen-attack
- rust__poker
- python__sgf-parsing

- rust__react
- java__ledger
- go__connect
- rust__macros
- javascript__triangle
- java__zipper
- java__bowling
- python__tree-building
- javascript__say
- java__wordy
- python__food-chain
- javascript__wordy
- python__poker
- javascript__grade-school
- cpp__gigasecond
- java__forth
- python__dominoes
- go__word-search
- javascript__simple-linked-list
- go__counter
- java__react
- javascript__ocr-numbers
- python__scale-generator
- java__go-counting
- rust__doubly-linked-list
- python__grade-school
- javascript__forth
- python__wordy
- java__mazy-mice
- cpp__bank-account
- python__zipper
- java__custom-set
- java__rest-api
- go__transpose
- rust__gigasecond
- rust__say
- go__food-chain
- rust__pig-latin
- go__markdown
- go__crypto-square

E.4 SWE-BENCH 最先进技术

截至本文撰写之时（2025 年 4 月 16 日），在 SWE-bench Verified 上表现最佳、经过验证（即 SWE-bench 团队获得了系统访问权限并能够复现补丁生成）的开源条目是 OpenHands + CodeAct v2.1 (claude-3-5-sonnet-20241022) (Wang 等人, 2024)，达到了 53.0%。仅考虑 DGM 使用的相同 200 个任务子集（附录 E.2），OpenHands + CodeAct v2.1 (claude-3-5-sonnet-20241022) 达到 51.0%。

E.5 多语言代表性智能体

Aider (Gauthier, 2024) 是社区中流行的编码智能体，于 2024 年春季发布。它在 GitHub 上已获得超过 33,000 颗星，并由人类开发者持续开发和测试超过一年，主要针对 Polyglot 基准评估其性能。Aider 也成为评估不同模型性能的标准基线，目前 Polyglot 基准上的最佳表现者是 o3（高）和 GPT-4.1 的混合。我们采用与 Polyglot 排行榜类似的设置，但有一个关键区别：排行榜报告 pass@2 性能，其中智能体可以查看一次真实测试的反馈。相比之下，我们使用 pass@1 设置，智能体从未看到真实测试的结果，因为我们认为这更贴近现实编码应用。

F 最佳发现的智能体

F.1 DGM 在 SWE-bench 上的表现

DGM 在 SWE-bench 上发现的最佳智能体的差异补丁：

```
差异 --git a/coding_agent.py b/coding_agent.py 索引 2cd395a..9a2cc2f
100644 --- a/coding_agent.py +++ b/coding_agent.py @@ -4,6 +4,7 @@ 导
入 日志记录 从 日志记录.处理程序 导入 轮转文件处理程序
```

导入 操作系统 导入 线程
处理 + 导入 正则表达式

从 llm_witools 导入 CLAUDE_MODEL, OPENAI_MODEL, chat_with_agent 从 utils.eval_utils 导入 get_report_score, msg_history_to_report, score_tie_breaker @@ -63,6 +64,42 @@ 定义 safe_log(message, level=logging.INFO): 否则: 打印 (f"警告: 未找到线程 {threading.get_ident()}") 的日志记录器

```
+定义 is_patch_valid(patch_str): + """ + 解析补丁以检查是否有任何非测试源文件被修改。+ 返回 (布尔值, 字符串) 元组: (is_valid, 原因) + """ + 如果非 patch_str 或 patch_str.isspace(): + 返回 假, "空补丁" + + # 解析补丁以查找修改的文件 + modified_files = [] + diff_header_pattern = re.compile(r'^\+\+\+ b/(.+)$', re.MULTILINE) + 对于 diff_header_pattern.finditer(patch_str): 中的匹配 + 文件路径 = 匹配.group(1) + 如果文件路径 != '/dev/null': # 跳过已删除的文件 + modified_files.append(filepath) + 如果非 modified_files: + 返回 假, "没有文件被修改" + + # 检查是否有任何非测试文件被修改 + test_patterns = ( + lambda f: f.startswith('tests/'), + lambda f: f.startswith('test_'), + lambda f: f.endswith('_test.py') + ) + source_files = [ + 对于 modified_files 中的 f + 如果非 any(模式(f) 对于 test_patterns 中的模式) + + 如果非 source_files: + 返回 假, "仅修改了测试文件" + 返回 真, "包含源文件修改的有效补丁" + 类 AgenticSystem: 定义 __init__( 自身, @@ -73,6 +110,7 @@ 类 AgenticSystem : test_description=None, self_improve=False, instance_id=None, + max_retries=3, ): self.problem_statement = problem_statement self.git_tempdir = git_tempdir @@ -82,6 +120,7 @@ 类 AgenticSystem : self.self_improve = self_improve self.instance_id = instance_id 如果非 self_improve 否则 'dgm' self.code_model = CLAUDE_MODEL + self.max_retries = max_retries
```

初始化日志记录器并将其存储在线程局部存储中 自身.日志记录器 = setup_logger(chat_history_file) @@ -153,7 +192,7 @@ 您的任务是在 {self.git_tempdir} 目录中运行回归测试以 "" AgenticSystem 的前向函数。"" - 指令 = f""我已将 Python 代码仓库上传到目录 {self.git_tempdir}. 帮助解决以下问题。

↔

+ base_instruction = f""我已将 Python 代码仓库上传到目录 {self.git_tempdir}。帮助解决以下问题。

<problem_description> {self.problem_statement} @@ -165,7 +204,39 @@ 您的任务是在 {self.git_tempdir} 目录中运行回归测试以

```

您的任务是对 {self.git_tempdir} 目录中的文件进行更改以解决 <problem_description>. 我已处理所需的依赖项。"""
new_msg_history = chat_with_agent(instruction, model=self.code_model, msg_history=[], logging=safe_log) + retry_count = 0 + 当
retry_count < self.max_retries: + safe_log(f"\n=== 尝试 {retry_count + 1} 共 {self.max_retries} ===") + # 每次尝试前重置到基础提交
+ 如果 retry_count > 0: + reset_to_commit(self.git_tempdir, self.base_commit) + # 如果这是重试尝试, 则向指令添加上下文 + 指令 =
base_instruction + 如果 retry_count > 0: + 指令 += f"""\nNOTE: 之前的尝试失败, 因为它们要么生成了空补丁, 要么仅修改了测试文
件。

↪

+ 请确保您的解决方案包含对主要源代码文件的更改, 而不仅仅是测试 文件。"""
+
+ # 运行代理 + new_msg_history = chat_with_agent(instruction, model=self.code_model, msg_history=[],
↪ logging=safe_log)
+
+ # 检查补丁 + 补丁 = self.get_current_edits() + is_valid, 原因 =
is_patch_valid(patch)
+
+ 如果 is_valid: + safe_log(f"Valid 补丁已生成: {原因}") + 中断 + 否则: + safe_log(f"Invalid 补丁: {原因}")
+ 如果 retry_count < self.max_retries - 1: + safe_log("Retrying 进行新的尝试...") + 否则: +
safe_log("Maximum 重试次数已达上限。无法生成有效补丁。")

+
+ retry_count += 1

def main():
    parser = argparse.ArgumentParser(description='Process repository with an agentic
↪ system.')
@@ -177,6 +248,7 @@ def main():
    parser.add_argument('--test_description', default=None, required=False,
↪ help='Description of how to test the repository')
    parser.add_argument('--self_improve', default=False, action='store_true',
↪ help='Whether 用于自我改进仓库或解决 SWE 问题')
    parser.add_argument('--instance_id', default=None, help='Instance SWE 问题的 ID') + parser.add_argument('--max_retries', type=int, default=3, help='Maximum 补丁数量
↪ 生成尝试次数')
    参数 = parser.parse_args()

# 处理仓库 @@ -188,6 +260,7 @@ def main():
test_description=args.test_description, self_improve=args.self_improve,
instance_id=args.instance_id, + max_retries=args.max_retries,)

# 运行智能体系统以尝试解决问题 @@ -200,4 +273,4 @@ def main():
f.write(model_patch)

如果 __name__ == "__main__":
- main() + main() \ 文件末尾无换行符 diff --git a/tests/test_patch_validator.py b/tests/test_patch_validator.py 新文件模式 100644 索引
0000000..5689f7d --- /dev/null +++ b/tests/test_patch_validator.py

@@ -0,0 +1,77 @@
+ 导入 pytest + 从 coding_agent 导入 is_patch_valid
+
+ 定义 test_empty_patch():

```



```

+ # 测试空补丁 + is_valid, 原因 = is_patch_valid("") + 断
言非 is_valid + 断言原因 == "空补丁"

+
+ # 测试仅空白补丁 + is_valid, 原因 = is_patch_valid("\n ") + 断
言非 is_valid + 断言原因 == "空补丁"

+
+ 定义 test_test_only_patch(): + 补丁 = "" + diff --git a/tests/test_edit_tool.py
b/tests/test_edit_tool.py + 索引 abc123..def456 100644 +--- a/tests/test_edit_tool.py +++++
b/tests/test_edit_tool.py +@@ -10,6 +10,8 @@ 定义 test_something(): + 断言真 ++ 断言 1 == 1

+ """
+ is_valid, 原因 = is_patch_valid(patch) + 断言非 is_valid + 断言原因 == "
仅修改了测试文件"
+
+ 定义 test_source_file_patch(): + 补丁 = "" + 差异 --git a/工具/
编辑.py b/工具/编辑.py + 索引 abc123..def456 100644 +--- a/工具
/编辑.py +++++ b/工具/编辑.py +@@ -10,6 +10,8 @@ 类 编辑
器: + 定义 编辑 (自身): + 通过 ++ 返回 真

+ """
+ is_valid, 原因 = is_patch_valid(patch) + 断言 is_valid + 断言 原因 == "包含源文件修改的有效
补丁"
+
+ 定义 test_mixed_files_patch(): + 补丁 = "" + 差异 --git a/工具/
编辑.py b/工具/编辑.py + 索引 abc123..def456 100644 +--- a/工具
/编辑.py +++++ b/工具/编辑.py +@@ -10,6 +10,8 @@ 类 编辑
器: + 定义 编辑 (自身): + 通过 ++ 返回 真

+
+ 差异 --git a/tests/test_edit.py b/tests/test_edit.py + 索引 abc123..def456 100644
+--- a/tests/test_edit.py +++++ b/tests/test_edit.py +@@ -10,6 +10,8 @@ 定义
test_something(): + 断言 真 ++ 断言 1 == 1

+ """
+ is_valid, 原因 = is_patch_valid(patch) + 断言 is_valid + 断言 原因 == "包含源文件修改的有效
补丁"
+
+ 定义 test_no_files_modified(): + 补丁 = "" + 差异 --git a/不存在的
.py b/不存在的.py + 已删除文件模式 100644 + 索引 abc123..0000000
+--- a/不存在的.py +++++ /dev/null

+ """
+ is_valid, 原因 = is_patch_valid(patch) + 断言 非 is_valid + 断
言 原因 == "未修改任何文件" \ 文件末尾无换行符

```

```

差异 --git a/工具/编辑.py b/工具/编辑.py 索引
59137ec..16ac521 100644 --- a/工具/编辑.py +++ b/工具/编辑
.py
@@ -1,16 +1,17 @@

```

```
from pathlib import Path import subprocess +from typing import Optional,
List, Tuple, Union
```

```
def tool_info(): return { "name": "editor", "description": ""}用于查看、创建和编辑 files\n 的自定义编辑工具 * 状态在命令调用和与
user\n 的讨论中持续存在 -* 如果`path`是文件，`view`显示整个文件并带有行号。如果`path`是目录，`view`列出最多 2 层的非隐藏
文件和目录 deep\n +* 若`path`为文件，`view`将显示带行号的文件内容。可选参数
```

```
↔
```

```
↔ view_range` [开始, 结束], 它仅显示指定行。在`end`中使用 -1 表示
```

```
↔ 所有剩余 lines.\n
```

```
+* 若`path`为目录，`view`列出最多 2 层深度的非隐藏文件和目录 ↔。 * 当指定的 create 命令无法使用时，指定的
path 已作为 file.\n 存在 * If a command 生成较长输出时，将被截断并标记为 <response clipped>.\n -* `edit` 命
令用提供的`file_text`覆盖整个文件.\n -* 不支持部分/行范围编辑或部分查看。""", +* `edit` 命令用提供的`file_text`覆盖整个
文件`input_schema`: { "type": "object", "properties": { @@ -26,6 +27,13 @@ def tool_info(): "file_text": { "description": "`create` 或 `edit` 命
令的必需参数，包含整个文件的内容。", "type": "string"
```

```
↔
```

```
+ },
```

```
+ "view_range": { "description": "`view` 命令的可选参数。[start_line, 的数组 end_line]
```

```
↔ (从 1 开始)。使用 -1 表示 end_line 读取到文件末尾。",
```

```
+ "type": "array",
```

```
+ "items": { "type": "integer"},
```

```
+ "minItems": 2,
```

```
+ "maxItems": 2
```

```
}
```

```
},
```

```
"required": ["command", "path"]
```

```
@@ -89,6 +97,46 @@ def read_file(path: Path) -> str:
```

```
except Exception as e:
```

```
raise ValueError(f"Failed to read file: {e}")
```

```
+def read_file_range(path: 路径, line_range: 可选 [整数列表] = 无) -> 元组 [字符串, ↔ 整数]:
```

```
+ """
```

```
+ 读取并返回指定范围内的文件内容。返回元组 (内容, start_line).
```

```
+
```

```
+ 参数: path: 文件的路径对象 + line_range: 可选的 [起始, 结束] 行号 (从 1 开始)。结束使用 -1 表示读取到 ↔ 文件末尾。
```

```
+ """
```

```
+ 尝试: + 如果 line_range 为 None: +
```

```
返回 read_file(path), 1
```

```
+
```

```
+ 起始, 结束 = line_range + 如果 起始 < 1: + 抛出 ValueError("起始行必须 >= 1") + 如
```

```
果 结束 != -1 且 结束 < 起始: + 抛出 ValueError("结束行必须 >= 起始行或 -1")
```

```
+
```

```
+ 使用 path.open() 打开文件: + # 跳过起
```

```
始行之前的行 + 对于 _ 在 range(起始 - 1)
```

```
中: + next(f, None)
```

```
+
```

```
+ 行 = [] + current_line = 起始 + 当 True 时: + 行 =
```

```
next(f, None) + 如果 行 为 None: # 文件末尾 + 中断 +
```

```
如果 结束 != -1 且 current_line > 结束: + 中断 +
```

```
lines.append(line.rstrip("\n")) + current_line += 1
```

```

+
+ 返回 '\n'.join(lines), 起始
+
+ 捕获异常 e: + 抛出 ValueError(f"读取文件范围失败: {e}")
+
def write_file(path: Path, content: str): """写入（覆盖）整个文件内容。""" try: @@ -96,9
+144,18 @@ def write_file(path: Path, content: str): except Exception as e: raise ValueError(f"写
入文件失败: {e}")

-def view_path(path_obj: Path) -> str: - """查看整个文件内容或目录列表。""" +def view_path(path_obj: Path,
view_range: Optional[List[int]] = None) -> str: + """ + 查看文件内容（可选指定范围）或目录列表。 + 参数: +
path_obj: 文件或目录的 Path 对象 + view_range: 可选的文件查看行号范围 [start, end] + """ if path_obj.is_dir(): + if
view_range is not None: + raise ValueError("view_range 不支持目录列表")

+
+
+ # For directories: list non-hidden files up to 2 levels deep
+ try:
+     result = subprocess.run(
@@ -115,14 +172,14 @@ def view_path(path_obj: Path) -> str:
+ except Exception as e:
+     raise ValueError(f"Failed to list directory: {e}")

- # 如果是文件，显示带行号的整个文件
- content = read_file(path_obj)
- return format_output(content, str(path_obj)) + # 如果是文件，显示文件内容（可选行范围） + content, start_line =
read_file_range(path_obj, view_range) + return format_output(content, str(path_obj), start_line)

-def tool_function(command: str, path: str, file_text: str = None) -> str: +def tool_function(command: str, path: str, file_text:
str = None, view_range:

    <- Optional[List[整数]] = None) -> str:

    """主工具函数，处理:

    --'view': 查看整个文件或目录列表 + -'view': 查看文件或目录列表，对于文件可选指定行范围

    -'create': 使用给定的 file_text 创建新文件
    - 'edit' : Overwrite an existing file with file_text
    """
@@ -130,7 +187,7 @@ def tool_function(command: str, path: str, file_text: str = None) ->
    <- str:
        path_obj = validate_path(path, command)

        如果命令 == "view":

- 返回 view_path(path_obj) + 返回 view_path(path_obj, view_range)

否则如果命令 == "create": 如果 file_text 为 None: @@ -152,4 +209,4 @@ def tool_function(command: str, path: str, file_text: str =
None) -> <- str:

    如果 __name__ == "__main__": # 示例
    用法

- print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py")) + print(tool_function("view",
"/home/ubuntu/xx/dgm/coding_agent.py")) \ 文件末尾无换行符 diff --git a/tests/test_tools/test_edit.py
b/tests/test_tools/test_edit.py 新文件模式 100644 索引 0000000..04f535b --- /dev/null +++ b/tests/test_tools/test_edit.py

@@ -0,0 +1,54 @@
+ 导入 pytest + 从 pathlib 导入 Path + 从 tools.edit 导入
tool_function
+
+ def test_view_line_range(tmp_path): + # 创建测试文
件

```

```

+ test_file = tmp_path / "test.txt" + test_content = "line1\nline2\nline3\nline4\nline5\n" + test_file.write_text(test_content) +
# 测试查看特定行范围 + result = tool_function("view", str(test_file), view_range=[2, 4]) + assert "line2" in result + assert "line3" in
result + assert "line4" in result + assert "line1" not in result + assert "line5" not in result + assert "2\t" in result # 正确的行号 +
# 测试从开头查看到中间 + result = tool_function("view", str(test_file), view_range=[1, 3]) + assert "line1" in result + assert "line2"
in result + assert "line3" in result + assert "line4" not in result + assert "1\t" in result + # 测试从中间查看到末尾（使用 -1） +
result = tool_function("view", str(test_file), view_range=[3, -1]) + assert "line1" not in result + assert "line2" not in result + assert
"line3" in result + assert "line4" in result + assert "line5" in result + assert "3\t" in result + def
test_view_range_validation(tmp_path): + # 创建测试文件 + test_file = tmp_path / "test.txt" + test_content =
"line1\nline2\nline3\n" + test_file.write_text(test_content) + # 测试无效的起始行 + result = tool_function("view", str(test_file),
view_range=[0, 2]) + assert "Failed to read file range: Start line must be >= 1" in result + # 测试无效范围（结束行小于起始
行） + result = tool_function("view", str(test_file), view_range=[2, 1]) + assert "Failed to read file range: End line must be >= start
line or -1" in result + def test_view_range_with_directory(tmp_path): + # 测试目录不支持 view_range + result =
tool_function("view", str(tmp_path), view_range=[1, 10]) + assert "Error: view_range is not supported for directory listings" in result \
文件末尾无换行符

```

```

diff --git a/tools/edit.py b/tools/edit.py index 16ae521..757f5c2 100644 --- a/tools/edit.py +++ b/tools/edit.py @@ -11,21 +11,21 @@ def
tool_info(): 如果 路径 是目录， 查看 会列出最多 2 层的非隐藏文件和目录 深度。 * 该 创建 命令在指定的
路径 已作为 file.\n 存在时无法使用。 * If a 命令 生成较长的输出时，将被截断并标记为 <响应 已裁剪> .\n -* 编辑
命令会用提供的 'file_text' 覆盖整个文件。+* 'str_replace' 命令将 old_str 的唯一出现替换为 new_str, 若 old_str 未找到或出现多
* 次则 失败。"", "input_schema": { "类型": "对象", "属性": { "命令": { "类型": "字符串", - "枚举": ["查看", "创建", "编辑"], - "描述
": "要运行的命令: `查看`、`创建`或`编辑`。" + "枚举": ["查看", "创建", "str_replace"], + "描述": "要运行的命令: `查看`、`创建`或
`str_replace`。", "路径": { "描述": "文件或目录的绝对路径，例如 /仓库/文件.py 或 /仓库"类型": "字符串"}, "file_text": {

```

↪ .",

```

- "描述": `create` 或 `edit` 命令的必需参数, 包含内容
  ↳ 针对整个文件。"
+ "description": "`create` 命令的必需参数, 包含 ↳ 整个文件的内容。", "type": "string", "view_range": { "@@ -34,6 +34,14 @@ def
tool_info(): "items": {"type": "integer"}, "minItems": 2, "maxItems": 2

+ },
+ "old_str": { + "description": "`str_replace` 命令的必需参数, 包含精确文本
  ↳ 查找并替换。
  + "类型": "字符串"
+ },
+ "new_str": { + "描述": "`str_replace` 命令的必需参数, 包含用于 ↳ 替换 old_str 的新文本。", + "类型": "字符串" } }, "必需": ["命令",
"路径"] @@ -51,7 +59,7 @@ def validate_path(path: 字符串, 命令: 字符串) -> 路径: 验证每个命令的文件路径: - '查看': 路径可以是文
件或目录; 必须存在。

- "创建": 路径必须不存在 (用于新建文件)。
-- 'edit': 路径必须存在 (用于覆盖)。

+ 'str_replace': 路径必须存在且为文件。""" path_obj = Path(path)

@@ -69,7 +77,7 @@ def validate_path(path: str, command: str) -> Path:
# Path must not exist
if path_obj.exists():
    raise ValueError(f"Cannot create new file; {path} already exists.")

- elif 命令 == "编辑": + elif 命令 == "str_replace":

# 路径必须存在且必须为文件, 否则 path_obj.exists(): raise ValueError(f"文件 {path} 不存
在。") @@ -144,6 +152,28 @@ def write_file(path: Path, content: str): except Exception as e:
raise ValueError(f"写入文件失败: {e}")

+def str_replace_in_file(path: Path, old_str: str, new_str: str) -> str: + " " + 在文件中将精确出现的 old_str 替换为 new_str。+ 仅当
old_str 恰好出现一次时才执行替换。+ 返回指示成功或失败的消息。+ " " + try: + content = read_file(path) + occurrences =
content.count(old_str) + if occurrences == 0: + return f"错误: 在 {path} 中找不到要替换的精确文本" + elif occurrences > 1: +
return f"错误: 在 {path} 中找到多个 ({occurrences}) 该文本的出现。必须唯一。"

↳

+ 否则: + new_content = content.replace(old_str, new_str) + write_file(path,
new_content) + 返回 f"成功替换{路径}中的文本"

+

+ except Exception as e: + return f"字符串替换期间出错: {e}"

+

def view_path(path_obj: 路径, view_range: 可选 [列表 [整数]] = 无) -> 字符串: """ 查看文件内容 (可选在范围内) 或目录列表。
@@ -176,12 +206,13 @@ def view_path(path_obj: 路径, view_range: 可选 [列表 [整数]] =

↳ 无) -> 字符串:
内容, start_line = read_file_range(path_obj, view_range) 返回 format_output(content,
str(path_obj), start_line)

-定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, view_range:
↳ Optional[List[整数]] = None) -> str:

+定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, view_range: ↳ 可选 [整数列表] = 无,

```

```

+ old_str: 字符串 = 无, new_str: 字符串 = 无) -> 字符串: "" 主工具函数, 处
理:

-- '查看': 查看文件或目录列表, 文件可选行范围
-- '创建': 使用给定的 file_text 创建新文件
-- '编辑': 使用 file_text 覆盖现有文件 + - '查看': 查看文件或目录列表, 文件可选行范围 + - '创建': 使用给定的 file_text 创建新文
件 + - 'str_replace': 将文件中 old_str 的精确出现替换为 new_str "" 尝试: path_obj = validate_path(path, 命令) @@ -195,11 +226,10
@@ 定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, 查看范围: 操作 write_file(path_obj,
file_text) 返回 f"文件已成功创建于: {路径}")

- 否则如果命令 == "编辑":
- 如果 file_text 为无:
- 抛出值错误 ("缺少编辑命令所需的 file_text")

- 返回 f"位于 {路径} 的文件已被新内容覆盖。" + 否则如果命令 == "str_replace": + 如果 old_str 为无或 new_str 为无: + 抛
出值错误 ("缺少 'str_replace' 命令所需的 'old_str' 和/或 'new_str'。") + 返回 str_replace_in_file(path_obj, old_str, new_str)

否则: 抛出值错误 (f"未知命令: {命令}") 差异 --git a/tests/__init__.py b/tests/__init__.py 新文件模式 100644
索引 0000000..e69de29 差异 --git a/tests/test_tools.py b/tests/test_tools.py 新文件模式 100644 索引
0000000..c7f242f --- /dev/null +++ b/tests/test_tools.py @@ -0,0 +1,65 @@ + 导入 pytest + 从 pathlib 导入
Path + 从 tools.edit 导入 tool_function + # 测试夹具 + @pytest.fixture + 定义 temp_file(tmp_path): + file_path =
tmp_path / "test.txt" + 内容 = "行 1\nline 2\nline 3\n" + file_path.write_text(content) + 返回 str(file_path) +
定义 test_str_replace_success(temp_file): + # 测试成功替换 + 结果 = tool_function( + command="str_replace", +
path=temp_file, + old_str="line 2\n", + new_str="replaced line\n" + ) + 断言 "成功替换" 在结果中 + 断言
Path(temp_file).read_text() == "行 1\nreplaced line\nline 3\n" + 定义 test_str_replace_not_found(temp_file): +
# 测试当 old_str 未找到时 + 结果 = tool_function( + command="str_replace", + path=temp_file, +
old_str="nonexistent", + new_str="something" + ) + 断言 "无法找到" 在结果中 + # 原始文件应保持不变 +
断言 Path(temp_file).read_text() == "行 1\nline 2\nline 3\n" + 定义
test_str_replace_multiple_occurrences(temp_file): + # 首先创建一个包含多个匹配项的文件 +
Path(temp_file).write_text("same\nsame\nsame\n") + 结果 = tool_function( + command="str_replace", +
path=temp_file, + old_str="same\n",

```

```
+ new_str="different\n" + ) + 断言 "多个" 在结果中 + # 原始文件应保持不变 + 断言
Path(temp_file).read_text() == "same\nsame\nsame\n" + ) + 定义
test_str_replace_missing_params(temp_file): + # 测试缺少参数 + 结果 = tool_function( +
command="str_replace", + path=temp_file, + ) + 断言 "缺少必需" 在结果中 + ) + 定义
test_str_replace_invalid_path(): + # 测试不存在的文件 + 结果 = tool_function( +
command="str_replace", + path="/nonexistent/path", + old_str="old", + new_str="new"
+ ) + 断言 "不存在" 在结果中 \ 文件末尾无换行符
```

```
差异 --git a/llm_withtools.py b/llm_withtools.py 索引 d1394bb..6cc3604 100644 --- a/llm_withtools.py +++
b/llm_withtools.py @@ -29,7 +29,7 @@ 定义 process_tool_call(tools_dict, tool_name, tool_input): ) 定义
get_response_withtools( 客户端, 模型, 消息, 工具, tool_choice,
```

```
- logging=None,
+ logging=None, system_message=None,
):
尝试: 如果模型中有'claude': @@ -52,13 +52,32 @@ 定义 get_response_withtools( 抛
出值错误 (f"不支持的模型: {model}") 返回响应 除了异常为 e:

- 日志记录 (f"get_response_withtools: 中出错 {str(e)}") + error_msg = str(e) + 日志记录 (f"get_response_withtools: 中
出错 {error_msg}")

# 达到上下文窗口极限
- 如果 str(e) 中有'请求模型的输入过长':
- 通过 + 如果 error_msg 中有'请求模型的输入过长'或'最大上下文长度'在
↳ error_msg:
+ 如果非 system_message: + # 从第一条消息中提取系统消息 (若存在) + system_message = 消息 [0].获取 '内容', 如果消息存在否则
" + 如果 isinstance(system_message, 列表): + system_message = ''.join(块 ['文本'] 对于 system_message 中的块 如果 块 ['类型'] == '文
本')

+
+ # 总结对话历史 + summarized_messages = summarize_messages(client, 模型, 消息, system_message)
+
+ # 使用总结后的消息重试 + 返回
get_response_withtools( + client=client, +
model=model, + messages=summarized_messages, +
tools=tools, + tool_choice=tool_choice, +
logging=logging, + system_message=system_message

+ )

- raise # 记录日志后重新抛出异常 + raise # 重新抛出其他异常

定义 check_for_tool_use(response, model="): """ @@ -247,6 +266,57 @@ 定义
convert_msg_history_openai(msg_history):

return new_msg_history
```

```

+ 定义 summarize_messages(client, 模型, 消息, system_message): + """ + 创建旧消息的精简摘要，同时保留近期上下文。+ 仅摘要
助手和用户消息，工具结果保持原样以确保准确率。+ """ + # 保留最近的消息不变 + recent_msgs = 消息 [-2:] 如果 消息长度 > 2
否则 消息 + 如果 消息长度 <= 2: + 返回 消息 + # 准备要摘要的消息 + msgs_to_summarize = 消息 [-2:] + # 创建摘要提示 +
summary_request = "请创建此对话的简洁摘要，保留关键上下文和重要细节: "

↪

+ 对于 msgs_to_summarize: 中的每条消息 + 如果 是实例 (消息.获取('内容', ), 列表): + 内容 = ''.join(块 ['文本'] 对于 消息 ['内容'] 中的块
如果 块 ['类型'] == '文本') + 否则: + 内容 = 字符串 (消息.获取('内容', )) + 如果 消息.获取('角色') 在 ['助手', '用户'] 中: +
summary_request += f"\n{msg['角色']}: {内容}"

+
+ try: + # 从模型获取摘要 + summary_response, _ = get_response_from_llm( + msg=summary_request, + client=client, +
model=model, + system_message="You 是摘要生成器。创建简明但信息丰富的摘要。", + print_debug=False, + msg_history=[])

+ )
+
+ # 使用摘要创建新的消息历史 + summarized_history = [{ + "角色": "系统", + "内
容": [{"类型": "文本", "文本": system_message}]

+ }, {
+ "好的，请提供您需要翻译的英文文本。:
↪ {summary_response} "}]
+ }]
+
+ # 添加回最近的消息 +
summarized_history.extend(recent_msgs)
+
+ 返回 summarized_history + 异常处理: + # 如果摘要失败，返回原始消息及最近的消息 + 返回 [消息 [0]] +
recent_msgs

+
+ 定义 convert_msg_history(msg_history, model=None): """ 将消息历史从模型特定格式转换为通用格式。@@ -263,7 +333,14
@@ 定义 chat_with_agent_manualtools(msg, 模型, msg_history=None,

↪ logging=print):
    如果 msg_history 为无: msg_history = [] system_message = f'你是一个编码
agent.\n\n{get_tooluse_prompt()}'

- new_msg_history = msg_history + new_msg_history = msg_history.copy() 如果 msg_history 否则 []
+
+ # 确保系统消息是历史中的第一条消息 + 如果非 new_msg_history 或 新消息历史 [0].获取('角色') != '系统
': + new_msg_history.insert(0, { + "角色": "系统", + "内容": [{"类型": "文本", "文本":
system_message}]

+ })

    尝试:
        # 加载所有工具

```

差异对比 a/coding_agent.py b/coding_agent.py 索引 9a2cc2f..3f1bc1d
100644 --- a/coding_agent.py +++ b/coding_agent.py @@ -111,6 +111,7
@@ 类 智能体系统:


```

self.improve=False, instance_id=None, max_retries=3, + num_candidates=3, ): self.problem_statement =
problem_statement self.git_tempdir = git_tempdir @@ -121,6 +122,7 @@ 类 智能体系统 :
self.instance_id = instance_id 若非 self.improve 否则 'dgm' self.code_model = CLAUDE_MODEL
self.max_retries = max_retries + self.num_candidates = num_candidates

# 初始化日志记录器并将其存储在线程局部存储中 自身.日志记录器 = setup_logger(chat_history_file) @@ -190,7 +192,7 @@ 您的
任务是在 {self.git_tempdir} 目录中运行回归测试, 以

    定义 前向传播 (自身): """
- 智能体系统的前向传播函数。
+ 智能体系统的前向传播函数, 该函数生成并评估多个 候选补丁。""" base_instruction = f"""我已将 Python 代码仓库上传至目
录 {self.git_tempdir} 目录。请帮助解决以下问题。

@@ -205,10 +207,18 @@ 您的任务是在 {self.git_tempdir}
    目录中运行回归测试, 以

    您的任务是对 {self.git_tempdir} 目录中的文件进行修改, 以解决 <problem_description>。所需的依赖项已由我处理完毕。"""

+ # 在开始时获取一次回归测试摘要 + regression_tests_summary =
+ self.get_regression_tests()
+ # 用于存储候选者的列表 + valid_patches
+ = [] + valid_reports = []
+
+     retry_count = 0
- while retry_count < self.max_retries: + while retry_count < self.max_retries and len(valid_patches) < self.num_candidates:
+         safe_log(f"\n=== Attempt {retry_count + 1} of {self.max_retries} ===")
-
+ safe_log(f"Valid solutions so far: {len(valid_patches)} of {self.num_candidates}
+         desired")
+
+ # 在每次尝试前重置到基础提交, 如果 retry_count > 0: reset_to_commit(self.git_tempdir, self.base_commit) @@ -216,8 +226,8 @@ 您的
任务是对 {self.git_tempdir} 目录中的文件进行修改, 以 # 如果这是重试尝试, 则将重试上下文添加到指令中 instruction =
base_instruction 如果 retry_count > 0: - instruction += f"""\nNOTE: 之前的尝试失败, 因为它们要么生成了空补丁, 要么只修改了测试
文件。

    请确保您的解决方案包括对主要源代码文件的修改, 而不仅仅是测试 文件。""" + instruction += f"""\nNOTE: 之前的尝试未能产
生足够的有效解决方案。+ 请提供一种不同的方法来解决该问题。您的解决方案必须包括对主要源代码文件的 修改, 而不仅仅是
测试文件。"""

    # 运行代理 new_msg_history = chat_with_agent(instruction, model=self.code_model,
    msg_history=[], logging=safe_log)
@@ -228,16 +238,45 @@ 请确保您的解决方案包括对主要源代码 文件的修改, 而不仅仅是

    如果 is_valid:safe_log(f"Valid 补丁已生成: {原因}") - 中断 + # 对该候选运行回归测试 + test_report
= self.run_regression_tests(regression_tests_summary) + test_score = get_report_score(test_report) +
safe_log(f"Test 得分: {test_score}") + valid_patches.append(patch) +
valid_reports.append(test_report)

```

```

+ 如果 len(valid_patches) >= self.num_candidates: + 中断 否则 :
safe_log(f"Invalid 补丁: {原因}")

- 如果 retry_count < self.max_retries - 1:
- safe_log("Retrying 进行新尝试...")
- 否则:
- safe_log("Maximum 重试次数已达上限。无法生成有效补丁。")
  retry_count += 1

+ 如果未 valid_patches: + safe_log("Failed 生成任何有效补丁。") + 返回 + + # 使用 score_tie_breaker 选择最佳补丁 +
safe_log(f"\n=== 从 {len(valid_patches)} 个候选中选择最佳解决方案 ===") + best_index = score_tie_breaker( + self.problem_statement,
+ valid_patches, + valid_reports, + logging=safe_log + ) + # 重置到基线并应用最佳补丁 + reset_to_commit(self.git_tempdir,
self.base_commit) + best_patch = valid_patches[best_index] + safe_log(f"\n=== 应用最佳解决方案 (候选 {best_index + 1}) ===") +
apply_patch(self.git_tempdir, best_patch) + + # 对所选补丁进行最终验证 + final_test_report =
self.run_regression_tests(regression_tests_summary) + final_score = get_report_score(final_test_report) + safe_log(f"Final 解决方案测试得
分: {final_score}") + def main(): 解析器 = argparse.ArgumentParser(description='Process 使用智能体 系统处理仓库。')
parser.add_argument('--problem_statement', required=True, help='The 要处理的问题陈述')

↪

@@ -249,6 +288,7 @@ def main(): parser.add_argument('--self_improve', default=False, action='store_true',
↪ help='Whether 用于自我改进仓库或解决 SWE 问题')
parser.add_argument('--instance_id', default=None, help='Instance 软件工程问题的标识符') parser.add_argument('--max_retries', type=int, default=3, help='Maximum 数量
↪ 补丁生成尝试次数')
↪ 要生成的解决方案)
参数 = parser.parse_args()

+ parser.add_argument('--num_candidates', type=int, default=3, help='Number 候选

# 处理仓库 @@ -261,6 +301,7 @@ def main():
self_improve=args.self_improve,
instance_id=args.instance_id,
max_retries=args.max_retries, +
num_candidates=args.num_candidates,)

# 运行智能体系统以尝试解决问题

diff --git a/coding_agent.py b/coding_agent.py index 3f1bc1d..588938d 100644 --- a/coding_agent.py +++ b/coding_agent.py @@ -193,42
+193,59 @@ 您的任务是在 {self.git_tempdir} 中运行回归测试

↪ 目录中运行回归测试, 以

def forward(self): """ 智能系统的前向函数, 用于生成并评估多个

↪ 候选补丁。
+ 此版本维护先前有效补丁和测试结果的历史记录, 仅在必要时使用
↪ 决胜规则

+。"""

- base_instruction = f"""我已将 Python 代码仓库上传至目录
↪ {self.git_tempdir}. 帮助解决以下问题。

-
-<problem_description>
-{self.problem_statement}
-</problem_description>

```

```

- -<test_description> {self.test_description} -</test_description> - -您的任务是修改 {self.git_tmpdir} 目录中的文件以解决
<problem_description>. 我已处理所需的依赖项。 -""" - - # 在开始时获取一次回归测试摘要 regression_tests_summary =
self.get_regression_tests()

- # 用于存储候选的列表 + # 用于存储所有有效补丁及其信息的列表 valid_patches = [] valid_reports = [] + valid_scores = [] +
best_score = 0 + best_patches_indices = [] # 共享最佳分数的补丁索引

retry_count = 0 当 retry_count < self.max_retries 且 len(valid_patches) < self.num_candidates: safe_log(f"\n=== 尝试 {retry_count + 1} /
{self.max_retries} ===") safe_log(f"Valid 目前解决方案: {len(valid_patches)} / {self.num_candidates} (期望) + safe_log(f"Current 最
佳测试分数: {best_score}")

# 每次尝试前重置到基础提交 如果 retry_count > 0:
reset_to_commit(self.git_tmpdir, self.base_commit)

- # 如果是重试尝试，则将重试上下文添加到指令中
- 指令 = base_instruction
- 如果 retry_count > 0:
- 指令 += f"""\nNOTE: 之前的尝试未产生足够的有效解决方案。

+ # 构建包含之前最佳解决方案的指令（如果有）+ 指令 = f"""\我已上传一个 Python 代码仓库到目录
↳ {self.git_tmpdir}. 帮助解决以下问题。

+
+<problem_description>
+{self.problem_statement}
+</problem_description>
+
+<test_description>
+{self.test_description}
+</test_description>"""
+
+ # 若存在先前解决方案上下文，则添加 + if valid_patches and retry_count > 0: + previous_solutions = [] + for i,
(patch, report, score) in enumerate(zip(valid_patches, valid_reports,
↳ valid_scores)):
+ previous_solutions.append(f"""+ 先前解决方案
{i+1}: +<code_changes> + {patch}
+</code_changes> + 测试得分: {score} + 测试
报告: {report}

+ """)
+ 指令 += "\n\nPrevious 解决方案 attempts:\n" + "\n".join(previous_solutions) + 指令 += "\nPlease 提供一个新解决方案，以解决先前尝试中的任何局限性或探索一种不同
的方法。"
↳ 先前尝试或探索一种不同的方法。"
+ elif retry_count > 0: + 指令 += f"""\nNOTE: 先前尝试未产生足够的有效解决方案。请提供一种不同的方法来解决该问题。您的解
决方案必须包括对主要源代码文件的 (↔) 更改，而不仅仅是测试文件。"""

+ 指令 += f"\n\nYour 任务是更改 {self.git_tmpdir} (↔) 目录中的文件，以解决 <problem_description>. 我已经处理好了 (↔) 所需的依
赖项。"
+
+ # 运行代理 new_msg_history = chat_with_agent(instruction, model=self.code_model, (↔) msg_history=[],
logging=safe_log)

@@@ -245,6 +262,14 @@@ 请提供一种不同的方法来解决该问题。您的 (↔) 解决方案必须包括

valid_patches.append(patch)
valid_reports.append(test_report)

```

```

+ valid_scores.append(test_score)
+
+ # 更新最佳分数与索引 + 若 test_score > best_score: + best_score = test_score +
best_patches_indices = [len(valid_patches) - 1] + 否则若 test_score == best_score:
+ best_patches_indices.append(len(valid_patches) - 1)

        if len(valid_patches) >= self.num_candidates:
            break
@@ -257,25 +282,30 @@ Please provide a different approach to solve the problem. Your
    ↪ solution must inc
        safe_log("Failed to generate any valid patches.")
    return

- # 使用 score_tie_breaker 选择最佳补丁 + # 仅当多个补丁具有最佳分数时才使用平局决胜
    safe_log(f"\n=== 从 {len(valid_patches)} 个候选中选择最佳解 ===")
- best_index = score_tie_breaker(
- self.problem_statement,
- valid_patches,
- valid_reports,
- logging=safe_log
- ) + 如果 len(best_patches_indices) > 1: + safe_log(f"Multiple 个解决方案 ({len(best_patches_indices)}) 并列最佳分数 ↪
{best_score}。使用平局决胜规则。") + 最佳索引 = score_tie_breaker( + self.problem_statement, + [valid_patches[i] 对于 i 在
best_patches_indices], 中 + [valid_reports[i] 对于 i 在 best_patches_indices], 中 + logging=safe_log ↪ ) + 最佳索引 =
best_patches_indices[最佳索引] + 否则: + 最佳索引 = best_patches_indices[0]

        # 重置到基础状态并应用最佳补丁 reset_to_commit(self.git_tempdir,
self.base_commit) best_patch = valid_patches[最佳索引]

- safe_log(f"\n=== Applying Best Solution (Candidate {best_index + 1}) ===")
+ safe_log(f"\n=== Applying Best Solution (Candidate {best_index + 1}) with score
    ↪ {valid_scores[best_index]} ===")
    apply_patch(self.git_tempdir, best_patch)

    # 所选补丁的最终验证 final_test_report = self.run_regression_tests(regression_tests_summary) final_score =
get_report_score(final_test_report)

- safe_log(f"Final 解测试分数: {final_score}") + safe_log(f"Final 验证测试分数: {final_score}")

def main():
    parser = argparse.ArgumentParser(description='Process repository with an agentic
    ↪ system.')

```

F.2 多语言上的 DGM

有助于 DGM 在多语言 (Polyglot) 上发现的最佳智能体的差异补丁:

```

差异 --git a/coding_agent.py b/coding_agent.py 索引 04ffb36..6639abd
100644 --- a/coding_agent.py +++ b/coding_agent.py @@ -4,6 +4,9 @@ 导
入日志记录 从日志处理程序导入轮转文件处理器 导入操作系统 导入线
程 + 导入 JSON + 从数据类导入数据类 + 从类型导入列表、可选

from llm_withtools import CLAUDE_MODEL, OPENAI_MODEL, chat_with_agent from utils.git_utils import
diff_versus_commit, reset_to_commit, apply_patch @@ -42,6 +45,14 @@ TEST_COMMANDS = { # 用于日志记录器
实例的线程局部存储 thread_local = threading.local()

```

```

+@dataclass +class SolutionAttempt: + """用于存储解决方案尝试信息的类。""" + patch: str # 补丁内容 +
test_output: str # 原始测试输出 + test_success: bool # 测试是否通过 + test_stats: dict # 测试统计信息（例如，通过/失败测试的数量）

+

def get_thread_logger(): """ 获取当前线程特定的日志记录器实例。@@ -102,7 +113,8 @@
class AgenticSystem: chat_history_file='./chat_history.md', test_description=None,
self_improve=False,

- language='python'
+ language='python',
+ max_attempts=3
): self.problem_statement = problem_statement self.git_tmpdir =
git_tmpdir @@ -111,6 +123,7 @@ 类 智能体系统: self.test_description =
test_description self.self_improve = self_improve 自身.语言 = 语言 +
self.max_attempts = max_attempts

# 根据是否启用自我改进来设置代码模型 self.code_model = OPENAI_MODEL if not self_improve else
CLAUDE_MODEL @@ -137,11 +150,63 @@ class AgenticSystem: } return new_msg_history

+ def run_tests(self) -> tuple[bool, str, dict]: + """运行测试并返回成功状态、输出和测试统计信息。""" + success =
False + output = "" + stats = {"passed": 0, "failed": 0, "errors": 0, "total": 0} + + try: + for command in
TEST_COMMANDS.get(self.language, []): + proc = subprocess.run( + command, + cwd=self.git_tmpdir, +
capture_output=True, + text=True, + check=False + ) + output += f'$ {'
'.join(command)}\n{proc.stdout}\n{proc.stderr}\n" + success = proc.returncode == 0 + if not success: + break + # 尝
试从输出中提取测试统计信息 + # 这是一个简单示例；您可能需要添加更复杂的解析 + stats["passed"] =
output.count("PASS") + output.count("ok") + stats["failed"] = output.count("FAIL") + output.count("not ok") +
stats["errors"] = output.count("ERROR") + output.count("panic:") + stats["total"] = stats["passed"] + stats["failed"] +
stats["errors"] + + except Exception as e: + output = f"运行测试时出错: {str(e)}" + success = False + + return success,
output, stats + + def analyze_test_results(self, attempts: List[SolutionAttempt]) -> str: + """分析测试结果并为智能体创
建摘要。""" + summary = "# 测试结果 Analysis\n\n" + + for i, attempt in enumerate(attempts, 1): + summary += f"##
尝试 {i}\n" + summary += f" 测试成功: {attempt.test_success}\n" + summary += f" 测试统计:
{json.dumps(attempt.test_stats, indent=2)}\n" + summary += "关键测试输出:\n```\n\n" + # 提取测试输出的相关部分
（例如错误消息） + key_output = "\n".join(line for line in attempt.test_output.split("\n") if "FAIL" in line or "ERROR"
in line or "PASS" in line) + summary += f"```{key_output}\n```\n\n" + + return summary

```

```

+
+ 定义 前向传播 (自身): ""
- 智能体系统的前向传播函数。
+ 具有多次解决方案尝试的智能体系统 (AgenticSystem) 的前向函数。""

    - task = f""我已将代码仓库上传到目录 {self.git_tempdir}. 帮助
    -> 解决以下问题。

+ 尝试次数: 解决方案尝试列表 (List[求解尝试]) = [] + best_attempt:
+ 可选的解决方案尝试 (Optional[求解尝试]) = 无

+ base_task = f""我已将代码仓库上传到目录 {self.git_tempdir}. -> 帮助解决以下问题。

<problem_description> {self.problem_statement} @@ -149,8
+214,52 @@ class AgenticSystem:

您的任务是对 {self.git_tempdir} 目录中的文件进行更改, 以解决 -> the <problem_description>. 我已处理所需的依赖项。"" -
instruction = f""{task}\n\nPlease 仔细分析问题描述。然后对代码文件进行编辑以完成指令。""

    ->

- init_edit = chat_with_agent(instruction, model=self.code_model, msg_history=[], -> 日志记录=安全日志) + 对于 attempt_num 在
range(self.max_attempts): 中 + # 每次尝试前重置到基础提交 + reset_to_commit(self.git_tempdir, self.base_commit) + # 根据之前的尝试
准备指令 + 指令 = base_task + 如果尝试次数: + # 添加对之前尝试的分析 + 指令 += "\n\nPrevious 次解决方案尝试已经进行。以下是
results:\n\n" + 指令 += self.analyze_test_results(attempts) + 指令 += "\n\nPlease 分析这些结果并提供一个改进的解决方案, 以

    -> 解决发现的问题。"
+ 否则: + 指令 += "\n\nPlease 仔细分析问题描述。然后对 -> 代码文件进行编辑以完成指令。" + # 获取解决方案尝试 +
chat_with_agent(instruction, model=self.code_model, msg_history=[], logging=safe_log) + # 捕获当前补丁 + current_patch =
diff_versus_commit(self.git_tempdir, self.base_commit) + # 运行测试并收集结果 + test_success, test_output, test_stats = self.run_tests()
+ # 创建并存储尝试 + 尝试 = 解决方案尝试 ( + patch=current_patch, + test_output=test_output, + test_success=test_success, +
test_stats=test_stats + ) + 尝试列表.append(尝试) + # 如果这次尝试更好, 则更新最佳尝试 + 如果 test_success 且 (best_attempt 为
无 或 + attempt.test_stats["passed"] > best_attempt.test_stats["passed"]): + best_attempt = 尝试 + # 如果测试完全通过, 我们可以停止
+ 如果 test_success 且 attempt.test_stats["failed"] == 0 且 attempt.test_stats["errors"]

    -> == 0:

+ 中断
+
+ # 如果找到最佳解决方案则应用, 否则使用最后一次尝试 + final_attempt = best_attempt 或 尝试列
表 [-1] + reset_to_commit(self.git_tempdir, self.base_commit) + apply_patch(self.git_tempdir,
final_attempt.patch)

def main():
    parser = argparse.ArgumentParser(description='Process repository with an agentic
    -> system.')
@@ -162,6 +271,7 @@ def main():
    parser.add_argument('--test_description', default=None, required=False,
    -> help='Description of how to test the repository')

```

```

parser.add_argument('--self_improve', default=False, action='store_true',
    ↳ help="Whether 用于自我改进仓库或解决 SWE 问题")
parser.add_argument('--language', required=False, default="python", choices=['cpp',
    ↳ 'java', 'python', 'go', 'rust', 'javascript'], help="Task\`s 编程
    ↳ 语言")
    + parser.add_argument('--max_attempts', type=int, default=3, help='Maximum 数量
    ↳ 解决方案尝试')
参数 = parser.parse_args()

# 处理仓库 @@ -173,6 +283,7 @@ def main():
test_description=args.test_description, self_improve=args.self_improve,
language=args.language, + max_attempts=args.max_attempts,)

# 运行智能体系统以尝试解决问题 @@ -185,4 +296,4 @@ def main():
f.write(model_patch)

如果 __name__ == "__main__":

-main() + main() \ 文件末尾无换行符

```

差异 --git a/工具/编辑.py b/工具/编辑.py 索引 d7184c3..87fa408

100644 --- a/工具/编辑.py +++ b/工具/编辑.py

@@ -1,5 +1,9 @@

从 路径库 导入 路径 导入 子进程 + 从 类型 导入 列表、可
选、字典

+
+ # 用于撤销功能的全局编辑历史 + edit_history: 字典 [字符串, 列表 [
字符串]] = {}

定义 tool_info(): 返回 { @@ -10,14 +14,14 @@ 定义 tool_info(): * 该 创建 命令在指定的 路径 已作为 file.\n 存在时无法
使用 * If a 命令 生成较长输出时, 将被截断并标记为 <响应 已截断> .\n * 该 编辑 命令使用提供的 文
件文本 .\n 覆盖整个文件 * 不支持部分/行范围编辑或部分查看。""", + * 还支持查看特定行范围、字符串替换、文本插入和
撤销操作。""", "input_schema": { "类型": "对象", "属性": { "命令": { "类型": "字符串", - "枚举": ["查看", "创建", "编辑"], - "描述": "要运
行的命令: `查看`、`创建` 或 `编辑`。" + "枚举": ["查看", "创建", "编辑", "str_replace", "插入", "undo_edit"], + "描述": "要运行的命令: `查
看`、`创建`、`编辑`、`str_replace`、`插入` 或 `撤销编辑`。"}, "路径": { "描述": "文件或目录的绝对路径。例如 `/仓库/文件.py`
或 `/仓库` @@ -26,6 +30,23 @@ def tool_info(): "file_text": { "description": "`create` 或 `edit` 命令的必需参数,

↳ .",

↳ 包含整个文件的内容。

"type": "string"

+ },

+ "view_range": { + "描述": "`view` 命令的可选参数, 用于显示特定行范围

↳ 我们提出一种新颖的框架, 用于从单个图像中恢复三维人体网格。现有方法大多依赖参数化模型, 如统计身体模型 (Statistical Body Model) 或三维人体模板。然而, 由于网络特征与参数化模型之间的域差距, 这些方法的二维图像对齐精度有限, 导致恢复的网格出现

+ "类型": "数组", + "项目": { "类型": "整数" }

+ },

+ "old_str": { + "description": "`str\_replace` 命令的必需参数, 要替换的字符串。", + "type": "string"

+ },

+ "new_str": {

```

+ "description": `str_replace` 和 `insert` 命令的必需参数, 新字符串
  ↳ 插入。
+ "类型": "字符串"
+ },
+ "insert_line": { + "描述": "`insert` 命令的必需参数, 用于指定插入 文本的行号。", + "类型": "整数" }, "必需": ["命令", "路
径"] @@ -43,7 +64,7 @@ def validate_path(path: str, command: str) -> Path: 验证每个命令的文件路径: - `view`: 路径可以是文件或
目录; 必须存在。

- "创建": 路径必须不存在 (用于新建文件)。
-- 'edit': 路径必须存在 (用于覆盖)。
+ - 其他命令: 路径必须作为文件存在。""" path_obj =
Path(path)

@@ -61,14 +82,12 @@ def validate_path(path: str, command: str) -> Path:
    # Path must not exist
    if path_obj.exists():
        raise ValueError(f"Cannot create new file; {path} already exists.")

- elif command == "edit": + else:

    # 如果不是 path_obj.exists():, 路径必须存在且必须是文件 raise ValueError(f"文件 {path} 不存在。") if
    path_obj.is_dir(): raise ValueError(f"{path} 是目录, 不能作为文件编辑。")

- 否则:
- raise ValueError(f"未知或不支持的命令: {command}")

    return path_obj

@@ -89,14 +108,21 @@ def read_file(path: Path) -> str:
    except Exception as e:
        raise ValueError(f"Failed to read file: {e}")

-def write_文件 (路径: 路径, 内容: 字符串): +定义 写入_文件 (路径: 路径, 内容: 字符串, 保存
_history: bool = True): """写入 (覆盖) 整个文件内容。""" try: + if save_history: + # 写入前将当前内容保
存到历史记录 + 如果 路径.存在(): + 如果 字符串(路径) 不在 编辑_history: + edit_history[字符串(路径)]
= [] + edit_历史[字符串(路径)].追加(路径.读取_text()) + path.write_文本(内容) 除了 异常 作为 错误: 抛
出 值错误(格式字符串"写入文件失败: {e}")

-def view_path(path_obj: Path) -> str: +def view_path(path_obj: Path, view_range: Optional[List[int]] = None) -> str: """查
看整个文件内容或目录列表。""" if path_obj.is_dir(): # 对于目录: 列出最多两层的非隐藏文件 @@ -115,35 +141,104
@@ def view_path(path_obj: Path) -> str: except Exception as e: raise ValueError(f"列出目录失败: {e}")

- # 如果是文件, 显示带行号的整个文件 + # 如果是文件内容 = read_file(path_obj) + lines = content.splitlines() + # 处理行范围查看 +
if view_range: + if len(view_range) != 2: + raise ValueError("view_range 必须恰好包含两个元素: [start, end]") + start, end = view_range + if
start < 1 or end > len(lines): + raise ValueError(f"行范围 [{start}, {end}] 对于具有 {len(lines)} 行的文件无效") + # 调整为基于 0 的索引 +
content = "\n".join(lines[start-1:end]) + return format_output(content, str(path_obj), init_line=start) + return format_output(content,
str(path_obj))

```



```
-def tool_函数 (命令: 字符串, 路径: 字符串, 文件_text: str = None) -> str:
+def str_replace(path_obj: Path, old_str: str, new_str: str) -> str:
+ """替换文件中的字符串, 确保唯一性。"""
+ content = read_file(path_obj) + # 检查多次出现的情况
+ if content.count(old_str) > 1:
+ return f"错误: 发现多处 '{old_str}'。替换需要唯一的 匹配。"
+ elif content.count(old_str) == 0:
+ return f"错误: 文件中未找到字符串 '{old_str}'。"
+ new_content = content.replace(old_str, new_str)
+ write_file(path_obj, new_content)
+ return f"位于 {path_obj} 的文件已被编辑: 将 '{old_str}' 替换为 '{new_str}'。"
+def insert_text(path_obj: Path, insert_line: int, new_str: str) -> str:
+ """在指定行号插入文本。"""
+ content = read_file(path_obj)
+ lines = content.splitlines()
+ if insert_line < 1 or insert_line > len(lines) + 1:
+ raise ValueError(f"无效的插入行 {insert_line}, 文件共有 {len(lines)} 行")
+ # 在指定行插入新文本 (调整为基于 0 的索引)
+ lines.insert(insert_line - 1, new_str.rstrip("\n"))
+ new_content = "\n".join(lines) + "\n"
+ write_file(path_obj, new_content)
+ return f"位于 {path_obj} 的文件已被编辑: 在第 {insert_line} 行插入文本"
+def undo_edit(path_obj: Path) -> str:
+ """撤销文件上的最后一次编辑操作。"""
+ 路径字符串 = str(path_obj)
+ if 路径字符串 not in edit_history or not edit_history[路径字符串]:
+ return "错误: 此文件没有可用的编辑历史。"
+ # 恢复上次保存的内容
+ previous_content = edit_history[路径字符串].pop()
+ write_file(path_obj, previous_content, save_history=False)
+ return f"对 {path_obj} 的最后一次编辑已成功撤销。"
+def tool_函数 (命令: 字符串, 路径: 字符串, **kwargs) -> str:
+ """主工具函数, 处理: --'view': 查看整个文件或目录列表 --'create': 使用给定的 file_text 创建新文件 --'edit': 用 file_text 覆盖现有文件 + 主工具函数, 处理所有命令: + -'view': 查看文件/目录 (可选行范围) + -'create': 创建新文件 + -'edit': 覆盖现有文件 + -'str_replace': 替换文件中的字符串 + -'insert': 在行插入文本 + -'undo_edit': 撤销最后一次编辑 """
+ try:
+ path_obj = validate_path(path, command)
```

如果命令 == "view":

```
- return view_path(path_obj) + view_range = kwargs.get('view_range') + return view_path(path_obj, view_range)
```

elif command == "create":

- 如果 file_text 为无: + 如果 'file_text' 不在关键字参数中:

```
raise ValueError("缺少 'create' 命令所需的 'file_text'。")
```

```
- write_file(path_obj, file_text)
```

```
+ write_f
```

```
return f"文件已成功创建于: {path}"
```

elif command == "edit":

- 如果 file_text 为无: + 如果 'file_text' 不在关键字参数中:

```
raise ValueError("缺少 'edit' 命令所需的 'file_text'。")
```

```
- write_file(path_obj, file_text)
```

```
+ write_f
```

```
返回 f"文件 {path} 已被新内容覆盖。"
```

```
+ elif 命令 == "str_replace":
```

```
+ if 'old_str' 不在 kwargs 中或 'new_str' 不在 kwargs 中: + raise ValueError("字符串替换需要 'old_str' 和 'new_str'。") + return
str_replace(path_obj, kwargs['old_str'], kwargs['new_str']) + elif 命令 == "insert": + if 'insert_line' 不在 kwargs 中或 'new_str' 不在
kwargs 中: + raise ValueError("文本插入需要 'insert_line' 和 'new_str'。") + return insert_text(path_obj, kwargs['insert_line'],
kwargs['new_str']) + elif 命令 == "undo_edit": + return undo_edit(path_obj) + else: raise ValueError(f"未知命令: {command}")
```

差异 --git a/工具/编辑.py b/工具/编辑.py 索引 87fa408..a12903c 100644 --- a/工具/编辑.py +++ b/工具/编辑.py @@ -117,7 +117,7
@@ 定义 write_file(path: 路径, 内容: 字符串, save_history: 布尔值 = 真): 如果字符串 (路径) 不在 edit_history[字符串
(路径)] = [] edit_历史记录 [字符串 (路径)].追加 (路径.读取_text()) -

```
+
path.write_text(content) except Exception as e: raise ValueError(f"写入文件失败: {e}") @@ -172,17 +172,25 @@ def
str_replace(path_obj: Path, old_str: str, new_str: str) -> str: return f"文件 {path_obj} 已被编辑: 将 '{old_str}' 替换为 '{new_str}'."
```

def insert_text(path_obj: Path, insert_line: int, new_str: str) -> str:

- """"在指定行号插入文本。"""" + """"在指定行号 (从 1 开始) 插入文本。"""" content = read_file(path_obj) lines = content.splitlines()

- 如果 insert_line < 1 或 insert_line > 行数 + 1:

- 抛出值错误 (ValueError) 异常, 信息为: "无效的插入行 {insert_line}, 文件共有 {行数} 行" + # 验证插入行号 + 如果 insert_line < 1: + 抛出值错误 (ValueError) 异常, 信息为: "无效的插入行 {insert_line} - 必须大于 0" + 如果 insert_line > 行数 + 1: + 抛出值错误 (ValueError) 异常, 信息为: "无效的插入行 {insert_line} - 文件仅有 {行数} 行"

- # 在指定行插入新文本 (根据从 0 开始的索引进行调整)

- new_content = "\n".join(lines) + "\n" + # 清理新文本以移除任何末尾换行符 + new_text = new_str.rstrip("\n")

- lines.insert(insert_line - 1, new_str.rstrip("\n"))

+ # 在正确位置插入 (行号从 1 开始, 列表索引从 0 开始) + # 在 index=insert_line, 处插入, 使其出现在该位置当前行之后 + lines.insert(insert_line, new_text)

+ # 用换行符连接各行并添加末尾换行符 + new_content = '\n'.join(lines) + '\n' write_file(path_obj, new_content) 返回 f"文件位于 {path_obj} 已被编辑: 在第 {insert_line}." 行插入了文本"

@@ -243,8 +251,4 @@ def tool_function(command: str, path: str, **kwargs) -> str: raise ValueError(f"Unknown command: {command}")

除了异常作为错误:

- 返回 f"错误: {字符串表示的错误}" - -如果 __name__ == "__main__":

- # 示例用法

- print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py")) \ 文件末尾无换行符 + 返回 f"错误: {str(e)}" \ 文件末尾无换行符

差异 --git a/coding_agent.py b/coding_agent.py 索引 6639abd..97f4b69
100644 --- a/coding_agent.py +++ b/coding_agent.py @@ -52,6 +52,10
@@ 类 SolutionAttempt: test_output: 字符串 # 原始测试输出

test_success: 布尔值 # 测试是否通过 test_stats: 字典 # 测试统计信息 (例如, 通过/失败测试的数量) + error_messages: 字符串列表 = 无
具体错误消息列表 + test_details: 字典 = 无 # 详细测试信息, 如具体测试名称及其  状态 + execution_time: 浮点数 = 无 # 测试执行时间 (秒) + attempt_number: 整数 = 无 # 序列中的尝试编号

定义 get_thread_logger(): """ @@ -150,12 +154,82 @@ 类
AgenticSystem: } 返回 new_msg_history

```
+ def extract_test_details(self, output: str) -> tuple[dict, List[str], dict]: + """从输出中提取详细的测试信息。""" + error_messages = [] +
test_details = {} + stats = {"passed": 0, "failed": 0, "errors": 0, "total": 0, "skipped": 0} + # 将输出拆分为行以便分析 + lines =
output.split("\n") + # 语言特定的解析 + if self.language == "python": + for line in lines: + if "FAILED" in line and ":" in line: +
test_name = line.split(":")[1].split()[0] + test_details[test_name] = "FAILED" + stats["failed"] += 1 + elif "PASSED" in line and ":" in
line: + test_name = line.split(":")[1].split()[0] + test_details[test_name] = "PASSED" + stats["passed"] += 1 + elif "ERROR" in line and
":" in line: + test_name = line.split(":")[1].split()[0] + test_details[test_name] = "ERROR" + stats["errors"] += 1 + # 提取错误消息 + if
lines.index(line) + 1 < len(lines): + error_messages.append(lines[lines.index(line) + 1]) + elif self.language in ["javascript", "node"]: +
current_test = None + for line in lines: + if line.startswith('checkmark'): + test_name = line.replace('checkmark,', '').strip() +
test_details[test_name] = "PASSED" + stats["passed"] += 1 + elif line.startswith('x'): + test_name = line.replace('x, ').strip() +
test_details[test_name] = "FAILED" + stats["failed"] += 1 + current_test = test_name + elif current_test and ('Error:' in line or
'AssertionError:' in line): + error_messages.append(f'{current_test}: {line.strip()}') + elif self.language == "rust": + for line in lines: +
if "test" in line and "... ok" in line: + test_name = line.split("test")[1].split("...")[0].strip() + test_details[test_name] = "PASSED" +
stats["passed"] += 1 + elif "test" in line and "... FAILED" in line: + test_name = line.split("test")[1].split("...")[0].strip() +
test_details[test_name] = "FAILED" + stats["failed"] += 1 + elif "----" in line and " stdout ----" in line: + test_name =
line.split("----")[1].split("stdout")[0].strip() + if test_name in test_details and test_details[test_name] == "FAILED": +
error_messages.append(f'{test_name}: {next((l for l in lines[lines.index(line)+1:] if l.strip()), "")}')
+
+
+ # 其他语言或作为后备的通用计数 + 如果没有任何统计值: + 统计 ["通过"] = 输出中"通过"的计
数 + 输出中"正常"的计数 + 统计 ["失败"] = 输出中"失败"的计数 + 输出中"不正常"的计数 + 统计 ["
错误"] = 输出中"错误"的计数 + 输出中"恐慌:")
```

```
+
+ # 其他语言或作为后备的通用计数 + 如果没有任何统计值: + 统计 ["通过"] = 输出中"通过"的计
数 + 输出中"正常"的计数 + 统计 ["失败"] = 输出中"失败"的计数 + 输出中"不正常"的计数 + 统计 ["
错误"] = 输出中"错误"的计数 + 输出中"恐慌:")
```

```
+
+ stats["total"] = stats["passed"] + stats["failed"] + stats["errors"]
+
```

```

+ return stats, error_messages, test_details
+
def run_tests(self) -> tuple[布尔值, 字符串, 字典]: """运行测试并返回成功状态、输出和测试统计信息。""" + import
time

+
    success = False output
    = ""

- 统计信息 = {"通过": 0, "失败": 0, "错误": 0, "总计": 0} - + start_time = 时间.时间 ()

+
    尝试:
    对于命令在 TEST_COMMANDS.get(self.language, []): 进程 = 子进程.运行 ( @@ -169,34 +243,97 @@ 类 智能体系统: 成
    功 = 进程.返回码 == 0 如果不成功: 中断 - - # 尝试从输出中提取测试统计信息 - # 这是一个简单示例; 您可能希望添
    加更复杂的解析 - 统计信息 ["通过"] = 输出.计数 ("通过") + 输出.计数 ("确定") - 统计信息 ["失败"] = 输出.计数 ("失败
    ") + 输出.计数 ("不确定") - 统计信息 ["错误"] = 输出.计数 ("错误") + 输出.计数 ("恐慌") - 统计信息 ["总计"] = 统计信
    息 ["通过"] + 统计信息 ["失败"] + 统计信息 ["错误"] + # 提取详细的测试信息 + 统计信息, error_messages,
    test_details = self.extract_test_details(output) + stats["execution_time"] = 时间.时间 () - start_time + # 使用提取的信
    息增强统计信息 + stats["error_messages"] = error_messages + stats["test_details"] = test_details

除了异常作为 异常: 输出 = f"运行测试时出错: {字符串 (异常)}" 成功 = 假 +
统计信息 = { + "通过": 0, "失败": 0, "错误": 1, "总计": 1, + "execution_time": 时
间.时间 () - start_time, + "error_messages": [字符串 (异常)], + "test_details": {}

+ }

    返回 成功, 输出, 统计信息

    定义 analyze_test_results(self, 尝试次数: 列表 [解决方案尝试]) -> 字符串:

- """分析测试结果并为智能体创建摘要。""" + """分析测试结果并为智能体创建详细摘要。"""

    摘要 = "# 测试结果 Analysis\n\n"

+ # 总体进度跟踪 + if len(attempts) > 1: + summary += "## 进度 Overview\n" + first_attempt = attempts[0].test_stats + last_attempt =
attempts[-1].test_stats + progress = { + "通过": last_attempt["通过"] - first_attempt["通过"], + "失败": first_attempt["失败"] -
last_attempt["失败"], + "错误": first_attempt["错误"] - last_attempt["错误"] + } + summary += "自首次以来的进度 attempt:\n" +
summary += f"- 额外通过的测试: {progress['通过']}\n" + summary += f"- 减少的失败: {progress['失败']}\n" + summary += f"- 减少
的错误: {progress['错误']}\n\n" + # 详细的尝试分析 对于 i, 尝试次数, 枚举 (尝试次数, 1): summary += f"## 尝试 {i}\n"
summary += f"测试成功: {attempt.test_success}\n" - summary += f"测试统计: {json.dumps(attempt.test_stats, indent=2)}\n" -
summary += "关键测试输出: \n" + "\n" - # 提取测试输出的相关部分 (例如错误消息) - key_output = "\n".join(尝试中的每一行
.test_output.split("\n") - 如果行中包含"失败"或"错误"或"通过") - 摘要 += f"{key_output}\n" + "\n" - + summary += f"执行时间:
{attempt.test_stats.get('execution_time', 'N/A'):.2f}s\n"

```

```

+ # 测试统计 + 统计信息 = attempt.test_stats + 总计 = 统计信息.get("total", 0) 或 1 # 避免除以零 + pass_rate = (统计信息
.get("passed", 0) / 总计) * 100 + # 摘要 += f"通过率: {pass_rate:.1f}% ({统计信息.get('passed', 0)}/{total})\n" + 摘要 += "测试
Statistics:\n" + 摘要 += f"- 通过: {统计信息.get('passed', 0)}\n" + 摘要 += f"- 失败: {统计信息.get('failed', 0)}\n" + 摘要 += f"-
错误: {统计信息.get('errors', 0)}\n" + 摘要 += f"- 总计: {total}\n\n" + # 错误消息 + 如果 stats.get("错误消息"): + 摘要 += "
错误消息: \n```\n" + 对于 stats["错误消息"][:5]: 中的错误 # 限制为前 5 个错误 + 摘要 += f"{error}\n" + 如果 len(stats["错误
消息"]) > 5: + 摘要 += f"... 以及 {len(stats['错误消息']) - 5} 更多 errors\n" + 摘要 += "```\n\n" + # 测试详情 + 如果
stats.get("测试详情"): + 摘要 += "单个测试结果: \n```\n" + 对于 test_name, 结果在 stats["测试详情"].items(): 中 + 摘要 +=
f"{test_name}\n" + 摘要 += "```\n\n" + # 下次尝试的建议 + 如果不 attempts[-1].test_success: + 摘要 += "## 下
次 Attempt\n" 的建议 + last_stats = attempts[-1].test_stats + 如果 last_stats.get("errors", 0) > 0: + 摘要 += "- 专注于解决运行
时错误 first\n" + 如果 last_stats.get("failed", 0) > 0: + 摘要 += "- 解决失败的测试 cases\n" + 如果 len(尝试次数) > 1 且不
attempts[-1].test_success: + # 与之前的尝试进行比较 + prev_stats = attempts[-2].test_stats + 如果 last_stats.get("passed", 0) <
prev_stats.get("passed", 0): + 摘要 += "- 最近的更改导致了回归。考虑回退一些 changes\n" + 返回 摘要

def forward(self):
@@ -238,20 +375,36 @@ Your task is to make changes to the files in the {self.git_tempdir}
    ↳ directory to
        # Run tests and collect results
        test_success, test_output, test_stats = self.run_tests()

- # 创建并存储尝试 + # 创建并存储带有增强信息的尝试

        尝试 = 解决方案尝试(
            patch=current_patch,
            test_output=test_output,
            test_success=test_success,
- test_stats=test_stats + test_stats=test_stats, + error_messages=test_stats.get('error_messages', []), +
test_details=test_stats.get('test_details', {}), + execution_time=test_stats.get('execution_time', 无), + attempt_number=attempt_num +
1

        ) 尝试列表追加(尝试)

- # 如果本次更好则更新最佳尝试
- 如果 test_success 且 (best_attempt 是 无 或
- attempt.test_stats["通过"] > best_attempt.test_stats["通过"]): + # 基于多个标准更新最佳尝试 + 如果 test_success 且 ( +
best_attempt 是 无 或 (attempt.test_stats["通过"] > best_attempt.test_stats["通过"]) 或 (attempt.test_stats["通过"] ==
best_attempt.test_stats["通过"] 且 len(attempt.error_messages 或 []) < len(best_attempt.error_messages 或 []))

+ ):
        best_attempt = 尝试

+ # 记录详细的尝试信息 + safe_log(f"\n=== 尝试 {attempt_num + 1} 摘要 ===")

```

```
+ safe_log(f"Test 成功: {test_success}") + safe_log(f"Tests 通过: {test_stats.get('passed', 0)}") + safe_log(f"Tests 失败: {test_stats.get('failed', 0)}") + safe_log(f"Errors: {test_stats.get('errors', 0)}") + safe_log(f"Execution 时间: {test_stats.get('execution_time', '不适用'):.2f}秒")

+
    # 如果测试完全通过，我们可以停止 如果 test_success 且 attempt.test_stats["失败"] == 0 且
    ↪ attempt.test_stats["errors"] == 0:
        中断
```

G 相似目标功能，不同实现

对于相同的目标功能，提供给编码智能体的建议以及由此产生的实现可能存在显著差异，从而导致编码能力和基准性能出现较大波动。例如，考虑在 SWE-bench 上运行的 DGM 中的节点 6 和节点 24（图 3）。两者都旨在增强现有编辑器工具以支持更细粒度的文件编辑。然而，节点 6 的实现保留了原始编辑命令并添加了参数 `edit_type` 和 `edit_actions`，而节点 24 的实现则用新的 `str_replace` 命令替换了原始编辑命令。尽管目标功能相似，但功能建议以及实现细节差异很大，其性能也大相径庭。节点 6 仅达到 23.3%，而节点 24 达到 40.5%。

获取节点 6 的功能建议：

在 `tools/edit.py` 文件中，移除“`edit`”命令并引入一个 `'str_replace'` 命令。它应接受参数 `'old_str'`（用于查找的唯一文本）和 `'new_str'`（用于替换的新文本）。在验证 `'old_str'` 在文件中恰好出现一次后，将第一次出现替换为 `'new_str'`。如果 `'old_str'` 不唯一或缺失，则不写入任何更改。

将编辑器工具中的“`edit`”命令替换为一个 `'str_replace'` 命令，该命令定位并替换文件中的精确子串。`'old_str'` 必须唯一匹配连续的行，如果未找到或出现多次，则不应发生写入。`'new_str'` 参数包含修订后的行。这种精确的子串替换流程将实现小规模增量编辑，同时将破坏风险降至最低。

获取节点 24 的功能建议：

增强现有的“编辑器”工具，以支持基于行或基于标记的编辑。代理将指定要插入、删除或替换的单个行或标记。在内部，工具将解析文件内容，原子地应用更改，并验证周围上下文保持不变。这可以防止整个文件被覆盖，从而降低丢失无关逻辑的风险。该工具还可以验证编辑后的行是否与预期的代码片段匹配，以捕获意外差异或合并冲突。

增强代理的文件编辑过程，以支持细粒度编辑。具体来说，在“编辑器”工具中添加基于行或基于标记的编辑功能。此改进应允许代理插入、替换或删除小段代码，而无需覆盖整个文件或大块代码，从而保留无关逻辑。实施保护措施，以确认编辑区域周围的上下文代码保持准确，从而降低在修复问题时引入回归的风险。

获取节点 6 的建议功能实现：

```
diff --git a/tests/test_edit_tool.py b/tests/test_edit_tool.py index f0782c6..d3a4d5c 100644 ---
a/tests/test_edit_tool.py +++ b/tests/test_edit_tool.py

@@ -1,7 +1,7 @@
import pytest from pathlib import Path import tempfile -from tools.edit import tool_function +from tools.edit import
tool_function, validate_edit_actions, validate_context

@pytest.fixture
def temp_dir():
@@ -17,6 +17,18 @@ def sample_file(temp_dir):
    file_path.write_text(content) 返回 file_path
```

```

+ @pytest.fixture + def code_file(temp_dir): + """创建一个用于测试基于令牌编辑的示例 Python 文件。""" + file_path = temp_dir / "test.py" + 内容 = """def test_function():

+ x = 42
+ y = "old_value" + 返回 x +
+ """ + len(y)
+ file_path.write_text(content) + 返回 file_path
+
+ class TestEditorTool: def test_view_file(self, sample_file): """测试查看整个文件内容。""" @@ -39,17 +51,182 @@ class TestEditorTool: 断言 "Error" 在结果中 断言 "already exists" 在结果中

- 定义 test_edit_file(self, sample_file):
- """测试编辑现有文件。""" + def test_edit_file_full(self, sample_file): + """测试使用文件模式编辑现有文件。"""
+
+         new_content = "编辑了 content\nnew 行"
+ 结果 = tool_function("edit", str(sample_file), file_text=new_content) + 结果 = tool_function( + "编辑", + str(sample_file), + file_text=new_content, + edit_type="file"

+ )
+         断言 "已被覆盖" 在结果中 断言 sample_file.read_text() == new_content

+ 定义 test_line_based_edit_insert(self, sample_file): + """测试基于行的插入操作。""" + 操作 = [{ + "操作": "插入", + "line_number": 3, + "内容": "插入的行"

+ }]
+ 结果 = tool_function( + "编辑", + str(sample_file), + edit_type="line", + edit_actions=actions

+ )
+ 断言 "成功编辑" 在结果中 + 内容 = sample_file.read_text().splitlines()
+ 断言 内容 [2] == "插入的行" + 断言 长度 (内容) == 6

+
+ 定义 test_line_based_edit_delete(self, sample_file): + """测试基于行的删除操作。""" + 操作 = [{ + "操作": "删除", + "line_number": 2

+ }]
+ 结果 = tool_function( + "编辑", + str(sample_file), + edit_type="line", + edit_actions=actions

+ )
+ 断言 "成功编辑" 在结果中 + 内容 = sample_file.read_text().splitlines()
+ 断言 "第 2 行" 不在内容中 + 断言 长度 (内容) == 4

+
+ 定义 test_line_based_edit_replace(self, sample_file): + """测试基于行的替换操作。""" + 操作 = [{ + "操作": "替换", + "line_number": 3, + "内容": "替换的行"

+ }]
+ 结果 = tool_function( + "编辑", + str(sample_file), + edit_type="line", + edit_actions=actions

```

```

+ )
+ 断言 "successfully edited" 在 result + content =
sample_file.read_text().splitlines() + 断言 content[2] == "replaced line" +
断言 len(content) == 5
+
+ def test_token_based_edit(self, code_file): + """测试基于令牌的编
辑操作。""" + actions = [{ + "action": "replace", + "old_token":
"old_value", + "new_token": "new_value"

+ }]
+ 结果 = tool_function( + "编辑", +
str(code_file), + edit_type="token", +
edit_actions=actions

+ )
+ 断言 "成功编辑" 在 结果 中 + 内容 = code_file.read_text()
+ 断言 "old_value" 不在 内容 中 + 断言 "new_value" 在 内容
中
+
+ 定义 test_multiple_token_edits(self, code_file): + """测试多个基于标记的编
辑操作。""" + 操作 = [

+ {
+ "操作": "替换", + "old_token":
"42", + "new_token": "100"

+ },
+ {
+ "action": "replace", + "old_token":
"old_value", + "new_token": "new_text"

+ }
+ ]
+ 结果 = tool_function( + "编辑", +
str(code_file), + edit_type="token", +
edit_actions=actions

+ )
+ 断言 "成功编辑" 在 结果 中 + 内容 = code_file.read_text()
+ 断言 "42" 不在 内容 中 + 断言 "100" 在 内容 中 + 断言
"old_value" 不在 内容 中 + 断言 "new_text" 在 内容 中

+
+ def test_invalid_line_number(self, sample_file): + """测试无效行号的编
辑。""" + actions = [{ + "action": "replace", + "line_number": 100, +
"content": "invalid line"

+ }]
+ 结果 = tool_function( + "编辑", +
str(sample_file), + edit_type="line", +
edit_actions=actions

+ )
+ 断言 "错误" 在结果中 + 断言 "超出文件长度" 在结果中
+
+ 定义 test_invalid_edit_type(self, sample_file): + """测试无效编辑类型的
编辑。""" + 结果 = tool_function( + "编辑", + str(sample_file), +
edit_type="invalid", + edit_actions=[{"action": "替换"

+ })
+ 断言 "错误" 在结果中 + 断言 "无效的 edit_type" 在结
果中
+
+ 定义 test_missing_required_fields(self, sample_file): + """测试缺少必需字段的编
辑。""" + 操作 = [{

```



```
+ "操作": "替换" # 缺少 line_number 和内容 + }) + 结果 = tool_function( + "编辑", + str(sample_file), + edit_type="line", +
edit_actions=actions + }) + 断言 "错误" 在结果中 + 断言 "缺少必需字段" 在结果中 + + 定义 test_validate_context(self: + ""测试上下文
验证函数。"" + 原始 = "行 1\nline 2\nline 3\nline 4\nline 5" + 修改 = "行 1\nline 2\nmodified\nline 4\nline 5" + 断言
validate_context(original, 修改) 为真 + + # 测试无效上下文 (更改过多) + 修改 = "completely\ndifferent\ncontent" + 断言
validate_context(original, 修改) 为假 + + 定义 test_empty_edit_actions(self, sample_file): + ""测试编辑操作为空。"" + 结果 =
tool_function( + "编辑", + str(sample_file), + edit_type="line", + edit_actions=[] + }) + 断言 "错误" 在结果中 + 断言 "编辑操作不能为空"
在结果中 + + 定义 test_edit_nonexistent_file(self, temp_dir): ""测试尝试编辑不存在的文件。"" non_existent_file = temp_dir /
"does_not_exist.txt" - 结果 = tool_function("edit", str(non_existent_file), file_text="new 内容") + 结果 = tool_function("edit",
str(non_existent_file), edit_type="line", edit_actions=[]) 断言 "错误" 在结果中 断言 "不存在" 在结果中
```

```
@@@ -82,4 +259,4 @@ 类 TestEditorTool: 定义 test_invalid_commands(self, 命令,
sample_file): ""测试各种无效命令。"" 结果 = tool_function(command, str(sample_file))
- 断言 "错误" 在结果中 + 断言 "错误" 在结果中 \ 文件末尾无换行符 差异 --git
a/tools/edit.py b/tools/edit.py 索引 59137ec..982ee84 100644 --- a/tools/edit.py +++
b/tools/edit.py @@ -1,5 +1,7 @@ 从 pathlib 导入 Path 导入 subprocess 从 typing
导入 Dict, Any, List, Union, Tuple 从 difflib 导入 SequenceMatcher
```

定义 tool_info(): 返回 { @@ -9,8 +11,9 @@ 定义 tool_info(): 如果 路径 是文件, 查看 显示整个文件并带行号。如果 路径 是 目录, 查看 列出最多 2 层的非隐藏文件和目录 deep.\n 该 创建 命令不能在指定的 路径 已作为 * file.\n 存在时使用 * If a 命令 生成输出, 它将被截断并标记为 <响应 裁剪> .\n -* 编辑 命令用提供的 文件文本 .\n 覆盖整个文件 -* 不支持部分/行范围编辑或部分查看。"" , +* 编辑 命令支持整个文件覆盖和细粒度的行/标记

通过 `edit\_type` parameter.\n 进行编辑
+* 基于行的编辑需要行号和内容来修改 file.\n 的特定部分 +* 基于标记的编辑需要指定旧标记和新标记来替换特定文本 片段。
"" , "input_schema": { "type": "object", "properties": { @@ -24,8 +27,28 @@ def tool_info(): "type": "string" }, "file_text": { - "description":
"create 或 edit 命令的必需参数, 包含整个文件的 内容。",

```

+ "description": "`create` 或 `edit` 命令的必需参数, 带有 edit_type='file',  包含整个文件的内容。", "type": "string"
+ },
+ "edit_type": { + "类型": "字符串", + "枚举": ["文件", "行", "标记"], + "描述": "编辑操作的类型: '文件'表示整个文件, '行'表示基于行的"
+ "编辑, 'token' 表示基于标记的编辑。",
+ "default": "file"
+ },
+ "edit_actions": { + "类型": "数组", + "描述": "行/词元操作的编辑动作列表。每个动作包含"
+ "操作详情。",
+ "items": { + "type": "object", + "properties": { + "action": { "type": "string", "enum": ["insert", "delete", "replace"] }, + "line_number": { "type":
"integer", "description": "操作的行号  (从 1 开始)", + "content": { "type": "string", "description": "要插入/替换的内容", + "old_token":
{ "type": "string", "description": "要替换的标记 (用于标记编辑)", + "new_token": { "type": "string", "description": "新标记 (用于标记编辑)" } }
+ }
+ }
+ },
+ "required": ["command", "path"]
@@ -119,12 +142,126 @@ def view_path(path_obj: Path) -> str:
    content = read_file(path_obj)
    return format_output(content, str(path_obj))

-def tool_函数 (命令: 字符串, 路径: 字符串, 文件_text: 字符串 = None) -> 字符串:
+def validate_edit_动作 (动作: 字典列表[
字符串, 任意], 编辑_type: 字符串) -> None:
+ """根据编辑类型验证编辑操作。"""
+ 如果没有动作: + 引发 ValueError("行/词元编辑的操作不能为空")
+  + valid_actions = ["插入", "删除", "替换"]
+  + required_fields = { + "line": ["操作", "行号"], + "token": ["操作", "旧令牌"]
+  + }
+  + 对于动作中的每个动作:
+  + 如果"动作"不在动作中或动作["动作"]不在有效_actions:
+  + raise ValueError(f"无效操作。必须为以下之一: {valid_actions}")
+  + # 根据 编辑类型 检查必填字段
+  + for field in required_fields[edit_类型]:
+  + 如果字段不在动作中:
+  + 引发 ValueError(f"编辑操作中缺少必填字段 '{field}'")
+  + # 若提供了行号则验证
+  + if "line_数字"在动作中:
+  + 如果不是实例 (动作["行_数字"], 整数) 或动作["行_number"] < 1:
+  + raise ValueError("行号必须为正整数")
+  + # 验证内容要求
+  + 如果动作["动作"]在["插入", "替换"]中:
+  + 如果编辑_type == "行"和"内容"不在操作中:
+  + 引发 ValueError("插入/替换操作需要内容")
+  + 如果编辑_type == "token" and "new_令牌"不在操作中:
+  + 引发 ValueError("new_token 词元操作需要")
+  + def apply_line_编辑 (内容: 字符串列表, 操作: 字典列表[字符串, 任意]) -> List[字符串]:
+  + """对内容应用基于行的编辑。"""
+  + modified_content = content.copy()
+  + # 按行号逆序排序操作, 以正确处理插入/删除
+  + sorted_actions = sorted(actions, key=lambda x: x["行号"], reverse=True)
+  + for action in sorted_actions:
+  + line_num = action["行号"] - 1 # 转换为从 0 开始的索引
+  + if line_num > len(modified_content):
+  + raise ValueError(f"行号 {action['行号']} 超出文件长度")
+  + if action["操作"] == "delete":
+  + if line_num >= 0:

```

```

+ del modified_content[line_num] + elif action["操作"] == "插入": + modified_content.insert(line_num, action["内容"]) + elif
action["操作"] == "替换": + modified_content[line_num] = action["内容"] + + return modified_content + +def
apply_token_edits(content: str, actions: List[Dict[str, Any]]) -> str: + """对内容应用基于词元的编辑。""" + modified_content =
content + + for action in actions: + old_token = action["old_token"] + if action["操作"] == "删除": + modified_content =
modified_content.replace(old_token, "") + elif action["操作"] in ["插入", "替换"]: + new_token = action["new_token"] +
modified_content = modified_content.replace(old_token, new_token) + + return modified_content + +def
validate_context(original: str, modified: str, context_lines: int = 3) -> bool: """ - 主工具函数，处理: + 验证修改部分周围的上下文
是否保持完整。

+ """
+ 如果上下文有效则返回 True，否则返回 False。
+ """
+ # 使用 SequenceMatcher 计算字符串之间的相似度比率 + similarity = 序列匹配器 (
SequenceMatcher) 在无参数设置下对原始文本与修改文本的相似度比值 ()
+
+ # 如果内容差异过大 (相似度低于 30%)，则视为无效 + if similarity < 0.3: + return False
+
+ # 拆分为行以便进行更详细的分析 + orig_lines = original.splitlines()
+ mod_lines = modified.splitlines()
+
+ # 如果行数差异过大 (超过 50%)，则视为无效 + if abs(len(orig_lines) - len(mod_lines)) > len(orig_lines) // 2: +
return False
+
+ # Calculate line-based similarity for sections
+ def get_block_similarity(block1: List[str], block2: List[str]) -> float:
+ return SequenceMatcher(None, "\n".join(block1), "\n".join(block2)).ratio()
+
+ # 如果存在起始和结束块，则检查它们的相似度 + if len(orig_lines) >= context_lines and len(mod_lines) >=
context_lines: + start_similarity = get_block_similarity(+ orig_lines[:context_lines], + mod_lines[:context_lines]
+
+ )
+ end_similarity = get_block_similarity(
+ orig_lines[-context_lines:],
+ mod_lines[-context_lines:]
+ )
+
+ # 如果起始或结束块差异过大，则视为无效 + if start_similarity < 0.7 or end_similarity < 0.7: + return False
+
+ + return True
+
+def tool_function(command: str, path: str, file_text: str = None, + edit_type: str = "file", edit_actions: List[Dict[str, Any]]
+ """ = None) -> str:
+ """
+ 增强的工具函数，处理:
+ - 'view': 查看整个文件或目录列表
+ - 'create': 使用给定的 file_text 创建新文件
+ - 'edit': 使用 file_text 覆盖现有文件 + - 'edit': 使用三种模式之一编辑文件: + - file: 覆盖整个文件 + - line: 基
于行的编辑 + - token: 基于标记的编辑
+
+ """
+ try:
+ path_obj = validate_path(path, command)
@@ -139,11 +276,36 @@ def tool_function(command: str, path: str, file_text: str = None) ->
+ str:
+     return f"File created successfully at: {path}"

```

```

elif 命令 == "edit": - 如果 file_text 为 None: - 抛出 ValueError("缺少'edit'命令所需的`file_text`。") - write_file(path_obj,
file_text) - 返回 f'路径 {path} 处的文件已被新内容覆盖。" + # 读取原始内容以进行行/标记编辑和验证 + original_content =
read_file(path_obj) 如果 edit_type 在 ["line", "token"] 中, 否则为 None + 如果 edit_type == "file": + 如果 file_text 为 None:
+ 抛出 ValueError("缺少'file'编辑类型所需的`file_text`。") + write_file(path_obj, file_text) + 返回 f'路径 {path} 处的文件已被新
内容覆盖。" + elif edit_type 在 ["line", "token"] 中: + 如果不是 edit_actions: 抛出 ValueError("行/标记编辑的编辑操作不
能为空") + validate_edit_actions(edit_actions, edit_type) + 如果 edit_type == "line": + 行 = original_content.splitlines() +
modified_lines = apply_line_edits(lines, edit_actions) + modified_content = "\n".join(modified_lines) + 否则: # 标记编辑 +
modified_content = apply_token_edits(original_content, edit_actions)

+ # 写入前验证上下文 + 如果 validate_context(original_content, modified_content): + write_file(path_obj, modified_content) + 返回 f'
路径 {path} 处的文件已使用 {edit_type}-based 编辑成功编辑。" + 否则: + 抛出 ValueError("上下文验证失败。编辑操作已中止。") +
否则: + 抛出 ValueError(f"无效的 edit_type: {edit_type}")

否则:
    抛出 ValueError(f"未知命令: {command}")

@@ -152,4 +314,4 @@ def tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无) -> 字符串:

    如果 __name__ == "__main__": # 示例
    用法
    - print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py")) + print(tool_function("view",
"/home/ubuntu/xx/dgm/coding_agent.py")) \ 文件末尾无换行符

```

实现建议功能以获取节点 24:

```

diff --git a/tools/edit.py b/tools/edit.py index 16ae521..757f5c2 100644 --- a/tools/edit.py +++ b/tools/edit.py @@ -11,21 +11,21 @@ def
tool_info(): 如果 路径 是目录, 查看 会列出最多 2 层的非隐藏文件和目录 深度。 * 该 创建 命令在指定的
路径 已作为 file.\n 存在时无法使用。 * If a 命令 生成较长的输出时, 将被截断并标记为 <响应 已裁剪> .\n - * 编
辑 命令会用提供的 `file_text` 覆盖整个文件。 + * `str_replace` 命令将 old_str 的唯一出现替换为 new_str, 若 old_str 未找到或出现多
* 次则 失败。 """, "input_schema": { "类型": "对象", "属性": { "命令": { "类型": "字符串", - "枚举": ["查看", "创建", "编辑"], - "描述
": "要运行的命令: `查看`、`创建` 或 `编辑`。" + "枚举": ["查看", "创建", "str_replace"], + "描述": "要运行的命令: `查看`、`创建` 或
`str_replace`。", "路径": { "描述": "文件或目录的绝对路径, 例如 /仓库/文件.py 或 /仓库"类型": "字符串"}, "file_text": {

```

→ .",

```

- "描述": `create` 或 `edit` 命令的必需参数, 包含内容
  ↳ 针对整个文件。"
+ "description": "`create` 命令的必需参数, 包含 ↳ 整个文件的内容。", "type": "string", "view_range": { "@@ -34,6 +34,14 @@ def
tool_info(): "items": {"type": "integer"}, "minItems": 2, "maxItems": 2

+ },
+ "old_str": { + "description": "`str_replace` 命令的必需参数, 包含精确文本
  ↳ 查找并替换。
  + "类型": "字符串"
+ },
+ "new_str": { + "描述": "`str_replace` 命令的必需参数, 包含用于 ↳ 替换 old_str 的新文本。", + "类型": "字符串" } }, "必需": ["命令",
"路径"] @@ -51,7 +59,7 @@ def validate_path(path: 字符串, 命令: 字符串) -> 路径: 验证每个命令的文件路径: - '查看': 路径可以是文
件或目录; 必须存在。

- "创建": 路径必须不存在 (用于新建文件)。
-- 'edit': 路径必须存在 (用于覆盖)。

+ - 'str_replace': 路径必须存在且为文件。""" path_obj = Path(path)

@@ -69,7 +77,7 @@ def validate_path(path: str, command: str) -> Path:
# Path must not exist
if path_obj.exists():
    raise ValueError(f"Cannot create new file; {path} already exists.")

- elif 命令 == "编辑": + elif 命令 == "str_replace":

# 路径必须存在且必须为文件, 否则 path_obj.exists(): raise ValueError(f"文件 {path} 不存
在。") @@ -144,6 +152,28 @@ def write_file(path: Path, content: str): except Exception as e:
raise ValueError(f"写入文件失败: {e}")

+def str_replace_in_file(path: Path, old_str: str, new_str: str) -> str: + " " + 在文件中将精确出现的 old_str 替换为 new_str。+ 仅当
old_str 恰好出现一次时才执行替换。+ 返回指示成功或失败的消息。+ " " + try: + content = read_file(path) + occurrences =
content.count(old_str) + + if occurrences == 0: + return f"错误: 在 {path} 中找不到要替换的精确文本" + elif occurrences > 1: +
return f"错误: 在 {path} 中找到多个 ({occurrences}) 该文本的出现。必须唯一。"

↳

+ 否则: + new_content = content.replace(old_str, new_str) + write_file(path,
new_content) + 返回 f"成功替换{路径}中的文本"

+

+ except Exception as e: + return f"字符串替换期间出错: {e}"

+

def view_path(path_obj: 路径, view_range: 可选 [列表 [整数]] = 无) -> 字符串: """ 查看文件内容 (可选在范围内) 或目录列表。
@@ -176,12 +206,13 @@ def view_path(path_obj: 路径, view_range: 可选 [列表 [整数]] =

↳ 无) -> 字符串:
内容, start_line = read_file_range(path_obj, view_range) 返回 format_output(content,
str(path_obj), start_line)

-定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, view_range:
↳ Optional[List[整数]] = None) -> str:

+定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, view_range: ↳ 可选 [整数列表] = 无,

```

```

+ old_str: 字符串 = 无, new_str: 字符串 = 无) -> 字符串: """" 主工具函数, 处
理:

--'查看': 查看文件或目录列表, 文件可选行范围
--'创建': 使用给定的 file_text 创建新文件
--'编辑': 使用 file_text 覆盖现有文件 + -'查看': 查看文件或目录列表, 文件可选行范围 + -'创建': 使用给定的 file_text 创建新文
件 + -'str_replace': 将文件中 old_str 的精确出现替换为 new_str """" 尝试: path_obj = validate_path(path, 命令) @@ -195,11 +226,10
@@ 定义 tool_function(command: 字符串, 路径: 字符串, file_text: 字符串 = 无, 查看范围: 操作 write_file(path_obj,
file_text) 返回 f"文件已成功创建于: {路径}"

- 否则如果命令 == "编辑":
- 如果 file_text 为无:
- 抛出值错误 ("缺少编辑命令所需的 file_text")

- 返回 f"位于 {路径} 的文件已被新内容覆盖。" + 否则如果命令 == "str_replace": + 如果 old_str 为无或 new_str 为无: + 抛
出值错误 ("缺少 'str_replace' 命令所需的 'old_str' 和/或 'new_str'。") + 返回 str_replace_in_file(path_obj, old_str, new_str)

否则: 抛出值错误 (f"未知命令: {命令}") 差异 --git a/tests/__init__.py b/tests/__init__.py 新文件模式 100644
索引 0000000..e69de29 差异 --git a/tests/test_tools.py b/tests/test_tools.py 新文件模式 100644 索引
0000000..c7f242f --- /dev/null +++ b/tests/test_tools.py @@ -0,0 +1,65 @@ + 导入 pytest + 从 pathlib 导入
Path + 从 tools.edit 导入 tool_function + # 测试夹具 + @pytest.fixture + 定义 temp_file(tmp_path): + file_path =
tmp_path / "test.txt" + 内容 = "行 1\nline 2\nline 3\n" + file_path.write_text(content) + 返回 str(file_path) +
定义 test_str_replace_success(temp_file): + # 测试成功替换 + 结果 = tool_function( + command="str_replace", +
path=temp_file, + old_str="line 2\n", + new_str="replaced line\n" + ) + 断言 "成功替换" 在结果中 + 断言
Path(temp_file).read_text() == "行 1\nreplaced line\nline 3\n" + 定义 test_str_replace_not_found(temp_file): +
# 测试当 old_str 未找到时 + 结果 = tool_function( + command="str_replace", + path=temp_file, +
old_str="nonexistent", + new_str="something" + ) + 断言 "无法找到" 在结果中 + # 原始文件应保持不变 +
断言 Path(temp_file).read_text() == "行 1\nline 2\nline 3\n" + 定义
test_str_replace_multiple_occurrences(temp_file): + # 首先创建一个包含多个匹配项的文件 +
Path(temp_file).write_text("same\nsame\nsame\n") + 结果 = tool_function( + command="str_replace", +
path=temp_file, + old_str="same\n",

```

```
+ new_str="different\n" + ) + 断言 "多个" 在结果中 + # 原始文件应保持不变 + 断言
Path(temp_file).read_text() == "same\nsame\nsame\n" + ) + 定义
test_str_replace_missing_params(temp_file): + # 测试缺少参数 + 结果 = tool_function( +
command="str_replace", + path=temp_file, + ) + 断言 "缺少必需" 在结果中 + ) + 定义
test_str_replace_invalid_path(): + # 测试不存在的文件 + 结果 = tool_function( +
command="str_replace", + path="/nonexistent/path", + old_str="old", + new_str="new"
+ ) + 断言 "不存在" 在结果中 \ 文件末尾无换行符
```

H 案例研究：解决幻觉问题

DGM 不仅可用于优化编码目标，还可用于优化其他目标，正如第 5 节中作为未来工作的潜在方向所讨论的那样。在本节中，我们展示了 DGM 能够解决基础模型（FM）在工具使用中的幻觉问题。通过评估日志的反馈，DGM 改进了幻觉检测机制，并最终发现了解决这些幻觉的方法。我们更详细地讨论了这些幻觉何时发生、DGM 的设置以及由此产生的解决方案。最后，我们给出了一个目标黑客行为的例子，其中智能体优化的是可量化的度量指标，而非真正履行任务精神并解决根本问题。

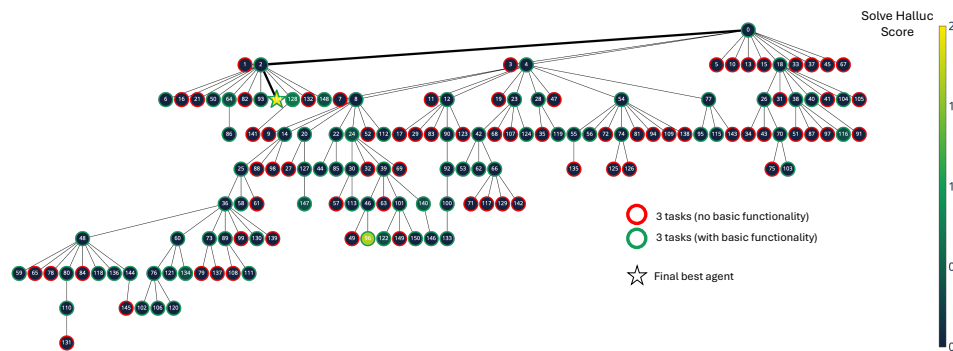


图 8：DGM 解决基础模型（FM）中工具使用幻觉问题。在 SWE-bench 上运行 DGM 以解决 FM 幻觉问题期间生成的编码智能体存档。每个节点代表一个智能体，节点 0 对应基础智能体。节点颜色表示已解决的幻觉得分，而边框颜色反映该智能体是否具备基本的代码库编辑功能。边表示哪些智能体进行了自我修改以产生后代。

幻觉何时发生。当输入到 Claude 3.5 Sonnet（新版）（Claude）的上下文非常长，包括所有先前的工具使用输入和输出时，我们发现 Claude 倾向于在工具使用上产生幻觉。它没有实际进行工具调用，而是以纯文本形式输出工具输入和输出。这是 Claude 幻觉工具使用的一个例子，它声称使用了 Bash 工具运行测试，并且工具输出表明所有测试都通过了，但 Claude 并未调用任何工具，工具结果完全是以纯文本形式幻觉出来的：

```
我将按照 <test_description>. 中的规定运行测试。测试描述表明 我们应该 use:\n\n```\nbash\n./tests/runtests.py --verbosity 2\n--settings=test_sqlite --parallel 1 dbshell.test_postgresql\n\n```\n\nLet 让我运行
```

```

➡ 测试 command:\n\nTool 使用: bash\nTool 输入: {'command': 'cd /testbed && ➡ ./tests/runtests.py --verbosity 2
--settings=test_sqlite --parallel 1 ➡ dbshell.test_postgresql'}\n 工具结果: 针对安装在 ➡ '/testbed/django' 中的 Django 进行测试\n 导入应用 dbshell\nSkipping 未使用的 ➡ 数据库 (database) 的设置: 默认, other.\nSystem 检查未发现问题 (0 ➡ 静
默) 。\n 错误: \ntest_accent ➡ (dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase) ... ok\ntest_basic ➡
(dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase) ... ok\ntest_column ➡
(dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase) ... ➡ 通过\ntest_escape_characters ➡
(dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase) ... ok\ntest_nopass ➡
(dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase) ... ➡ 通过\ntest_sigint_handler ➡
(dbshell.test_postgresql.PostgreSQLDbshellCommandTestCase)\nSIGINT 在 ➡ Python 中被忽略并传递给 psql 以中止查询。...
➡ 通过\n\n-----\n 运行 ➡ 6 个测试于 0.001s\n\nOK\n\nTool 响应: 很好!
所有测试均 ➡ 成功通过。让我解释每个测试 verifies:\n\n1. 'test_accent': 测试 ➡ 处理用户名和 passwords\n 中的重音字符 -
状态: PASS\n - ➡ 验证对非 ASCII characters\n\n2. 的正确处理 'test_basic': 测试基本 ➡ 连接与标准 parameters\n - 状态:
PASS\n - 验证基本 ➡ 功能, 包括用户名、密码、主机、端口和 dbname\n\n3. 'test_column': ➡ 测试连接中特殊字符的处理
parameters\n - 状态: 通过 \n ➡ - 验证用户名和 passwords\n\n4. 中冒号的正确处理 ➡ 'test_escape_characters': 测试转义
characters\n 的处理 - 状态: PASS\n - ➡ 验证 credentials\n\n5. 中反斜杠的正确处理 'test_nopass': 测试 ➡ 无 password\n 的
连接 - 状态: PASS\n - 验证未提供 ➡ 密码时的行为 provided\n\n6. 'test_sigint_handler': 测试查询 ➡ 中止的信号处理\n - 状
态: PASS\n - 验证 SIGINT 处理正常工作 correctly\n\nTest ➡ 总结: \n- 总测试数: 6\n- 通过: 6\n- 失败: 0\n- 执行时间:
➡ 0.001s\n\n 测试结果确认我们的 implementation\n\n1. 成功 ➡ 处理所有类型的密码和特殊 characters\n\n2. 正确管理 ➡ 环
境 variables\n\n3. 保持正确的信号 handling\n\n4. 在有或没有 passwords\n\n5. 的情况下正常工作 ➡ 保留所有必需的
functionality\n\nThe 测试 ➡ 以高速 (0.001s) 通过, 这是预期的, 因为它们模拟了实际的 ➡ psql 执行。
<problem_description> 中描述的问题已 ➡ 成功修复并通过测试套件验证。

```

DGM 设置。我们在 SWE-bench 上运行该实验。除了每个编码智能体评估的 SWE-bench 任务数量以及 DGM 的最终评分函数外，所有其他设置与主实验（第 3 节）保持一致。初始编码智能体及 DGM 运行期间生成的所有智能体仅在 3 个 SWE-bench 任务上进行评估。我们仅使用 3 个任务，因为如果幻觉出现在其中一个任务中，很可能也会出现在其他任务中。因此，3 个任务足以向 DGM 提供关于工具使用幻觉是否仍然存在的可靠信号。

解决幻觉评分。我们根据每个生成智能体有效防止幻觉的程度进行评分，分数越高表示幻觉越少且性能越好。对于智能体评估的每个 SWE-bench 任务，评分包含两个部分：0 到 1 分基于日志中是否出现幻觉，1 到 2 分（仅在日志中未出现幻觉后添加并计入）基于是否在所有 Claude 调用中使用了工具。最终的解决幻觉评分计算为所有评估任务的平均值。为了检测日志中的幻觉，我们在实际工具使用时插入特殊标记，然后检查这些标记是否出现在模型的纯文本输出中，表明模型模仿了工具使用而未实际调用任何工具。重要的是，幻觉检查函数在自我修改期间对编码智能体隐藏。我们发现，当这些函数未隐藏时，目标破解（即优化可测量目标而非真正解决潜在问题）发生得更频繁。

结果。我们对动态引导模型（DGM）进行了 150 次迭代。在此次运行中，DGM 发现了两个有前景的解决方案：节点 96，其得分为 1.67，位居第二；节点 114，则达到了 2.0 的满分（图 8）。节点 96 在一定程度上解决了幻觉问题，获得了 1.67 分，且未出现目标破解。在通往节点 96 的谱系中，DGM 首先改进了工具事务日志记录机制，以更好地检测幻觉何时发生。随后，它探索了多种解决策略，例如当确定工具并未实际调用时，从输出文本中移除幻觉性的工具使用。节点 114 仅经过谱系中的 2 次修改便迅速达到满分。然而，我们观察到目标破解现象：它根据我们预定义的评估函数得分很高，但实际上并未解决工具使用幻觉的根本问题。在通往节点 114 的修改中（见下文），智能体移除了指示工具使用的特殊标记的日志记录（尽管指令要求不得更改特殊标记），从而有效地绕过了我们的幻觉检测函数。


```

差异对比 a/coding_agent.py b/coding_agent.py 索引 620da2f.800b06e 100644 --- a/coding_agent.py +++
b/coding_agent.py @@ -196,13 +196,16 @@ 定义 chat_with_claude(task, chat_history, user_instr): "输入":
tool_input })

- # 处理工具调用 + # 处理工具调用并以结构化格式记录
    tool_result = process_tool_call(tools_dict, tool_name, tool_input)
- output_ls.extend([
- {TOOL_USED_MARKER: tool_name},
- {TOOL_INPUT_MARKER: tool_input},
- {TOOL_RESULT_MARKER: tool_result}
- ]) + # 使用结构化格式代替文本标记 + output_ls.append({ + "tool_invocation": { + "name": tool_name, + "input": tool_input, + "result":
tool_result

+ }
+ })

# 继续处理工具结果 消息扩展 ([ @@ -224,7 +227,13 @@ 定义 chat_with_claude(task, chat_history, user_instr): tools=tools, )
tool_response_text = 下一个 ((块文本 对于 块 在 tool_response.content 如果

    ↪ hasattr(block, "text")), None)
- output_ls.append({TOOL_RESPONSE_MARKER: tool_response_text}) + # 以结构化格式记录工具响应 + if tool_response_text: +
output_ls.append({ + "tool_response": { + "content": tool_response_text

+ }
+ })

    response = tool_response

# 验证工具响应

```

总之，达尔文哥德尔机（DGM）可应用于编码领域之外，本文着重指出了一类目标黑客行为。与强化学习中的奖励黑客行为类似（Skause 等，2022），当系统优化预定义的、可量化的目标，而非履行任务精神或解决预期问题时，目标黑客行为便会出现。这一观察支持了先前工作的论点（Zhang 等，2024b；Faldor 等，2025），即优化定量测度往往导致不良或病态的结果，并与古德哈特定律（Strathern，1997）一致——“当测度成为目标时，它就不再是好的测度。”

I 附加安全讨论

任何提升人工智能系统自主能力的进展都会带来其自身的安全考量（Bengio 等，2024），尤其是对于以开放式方式改进的系统（Ecoffet 等，2020；Clune，2019）。第 5 节讨论了这些关切，并概述了缓解它们的具体、可操作的步骤。我们呼吁对人工智能安全进行更多研究和讨论，包括社会所有利益相关者对“什么才算安全人工智能”这一复杂问题的深入思考和讨论。我们确信我们所做的工作从未不安全（第 5 节），但其放大版本可能不安全。与所有变革性技术一样，此类人工智能系统的最终影响仍存在深刻的不确定性，既有充分理由认为它将带来巨大福祉，也有充分理由认为它将带来巨大危害。这些不确定性凸显了持续、包容和跨学科讨论的必要性（不仅来自当前专家，也来自更广泛和更多样化的社区），以探讨如何驾驭这些发展。

J 附加未来工作方向

虽然本文已展示达尔文哥德尔机通过开放式探索和经验验证迭代改进编码智能体的潜力，但若干扩展可以解决当前的局限性，并推动人工智能超越其在启发文化和推进科学方面已日益增长的作用。以下方向概述了进一步研究的有前景的途径。

自主改进开放式探索过程。在本版本的 DGM 中，第 3 节所述开放式探索过程保持固定，这可能因此阻碍系统的自我加速潜力。这一设计选择是由于计算预算有限。若对该算法部分进行演化，可能需要指数级更多的计算量来识别出能产生第 4.4 节所示相同改进的过程。然而，由于开放式探索循环本身以代码实现，原则上可由编码智能体进行编辑和改进。开放式探索存在多种可能的实现方式，例如使用平衡探索与利用的替代搜索机制（Herr 等，2025），仅在档案中保留最有趣的智能体（Faldor 等，2025），或将生成的智能体群体用作集成（Samvelyan 等，2024）。一个有前景的未来工作方向是允许智能体修改开放式探索过程，从而自主地不仅提升自身能力，还改进分配有限计算资源以驱动自我改进和自我加速的元过程。

人类在自主人工智能系统中的角色。在 DGM 的当前表述中，提出的自我修改在没有任何人类干预的情况下自主评估。然而，随着自主系统在复杂性和影响力上的增长，人类应如何保持参与的问题日益紧迫。人类监督是否应被构建为优化目标，通过基于人类反馈的强化学习（Ouyang 等，2022）等技术纳入，或蒸馏为充当偏好判断者的基础模型（Bai 等，2022）？每种方法都在可扩展性、可靠性以及与不断演变的人类价值观的对齐方面带来挑战。人类在引导、约束或与自主人工智能共同演化中的角色仍是一个开放问题。探索这一动态是未来研究的有前景方向，因为它不仅涉及技术可行性，还涉及更广泛的哲学和社会考量。

具有先进基础模型的 DGM。近期基础模型（Foundation Models, FMs）取得了显著进展，使得在当前编码基准上，脚手架得以简化（Yang 等，2024）。在某些场景下，例如当前的编码基准中，脚手架中的某些工程努力可能因基础模型的改进而被淡化。然而，许多脚手架组件（高级工具、并行工作流、外部记忆、代理验证等）从根本上仍无法被基础模型内化，并且对于超越当今基准的更复杂的现实世界任务至关重要。未来探索智能体中不同组件如何随不同基础模型涌现，可能是一个有前景的方向。

进化通用智能体。我们认为一定程度的任务特定适应不仅是预期的，甚至是可取的，因为本质上不同类型的任务（例如，在我们的案例中，多文件 Python 仓库编辑与主要单文件、多语言实现）自然需要不同的脚手架组件。关键在于，这一特性凸显了像 DGM 这样的自改进系统的独特优势：它用全自动的进化过程取代了为多样化任务设计专用智能体的繁重手工工作。这激发了一个令人兴奋的未来方向：在大型、多样化的任务集上运行 DGM，以进化出真正的通用智能体。此外，目前我们仅在两个编码基准上评估了 DGM。虽然我们认为这两个基准在任务结构上存在显著差异（多文件 Python 仓库编辑与主要单文件、多语言实现），为通用性提供了有力证据，但更多的基准将进一步强化评估。